

Custom Compiler™ and IC Compiler™ Co-Design Assistant User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	8
Related Products, Publications, and Trademarks	8
Conventions	8
Customer Support	9
Statement on Inclusivity and Diversity	10

1. About Analog and Digital Co-designing	11
Analog and Digital Co-Designing in the Custom Compiler Environment	11
Software User Interface	12
Library Manager	12
Schematic Editor and Layout Editor	13
Create FRAM	15
Generating FRAM Views	15
Main Tab (Required Information)	17
Layer Blockage Tab	19
PR Boundary Tab (Optional)	22
Export Options Tab	24
Script Tab	25
Create FRAM Operation	25
Validate for ICC	26
Liberty Data Editor	27
Opening the Liberty Data Editor Tool	27
Viewing and Adjusting Liberty Data Parameters	28
Editing Extracted Data Using the Liberty Data Editor Tool	29
Edit Liberty Data Dialog Box Reference	31
Main Tab	31
Extracted Data Tab	33

2. Getting Started	34
Workflows for Custom Compiler and IC Compiler Co-designing	34
Workflow for Preroute Editing	34
Workflow for Developing and Integrating Digital Blocks	35

Starting Custom Compiler Sessions and Opening Designs	35
Enabling the Co-design Assistant	36
Custom Compiler Products and Tools	37
3. Importing and Exporting Designs	39
Importing Design Data from the IC Compiler Tool	39
Loading Custom Display Resources	39
Importing Libraries	40
IC Compiler - Custom Compiler Library Mapping Files	44
Creating Target Libraries for IC Compiler Import	46
Importing Individual Designs from the IC Compiler Tool	48
Importing Designs Containing Colored Shapes	48
Importing Design Error Data from the IC Compiler Tool	49
Exporting Designs and Libraries to the IC Compiler Tool	51
About the Export Process	52
Exporting Libraries to the IC Compiler Tool	52
Creating IC Compiler Libraries for Export	57
Exporting Individual Designs to the IC Compiler Tool	59
Monitoring the Import and Export Progress	59
4. Designing	62
Opening a Design From the Library Manager	62
View Mapping between IC Compiler and Custom Compiler	63
Working on Schematic Designs	64
Working on Layout Designs	64
Closing Designs	64
5. Preparing Libraries for Co-Design	65
Designs Originating in the Custom Compiler Environment	65
Designs Originating with the IC Compiler Tool	66
Creating Timing Libraries for Multi-Voltage Power in Place and Route	68
6. Editing IC Compiler Designs in the Custom Compiler Environment	70
Custom Compiler Tools Used in IC Compiler Co-design Flows	70

Contents

View Nets, Instances, and Terminals	71
Set Design-Specific Constraints	71
Adjust Components	71
Create Interconnect	71
Create Routes	72
Check Metal Density	73
Check EM and Check Resistance	73
Implementing Analog Nets	73
Safe Editing	74
Namespace Mapping	74
Object Mapping	74
Adjusting Placement of Cells	76
Editing Route Guides	78
Creating Routes Automatically	81
Creating Routes Interactively	82
Adding Shielding	84
Performing IC Compiler Operations on Design Layouts	84
Running the IC Compiler Tool Using Tcl Scripts for Input	85
Running the IC Compiler Tool Using Console Commands	85
Setting IC Compiler Shortcut Keys for the Custom Compiler Environment	86
Use Model Example	89
Creating Analog Pre-Routes Targeting a Specific Resistance Value	89
Cleaning Up the Layout after Routing	90
Working with Routing Objects in an ICC-Custom Compiler Flow	90
Paths	91
Rectangles	92
 7. Digital Timing Shells for Analog Blocks	 93
Timing Shells	94
Creating Timing Shells for Analog Blocks	95
Preparing to Create a Timing Shell	95
Creating Connection Definitions for the Timing Shell	97
Defining the IC Compiler Black Box	97
 8. Developing Digital Blocks with Co-design	 102
Adding Floorplanning Conditions for Digital Blocks	102

Contents

Implementing Digital Blocks	103
Running IC Compiler P & R from Design Schematics or Verilog Netlists	103
Tracking Progress and Viewing Output Files for IC Compiler Operations	107
Designing with Place and Route Results	108
Customizing the IC Compiler Place and Route Command File	109
Workflow for Customizing the Command File	109
Tcl Variables to Use in the Command File	110
9. Tutorial: Constraint Rules Mapping	112
Before You Start	113
Task 1: Analyze the Design Using the IC Compiler Tool	113
Task 2: Add IC Compiler Libraries to the Custom Compiler Environment	116
Task 3: Open the Design Layout	117
Task 4: Review NDRs with the Custom Compiler Layout Editor	117
Task 5: Edit and Create NDRs and Shield Constraints	120
Task 6: Evaluate Edits with the IC Compiler Tool	128
Task 7: Edit Routing Rules in the IC Compiler Tool	132
Summary	133
Limitations	134
A. Setting the User Environment	135
Technology Mappings	135
DataType Mapping	135
Display Attribute Mapping	137
Export to the IC Compiler Tool	138
Scenario 1: Without Available IC Compiler Technology or Design Data	139
Generate an IC Compiler Technology File	139
Generate IC Compiler Tech File and Library	139
Scenario 2: To an Existing IC Compiler Library with Matching Layers	141
Scenario 3: To an Existing IC Compiler Library With Different Layer Names	142
Scenario 4: To an Existing IC Compiler Library with No Matching Layers	145
Using the Automated Method of Exporting to IC Compiler	146
Using the Layer Mapping File	147
Import From the IC Compiler Tool	150
Scenario 1: Without Available Technology or Design Data	150

Contents

Generate a Custom Compiler Technology File	150
Generate a Custom Compiler Technology File and Library	151
Scenario 2: Into an Existing Library With Matching Layers	152
Scenario 3: Into an Existing Library With Different Layer Names	153
Scenario 4: Into an Existing Library With No Matching Layers	155
Using the Automated Import From the IC Compiler Tool	156
Using the Layer Mapping File	157
Scenario 5: Into an Existing Library With Read-Only Technology	159
Pre-Route Flow	162
Creating Guard Ring Definitions for an IC Compiler Design	163
Using Separate OA Technology for an Opened IC Compiler Library	164
Setup Using LEF Files	165
Scenario 1: Importing LEF Files to Create an IC Compiler Library	165
Scenario 2: Importing LEF files to create a Custom Compiler library	166
B. Basic Troubleshooting	168
Error Messages	168
E MWBRIDGE-002	169
E MWBRIDGE-011	169
E MWBRIDGE-050	169
E MWBRIDGE-051	170
E MWBRIDGE-060	170
E MWBRIDGE-101	170
E MWBRIDGE-171	171
E MWBRIDGE-209	172
E MWBRIDGE-225	172
E MWBRIDGE-226	173
E MWBRIDGE-247	174
E MWBRIDGE-306	175
E MWBRIDGE-369	175
E MWBRIDGE-371	175
E MWBRIDGE-397	176
Warning Messages	177
W MWBRIDGE-419	177

About This User Guide

This user guide describes how to use the Custom Compiler™ co-design assistants for analog and digital co-designing. The co-designing process involves iterative steps between the IC Compiler™ tool and the Custom Compiler environment.

This user guide assumes that you are familiar with the Custom Compiler and IC Compiler tools.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys IC Compiler™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

About Analog and Digital Co-designing

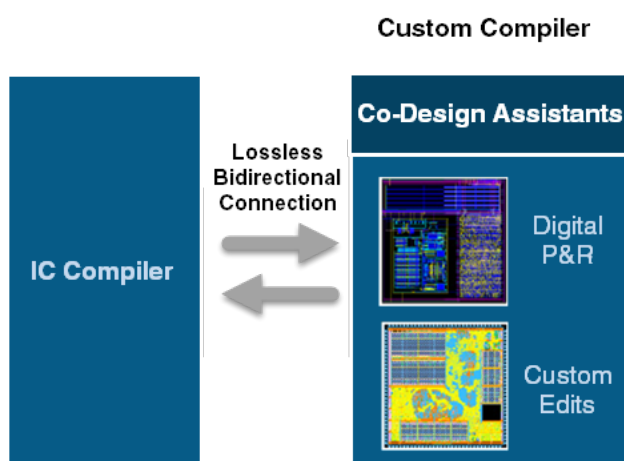
This chapter provides a general overview of digital and analog co-designing using Custom Compiler co-design assistants. This chapter also provides an overview of the menu commands that you would use for co-designing on the Custom Compiler environment.

This chapter describes the following topics:

- [Analog and Digital Co-Designing in the Custom Compiler Environment](#)
- [Software User Interface](#)
- [Create FRAM](#)
- [Validate for ICC](#)
- [Liberty Data Editor](#)

Analog and Digital Co-Designing in the Custom Compiler Environment

You can use the Custom Compiler Co-Design Assistants for digital and analog co-design.



In the Custom Compiler environment, you can:

- Use Custom Compiler Co-Designing Assistants to design analog nets in a digital context, creating pre-routes before place and route.
- Run place and route in IC Compiler within the Custom Compiler environment.
- Open the IC Compiler design in the Custom Compiler environment at any time during floorplanning, placement, or power and ground route creation and clock-tree synthesis.
- Create FRAM views of the design.
- Connect to IC Compiler data, such as DRC data, and view them in the Custom Compiler environment.
- Perform IC Compiler operations directly in the Custom Compiler environment.
- View Custom Compiler design data in the IC Compiler tool.

For instructions on how to create designs in IC Compiler, see the [IC Compiler user documentation](#) available on SolvNet.

For information about the co-design features available in the Custom Compiler environment, see [Software User Interface](#).

Software User Interface

This topic describes the following:

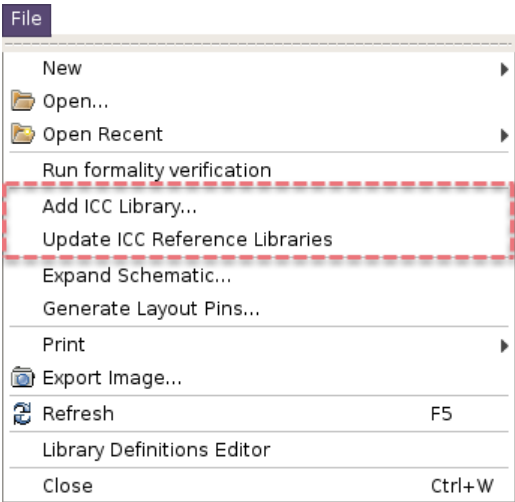
- [Library Manager](#)
- [Schematic Editor and Layout Editor](#)

Library Manager

Before designing analog blocks or implementing digital blocks, you must use the Custom Compiler Library Manager to create or add the IC Compiler libraries in the Custom Compiler environment. The criteria for whether you create or add the IC Compiler libraries are described in [Preparing Libraries for Co-Design](#).

After you create or add the IC Compiler libraries in the Custom Compiler environment, you can use the Library Manager to view information about the designs. You can also open and manage libraries, cells, and views for the design process. For detailed information, see the *Custom Compiler Environment User Guide*.

The following Library Manager menu commands are available for co-designing processes:



Command	Description
File > Add ICC Library	Opens existing IC Compiler libraries in the Custom Compiler environment. Tcl command: <code>dm::showAddICC2Library</code>
File > Update ICC Reference Libraries	Updates the associated reference libraries.
Run Formality Verification	Checks for modifications introduced in the design during development.

Other Library Manager commands are also available, such as Copy, Move, and Update References. You use these commands for general library operations.

Schematic Editor and Layout Editor

This section lists the commands in the **ICC** menu. The list of commands in the menu can differ, depending on whether you have an ICC or OA design open, and whether you are viewing the design in the Schematic Editor or the Layout Editor.

Note:

To display the co-design menu and commands, you must first [enable the co-design assistant](#).



Table 1 ICC Menu Commands

Command	Available in	Description
Create FRAM	OA and ICC designs	See Create FRAM .
Place & Route	OA design only	See Running IC Compiler P & R from Design Schematics or Verilog Netlists .
Validate for ICC	OA design only	See Validate for ICC .
Export	OA and ICC designs	See Exporting Designs and Libraries to the IC Compiler Tool .
Save and Export	OA design only	See Exporting Designs and Libraries to the IC Compiler Tool .
Import	OA and ICC designs	See Importing Design Data from the IC Compiler Tool .
Revert to ICC	OA design only	Choose ICC > Revert to ICC to replace the edit design with a copy imported from IC Compiler. ICC > Revert to ICC opens the Import from ICC dialog box with the Input region of the dialog box set to the specified design and disabled. See Importing Libraries .
Run ICC	OA and ICC designs	See Performing IC Compiler Operations on Design Layouts for more information.
Open in ICC	ICC design only	Choose ICC > Open in ICC to perform IC Compiler operations in the Custom Compiler environment.

Table 1 ICC Menu Commands (Continued)

Command	Available in	Description
Import DRC Violations	ICC design only	See Importing Design Error Data from the IC Compiler Tool .
Liberty Data Editor	OA design only	See Liberty Data Editor .

Create FRAM

This topic describes the following:

- [Generating FRAM Views](#)
- [Main Tab \(Required Information\)](#)
- [Layer Blockage Tab](#)
- [PR Boundary Tab \(Optional\)](#)
- [Export Options Tab](#)
- [Script Tab](#)
- [Create FRAM Operation](#)

Generating FRAM Views

This section describes how to generate ICC FRAM views using the **Create ICC FRAM** dialog box. To use `xt::CreateICCFRAM Tcl` command to create ICC FRAM views, see the *Custom Compiler Tcl Reference Manual*.

To generate ICC FRAM views:

1. Choose **ICC > Create FRAM**.

The **Create ICC FRAM** dialog box opens.

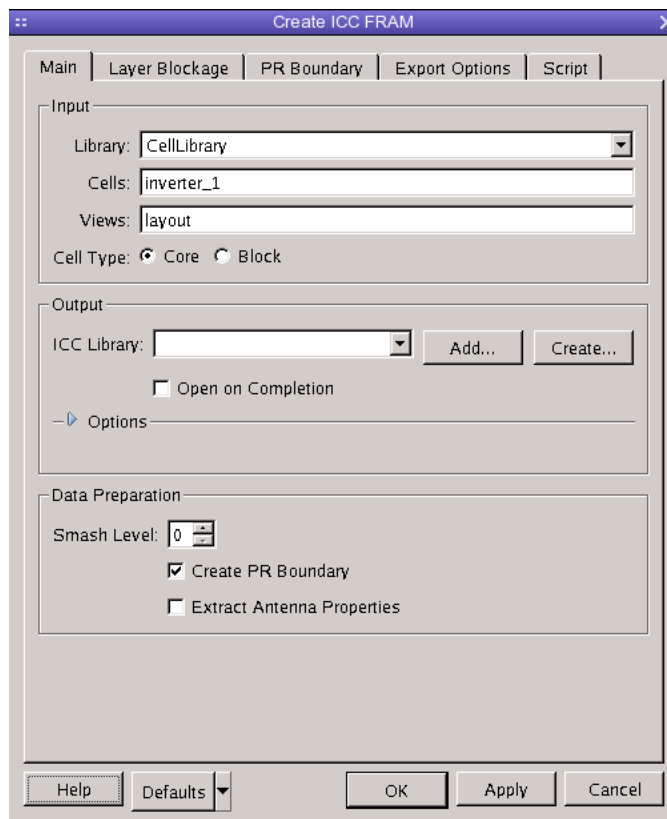
Note:

A different version of the dialog opens depending on the library type, OA or ICC.

2. Specify the input and layer blockage parameters in each of the following tabs:
 - [Main Tab \(Required Information\)](#)
 - [Layer Blockage Tab](#)

- [PR Boundary Tab \(Optional\)](#)
 - [Export Options Tab](#)
 - [Script Tab](#)
3. Click **OK** or **Apply** to start the operation.

Main Tab (Required Information)



Use the following procedure to specify the required inputs on the **Main** tab of the dialog box.

1. In the **Input** area, specify the source cellview for which to create a FRAM view.

When first opened, the tool populates the dialog box with information about the edit cellview in the active Layout Editor tab.

2. In the **Output** area, specify the target library to store the FRAM views by doing one of the following:

- If you do not have an existing target library, accept the default Library:

`<workspace>/icclibs/<source_library_name>`

where:

`<workspace>` is your current design directory.

`source_library_name` is the same name as the Top Library name in the Input region of the dialog box.

- If you have an existing target library, browse to select it.

Note:

If the input cellviews are in an ICC library, the **Output** area contains only the option to open the FRAM view in ICC when the operation finishes.

(Optional) Specify whether to open the FRAM view in ICC when the operation finishes.

(Optional) Specify the following additional options:

- Whether to overwrite any existing cells
- Whether to create an embedded netlist
- Whether to create layers
- Whether to remaster the instances from CEL to FRAM

3. (Optional) In the **Data Preparation** region of the dialog box, you can specify the following:

- **Smash Level**

This option lets you control processing access to geometries inside the hierarchy of a layout cell. You can set a level that either allows or excludes access.

- **Create PR Boundary**

When selected, the **PR Boundary tab** and options are available. See [PR Boundary Tab \(Optional\)](#).

Note:

This field is only available when **Cell Type: Core** is selected in the **Input** region of the dialog box.

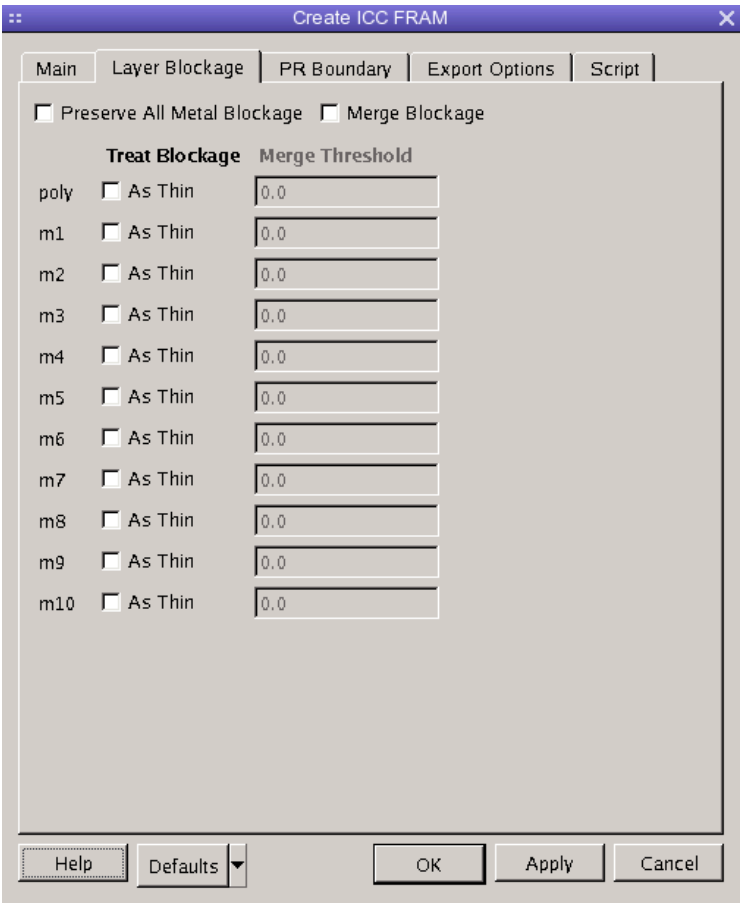
- **Extract Antenna Properties**

Note:

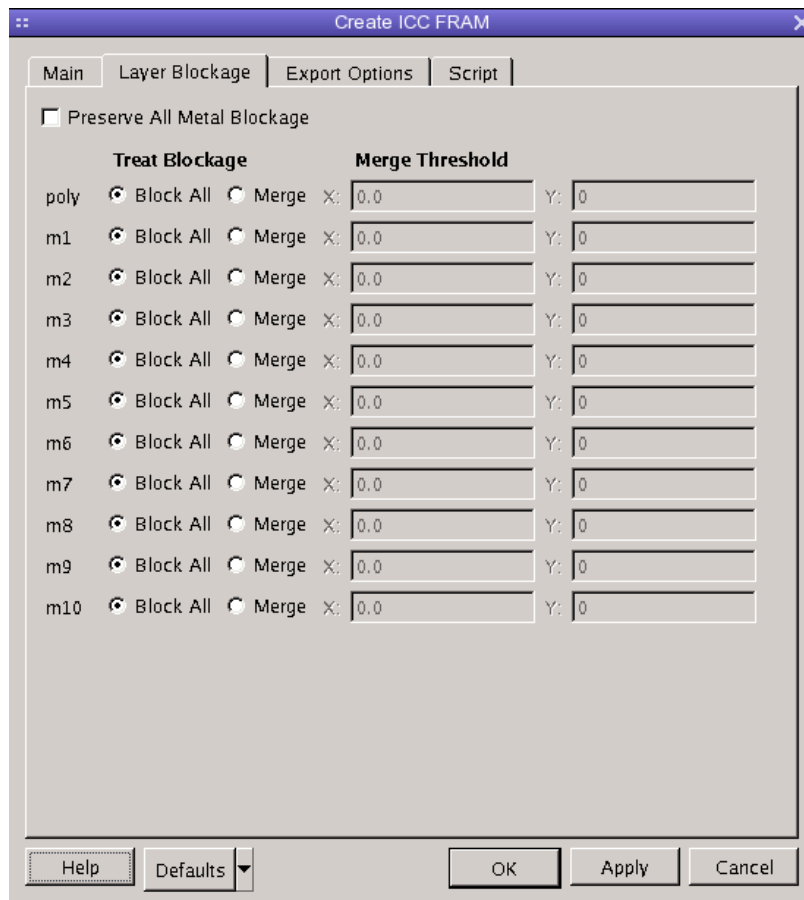
The Hercules or IC Validator executable is required to use the **Extract Antenna Properties** option.

Layer Blockage Tab

The **Layer Blockage** tab shows all the poly and metal layers in the current technology. The tab content depends the input library selection. The following figure shows the **Layer Blockage** tab of the dialog box for **Cell Type: Core**.



The following figure shows the **Layer Blockage** tab of the dialog box for **Cell Type: Block**.



There are two methods for specifying the layer blockages:

- Enable the **Preserve All Metal Blockages** option.

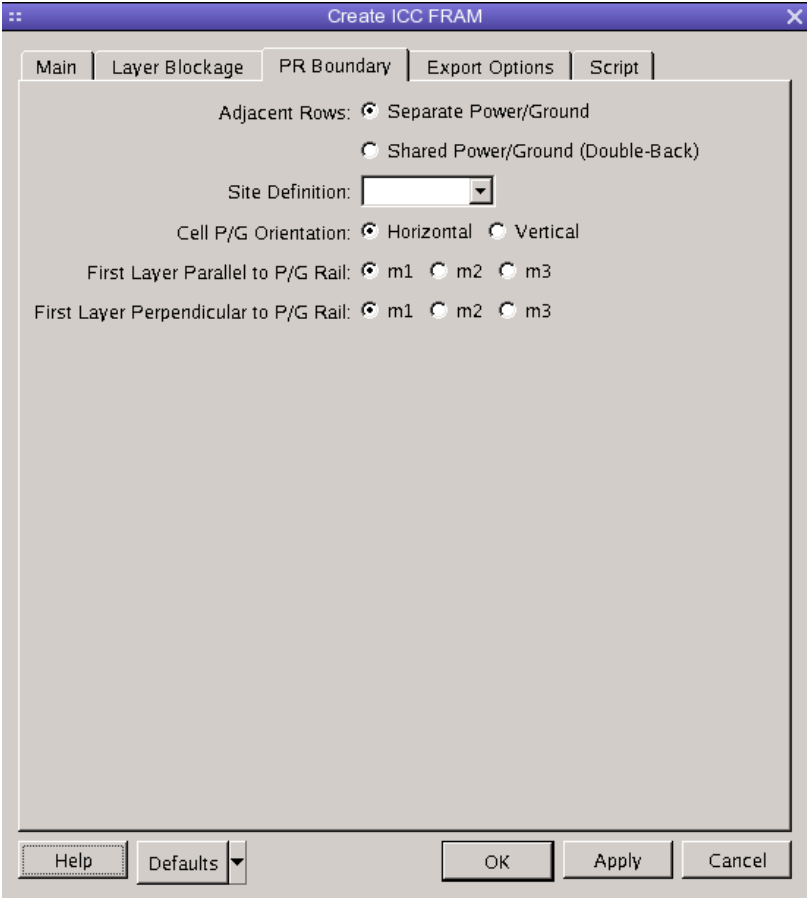
When selected, all metal blockages retain their original shapes. They will not be trimmed or cut to permit pin access.

- Manually configure individual layers.

For each layer, you can specify the following:

Setting	Description
Treat Blockage	Specifies how to treat blockage layers: ◦ As Thin: Treats the blockage layer as thin. Thin wires are wires/blockages that have the minWidth value. By default, real metal blockages are treated as fat wire if fat or thin wire if thin. It causes real metal blockages to be treated as thin wire for design rule checking purposes. ◦ Block All: Creates a blockage for the specified layer. Select this option when you want the FRAM view to contain few areas for over-the-cell routing. ◦ Merge: Merges nearby shapes on the specific layer according to the threshold values you specify.
Merge Threshold	Specifies how the blockage shapes are merged depending on the Cell Type set in the Main tab: ◦ <i>Core Cell Type</i>: Merges threshold with a single value. ◦ <i>Block Cell Type</i>: Specifies the X and Y directions for merging nearby shapes. When the X and Y values are both 0, the original blockage shape is retained. If you specify a large value, nearby shapes will merge into a larger blockage.

PR Boundary Tab (Optional)



The options on the **PR Boundary** tab are available only when you enable the **Create PR Boundary** option on the **Main** tab.

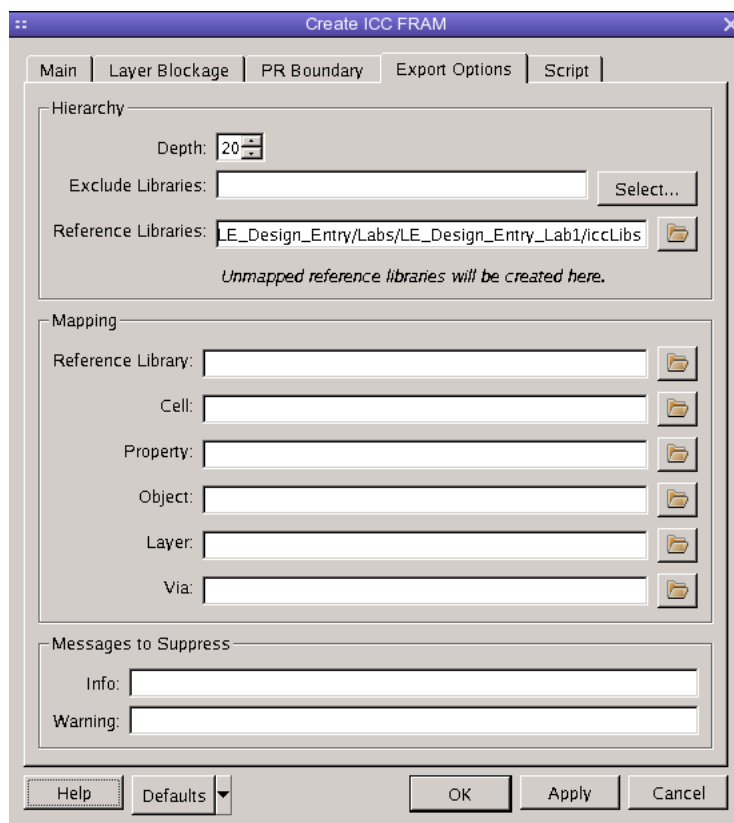
Note:

This field is only available when **Cell Type: Core** is selected in the **Input** region on the **Main** tab.

The following table describes the options. For more information, see the *Milkyway Environment* online help.

PR Boundary Option	Description
Adjacent Rows	Sets the height of the PR boundary based on a cell placement: • <i>Separate</i>: The cells do not overlap. • <i>Shared</i>: The cell rows are paired, with one row in each pair flipped and the cells placed so that the bottom power and ground rails of the flipped cells overlap with the top power and ground rail of the unflipped cells.
Site Definition	A list of the sites (unit tiles) where you can place cells in a row.
Cell P/G Orientation	Specify the orientation — horizontal or vertical — for the power and ground rails.
First Level Parallel to P/G Rail	Select the first metal layer that you want to be parallel to the power and ground rail.
First Level Perpendicular to P/G Rail	Select the first metal layer that you want to be perpendicular to the power and ground rail.

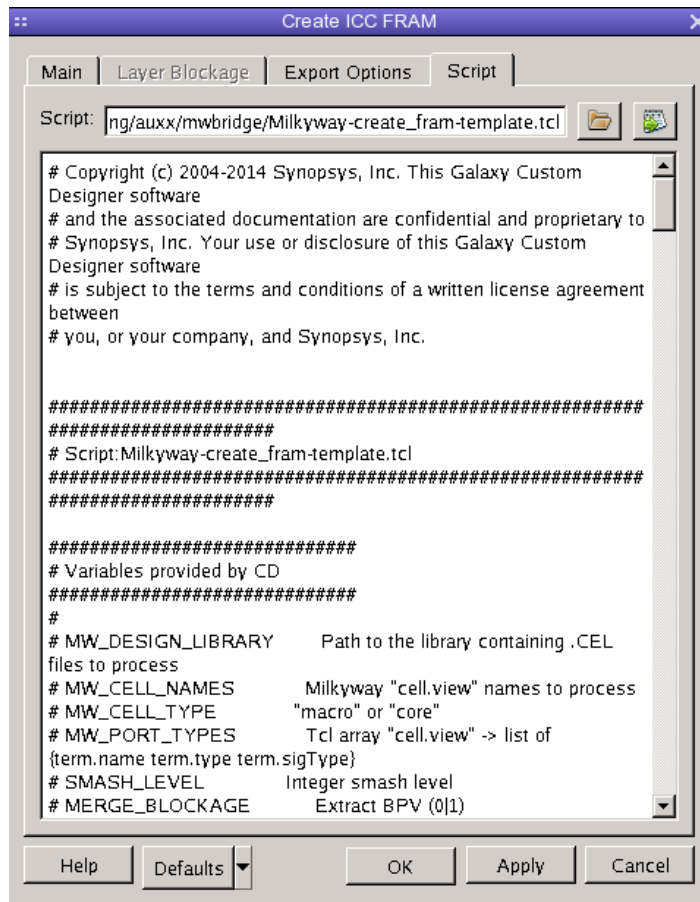
Export Options Tab



The **Export Options** tab specifies the options used for export.

- In the **Hierarchy** region of the dialog box, specify the hierarchy **Depth**, **Exclude Libraries**, and the **Reference Libraries** location where unmapped reference libraries are created. If the depth is set to 0, the libraries options are disabled.
- In the **Mapping** region of the dialog box, specify optional mapping files for **Reference Library**, **Cell**, **Property**, **Object**, **Layer**, and **Via**.
- In the **Messages to Suppress** region of the dialog box, specify the **Info** and **Warning** messages to suppress entered as space-delimited lists of message IDs as reported by the translator.

Script Tab



Note:

The **Script** tab displays a template script only. You can copy this template and add your own code to customize the way FRAM views are created. The tab does not generate a script based on input in the dialog box.

Create FRAM Operation

Prior to performing the FRAM creation operation, the tool performs the following checks to ensure that the external applications can be found in your `PATH`:

- The Milkyway executable

If this executable cannot be found, the **Locate Milkyway Executable** dialog box prompts you to specify the path to the executable. After entering a valid path,

clicking **OK** adds the specified path to your `PATH` variable. Clicking **Cancel** aborts the operation.

- The `icc_shell` executable is required if the **Open on Completion** option is enabled and the `leCreateFRAMViewer` preference is set to ICC.
- The Hercules or IC Validator executable is required to use the **Extract Antenna Properties** option.

When you click **OK** or **Apply** in the **Create ICC FRAM** dialog box, the bridge:

1. Creates a script that performs the FRAM view creation.
2. Saves the current state to the `prefs.xml` file within the `iccbridge` directory.
3. Exports the specified designs to ICC.

Cells are created according to the output specified in the **Create ICC FRAM** dialog box.

4. Generates SMASH and FRAM views for each converted design.
5. (Optional) Opens all resulting FRAM views with ICC.

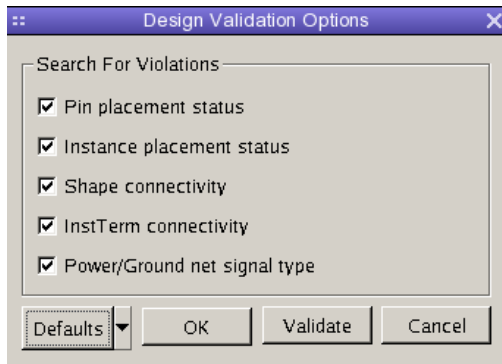
Validate for ICC

Many design requirements need to be added in the Custom Compiler environment to export a block as an IC Compiler-ready CEL using **Export To ICC** with **Create Embedded Netlist** enabled. For this purpose, **Validate for ICC** checks whether the design to be exported satisfies some minimum conditions, as follows:

- All the pins should have a placement status.
- All the shapes should have connectivity.
- All the instances' `instTerms` have connectivity.
- All the instances within the boundary have a placement status.
- All the instances outside the boundary does not require a placement status.
- Correct `sigType` for power/ground nets.

If there are any errors, they will be marked by error markers. Use the Marker Browser assistant to browse and fix the errors.

Choose **ICC > Validate for ICC** to open the **Design Validation Options** dialog.



Choose to search for the following design violations:

- **Pin placement status**
- **Instance placement status**
- **Shape connectivity**
- **InstTerm connectivity**
- **Power/Ground net signal type**

Liberty Data Editor

You can use the Liberty Data Editor to view and define settings and values for a Liberty library file. The tool generates the file as specified. To review the file, open it in a text editor.

To export the Liberty Library file after it has been generated, see [Exporting Design Data in Liberty Library File Format](#).

These sections cover:

- [Opening the Liberty Data Editor Tool](#)
- [Viewing and Adjusting Liberty Data Parameters](#)
- [Editing Extracted Data Using the Liberty Data Editor Tool](#)

Opening the Liberty Data Editor Tool

To open the Liberty Data Editor, be sure ICC is activated. Then:

- In any design window, choose **ICC > Show Liberty Data Editor**.

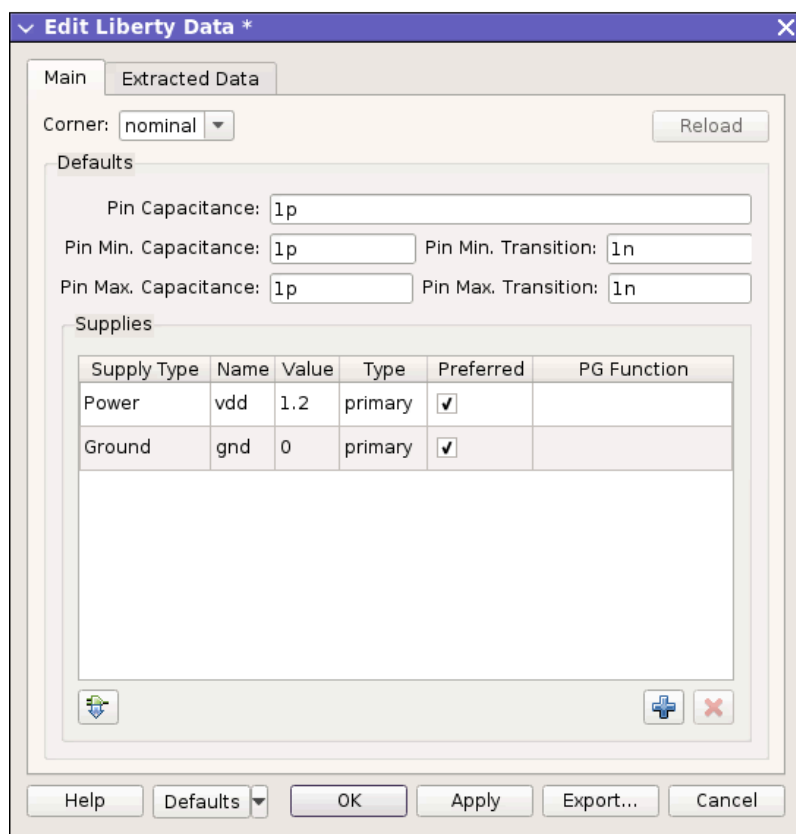
See next

[Viewing and Adjusting Liberty Data Parameters](#)

More about using the [Liberty Data Editor](#)

Viewing and Adjusting Liberty Data Parameters

You can view and adjust settings in the **Main** tab of the **Edit Liberty Data** dialog box.

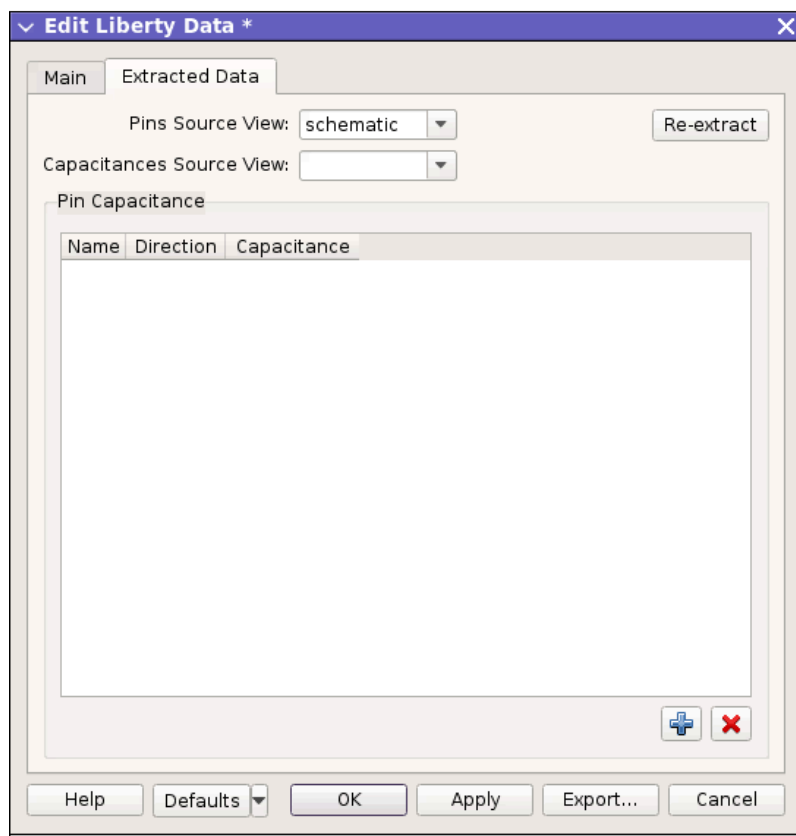


To view and adjust settings:

1. Begin by [Opening the Liberty Data Editor Tool](#).
2. In the **Edit Liberty Data** dialog box, the **Corner** drop-down menu shows corners specified in the current design. This does not allow you to specify a corner name. If there is no corner in the current design, the corner "nominal" is created automatically. Its data is extracted from the pins of the Pins Source View and from the capacitance information of the Capacitances Source View in the **Extracted Data** tab. Multiple corners can be created for the same design using the command `db::copyLibertyData`.

3. Click the **Reload** button to reload the corner data to revert changes. This is only enabled when there is stored corner data in the design.
4. Examine the **Defaults** section, which contains default pin capacitance (used when capacitance extraction was not successful) and the **Supplies** table. The loaded (or extracted) Liberty data is shown in the **Supplies** table and in the **Pin Capacitance** table of the **Extracted Data** tab. The default pin capacitance is also used when **Extracted Data: Capacitances Source View** is empty, or when there is no capacitance information in this source for the given pin.
5. Notice the **Supplies** table section contains a button for copying supply pins  and buttons for adding and deleting supplies, **+** and **X**. **Copy Supply Pins** optionally copies power supply pins from a cellview you specify in the **Copy Supply Pins** dialog box, along with **View Search List** and **View Stop List**.
6. Click **OK**, **Apply**, and **Export**. The **Export** button opens the **Export Liberty File** dialog box (described in Exporting Design Data in Liberty Library File Format).

Editing Extracted Data Using the Liberty Data Editor Tool



The **Extracted Data** tab shows the pins and capacitances source view names, as well as the **Pin Capacitance** data table.

- **Pins Source View** is the current view name by default.
- **Capacitance Source View** is empty by default.
- The **Re-extract** button re-extracts the Liberty data from the desired pins and/or capacitances' source cell. To extract pins only, set the capacitance cell value to empty. To extract capacitances only, set the pins cell value to empty. To extract both pins and capacitances, set the corresponding table cell values to the desired cell views. After re-extracting, the current data in the **Pins Capacitance** table is replaced.
- The **Pin Capacitance** table is populated with Liberty data extracted from these cell views if the corner is created automatically. If the corner pre-exists, the values of pins and capacitances source views are set from the stored liberty information associated with the corner.

The pin names in the **Pin Capacitance** table are editable. You can also add a new pin using the **+** button, or delete a selected pin using the **X** button.

To show/hide columns in the **Pin Capacitance** table, right-click in the table header to access the **Show/Hide Columns** dialog box. The following columns are available:

- **Name**
- **Direction**
- **Capacitance**
- **Min. Capacitance**
- **Max. Capacitance**
- **Min. Transition**
- **Max. Transition**
- **Related Power**
- **Related Value**
- **Function**

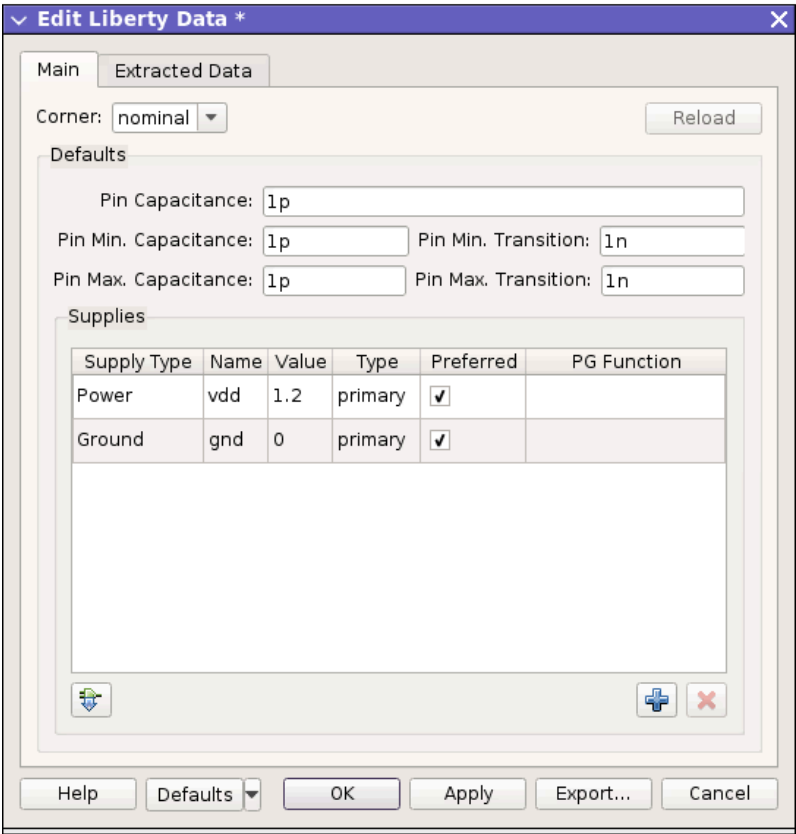
Double-click on the **+** button to the left of a pin name in the **Extracted Data** tab to open the **Bus Bits** tab.

Edit Liberty Data Dialog Box Reference

The Edit Liberty Data dialog box consists of the following tabs:

- [Main Tab](#)
- [Extracted Data Tab](#)

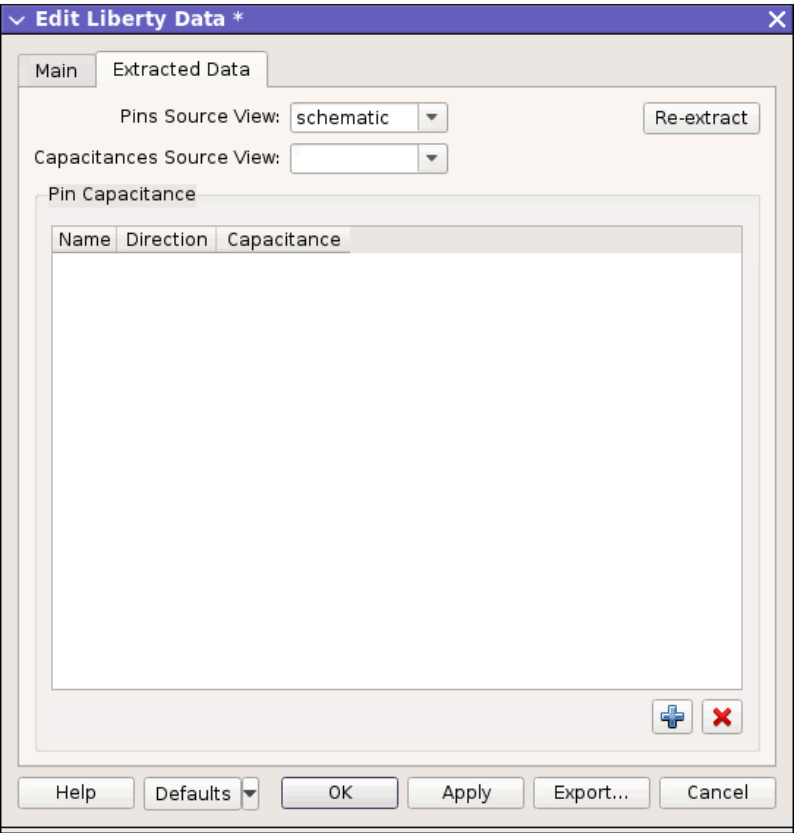
Main Tab



Option	Description
Corner	Characterization corner. Default: nominal
Defaults	
Pin Capacitance	Capacitance of the pin.
Pin Min. Capacitance	Minimum capacitance of the pin.
Pin Min. Transition	Minimum transition of the output pin.

Option	Description
Pin Max Capacitance	Maximum capacitance of the pin.
Pin Max Transition	Maximum transition of the output pin.
Supplies	
Supply Type	Type of power supply: power and ground.
Name	Name of the net.
Value	Power voltage value.
Type	Type of power supply: primary or backup.
Preferred	Indicator that the supply is preferred.
PG Function	Description of how the power and ground is used.
	Opens the Copy Supply Pins dialog box and lets you specify the cellview from which to copy the pins.
	Adds a new power supply to the Supplies table.
	Deletes the selected power supply from the Supplies table.

Extracted Data Tab



Option	Description
Pins Source View	View of the pin source.
Capacitance Source View	View of the capacitance source.
Re-extract	Reruns extraction.
Pin Capacitance	
Name	Net name of the pin.
Direction	Direction of signal.
Capacitance	Capacitance value.

2

Getting Started

To start using the co-design features as quickly as possible, browse the information and resources provided in this topic.

This chapter describes the following topics:

- [Workflows for Custom Compiler and IC Compiler Co-designing](#)
- [Starting Custom Compiler Sessions and Opening Designs](#)
- [Enabling the Co-design Assistant](#)
- [Custom Compiler Products and Tools](#)

Workflows for Custom Compiler and IC Compiler Co-designing

This topic describes the following:

- [Workflow for Preroute Editing](#)
- [Workflow for Developing and Integrating Digital Blocks](#)

Workflow for Preroute Editing

One method of co-designing is to edit an IC Compiler design through roundtrips between the IC Compiler and Custom Compiler environment, as the following diagram shows.

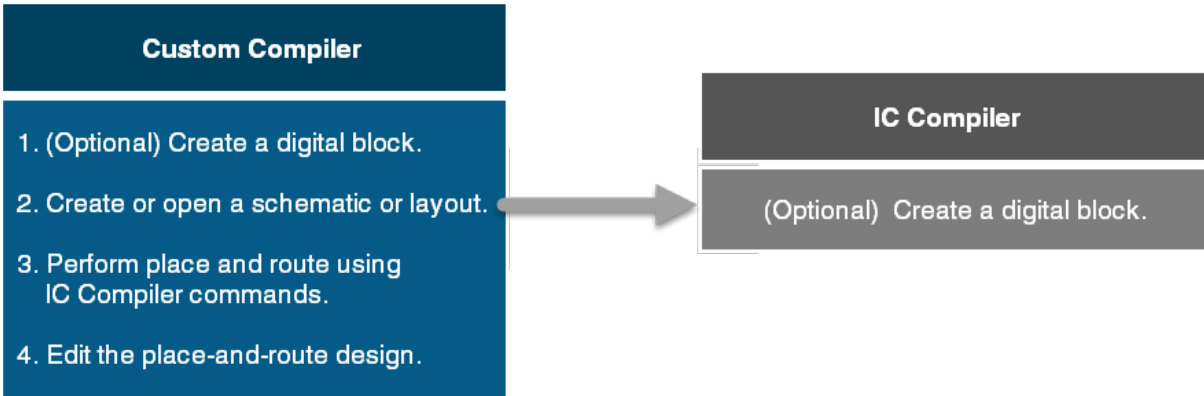


Use this method if you have a predominantly digital design and you want to:

- Add analog components, such as preroutes.
- Make custom edits.

Workflow for Developing and Integrating Digital Blocks

Another method of co-designing is to stay completely or mostly within the Custom Compiler environment throughout the co-design process, as the following diagram shows.



Use this method if you have an existing Custom Compiler mixed-signal design and you want to do one or both of the following:

- Develop a small digital block.
- Implement the digital block in the design. This task includes placing and routing the block, and then integrating it in the mixed-signal design.

You have the option of using a Verilog file, design schematic, or design layout as input for the place-and-route process.

Starting Custom Compiler Sessions and Opening Designs

To start a Custom Compiler session and to open designs within the Custom Compiler environment, see the following:

Task	See...
Start a Custom Compiler session.	Starting Sessions in the <i>Custom Compiler Environment User Guide</i>

Task	See...
Display the Library Manager.	Displaying the Library Manager in the <i>Custom Compiler Environment User Guide</i>
Open a design schematic.	Opening Designs in the <i>Custom Compiler Environment User Guide</i>
Open a design layout.	Opening Designs in the <i>Custom Compiler Environment User Guide</i>

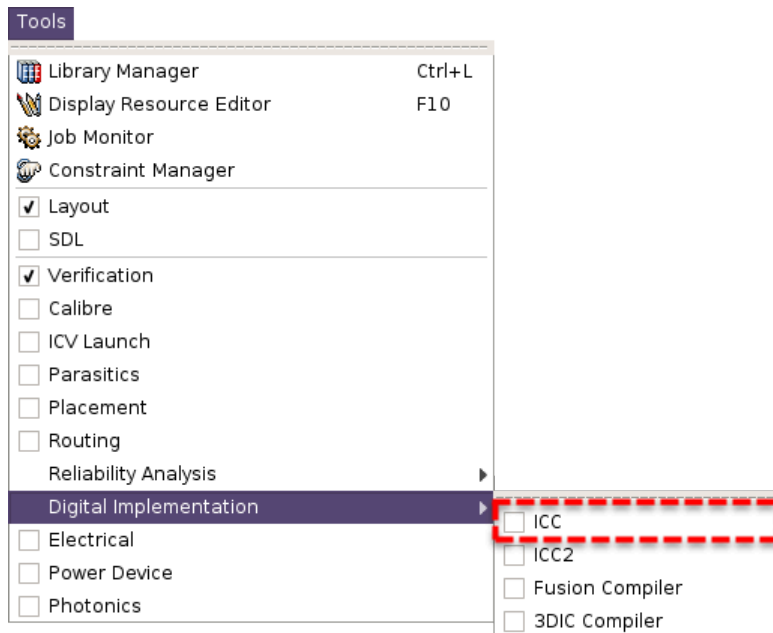
Enabling the Co-design Assistant

After you open a design in the Custom Compiler environment, enable the co-design assistant as follows:

1. In the Custom Compiler Console, set the following preference:

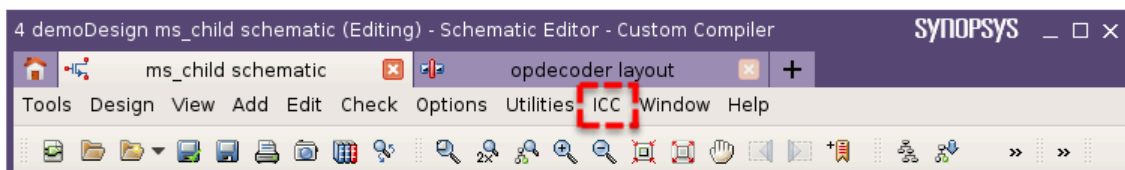
```
db::setPrefValue xtDigitalImplementationTool -value icc
```

2. In the Schematic Editor or the Layout Editor, choose **Tools > Digital Implementation > ICC**.

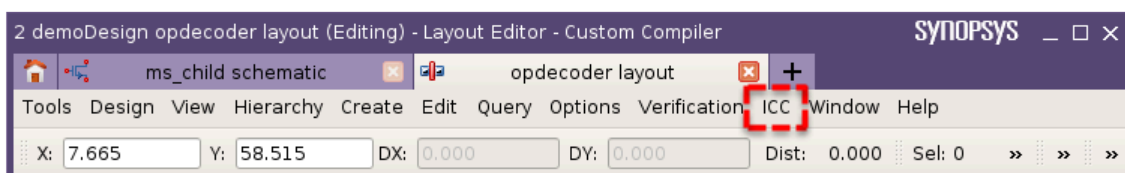


The **ICC** menu appears on the design editor menu bar.

Schematic Editor



Layout Editor



See [Software User Interface](#) for descriptions of the co-design menu commands and dialog boxes.

Custom Compiler Products and Tools

The Custom Compiler environment offers a variety of products and tools you can use when creating and editing mixed-signal designs. The following table lists some of the typical tasks and the Custom Compiler products and features you might use when co-designing. For the comprehensive list of available products and tools, see Design Phases and Products to Use in the *Custom Compiler Environment User Guide*.

Task	Custom Compiler Product or Tool
View nets, instances, terminals	Design Navigator Assistant
Set design-specific constraints	Constraint Editor Assistant
View and adjust design component properties, such as route guides	Property Editor Assistant
Add routes	Custom Router (automatically or interactively) Layout Editor (manually)
Add shielding	Adding Post-Route Shielding and Shield Tab
Check and fix design errors	DRD tool Via Checker Metal Density Checker Color Decomposition Automation

Task	Custom Compiler Product or Tool
Run electrical reports and checks	Electrical Reporter, Shield Coverage Reporter
Physical verification	Synopsys IC Validator Synopsys Hercules Mentor Graphics Calibre and Calibre Interactive StarRC CustomSim SPRES Viewer CustomSim Reliability Analysis feature

3

Importing and Exporting Designs

This chapter describes how to import and export design libraries and individual designs from the IC Compiler tool.

This chapter describes the following:

- [Importing Design Data from the IC Compiler Tool](#)
- [Exporting Designs and Libraries to the IC Compiler Tool](#)
- [Monitoring the Import and Export Progress](#)

Importing Design Data from the IC Compiler Tool

This topic describes the following:

- [Loading Custom Display Resources](#)
- [Importing Libraries](#)
- [Importing Individual Designs from the IC Compiler Tool](#)
- [Importing Designs Containing Colored Shapes](#)
- [Importing Design Error Data from the IC Compiler Tool](#)

Loading Custom Display Resources

When you open an IC Compiler design in the Layout Editor, the tool automatically creates a default `display.tcl` file, stores it in the `iccbbridge` folder, and loads it with the IC Compiler design. The `display.tcl` file contains the default display resource data for the design.

Instead of using the default `display.tcl` file that the Layout Editor creates, you can load your own `display.tcl`. To do this:

1. Store your `display.tcl` file in the Custom Compiler startup folder.
2. In the Custom Compiler Console, set the `xtICCLoadDisplayResources` Tcl preference to `false`:

```
db::setPrefValue xtICCLoadDisplayResources -value false
```

3. Open the IC Compiler design in the Layout Editor.

For information about the display resources, see the Display Resource Editor Tool topics in the *Custom Compiler Environment User Guide*.

For information about the `xtICCLoadDisplayResources` Tcl preference, see the *Custom Compiler Tcl Reference Manual*.

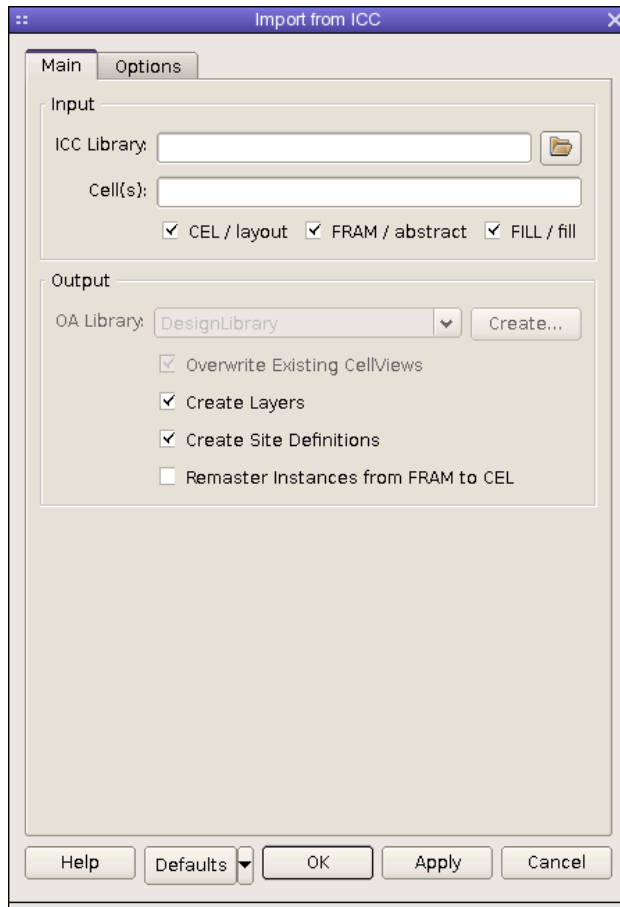
Importing Libraries

Import libraries from the IC Compiler tool when you want to design using both the Custom Compiler tool and the IC Compiler tool.

When you import IC Compiler design data, a bridge process between the two tools takes the source library containing the data you want to import, converts it for Custom Compiler tool access, and adds it to a target library for your use.

To import a design library from the IC Compiler tool:

1. (Optional) If there is no designated Custom Compiler target library to receive imported data, begin by [Creating Target Libraries for IC Compiler Import](#).
2. From the Console, choose **File > Import > From ICC**. The **Import from ICC** dialog box opens.



3. In the **Main** tab, specify required input details:
 - a. In the **ICC Library** field, enter the path to a valid IC Compiler library. This is the source file to be converted.
 - b. Enter a space-separated list of cells. If empty, all cells are converted.
 - c. Select the views to be converted: **CEL/layout**, **FRAM/abstract**, **FILL/fill**.
 - d. In the **Output** area, from the **OA Library** menu, select a target library to receive converted data.

The menu displays the libraries listed in your `lib.defs` file, if the file exists. If the menu is empty, or if you want to create a new library, click **Create** and follow the procedure for [Creating Target Libraries for IC Compiler Import](#).
 - e. Use the check boxes to choose whether or not to **Overwrite Existing CellViews**, **Create Layers**, or **Create Site Definitions**.

Check **Create Layers** to import shapes on layers not referenced by the technology. If unset, these shapes will be skipped.

Check **Create Site Definitions** to create site definitions in the target technology during data translation.

- f. (Optional) To have the detailed layout in the Custom Compiler environment, specify **Remaster Instances from FRAM to CEL** to change (remaster) the instance reference from IC Compiler FRAM view to IC Compiler CEL view during the import operation.

4. In the **Options** tab, set the following:

The screenshot shows the 'Import from ICC' dialog box with the 'Options' tab selected. The dialog has a title bar with a close button (X) and a maximize button (double arrow). The 'Options' tab is active, showing several sections:

- Hierarchy**: A 'Depth' field set to 20 with up/down arrows, and an 'Exclude Libraries' field with a 'Select...' button.
- Reference Libraries**: A 'Library Path' field containing 'mentation_Lab1/synopsys_custom/oaLibs' and a folder icon button.
- Mapping**: Five fields for 'Reference Library', 'Cell', 'Property', 'Object', and 'Layer', each with a folder icon button. Below these is a checkbox labeled 'Don't import unmapped layers' which is unchecked. At the bottom of this section is a 'Via' field with a folder icon button.
- Messages to Suppress**: Two fields, 'Info' and 'Warning', for suppressing specific messages.

At the bottom of the dialog are five buttons: 'Help', 'Defaults' (with a dropdown arrow), 'OK', 'Apply', and 'Cancel'.

Setting	Description
Hierarchy	Depth limits the amount of design information exported. Exclude Libraries if depth is greater than 0, you can choose to exclude libraries you enter here from the export. Caution: If the OA reference library and Milkyway reference library have the same name, the import process will overwrite the target Milkyway or OA reference library if the master cells from the library are imported. To prevent the overwrite, use the Exclude Libraries option to exclude the libraries that have the same name during the import process.
Reference Libraries	Library Path browse to the directory where you want unmapped reference libraries to be created. You can choose whether or not to Inherit reference libraries from main library.
Mapping	Adding mapping information makes it possible to include instances in your design project that reference masters in external libraries. To instantiate an instance that references an external library, you can map an external library to the library that will be used in the target project. The system looks for your user-provided mapping information when you start an import or export operation. If your user-provided information is found, the system writes that information to the following mapping files in the iccbridge directory: layer.map, via.map, lib.map, cell.map, property.map, and object.map. Reference Library maps using the external reference <code>lib.map</code> file you browse to select. Cell maps using the <code>cell.map</code> file you browse to select. Property maps using the <code>property.map</code> file you browse to select. Object maps using the <code>object.map</code> file you browse to select. Layer maps using the <code>layer.map</code> file you browse to select. Select Don't import unmapped layers if you want to exclude unmapped layers during the import process. Via maps using the <code>via.map</code> file you browse to select.
Messages to Suppress	Info Is a space-separated list of Information message IDs you want the tool to suppress Warning Is a space-separated list of Warning message IDs you want the tool to suppress

5. Click **OK**.

The import process creates the iccbridge run directory at the path `$SYNOPSIS_CUSTOM_LOCAL/iccbridge` and saves the current state to `prefs.xml` within the `iccbridge` directory.

If the source library is new, the import process also optionally creates a technology file and a display resources file. It creates the Custom Compiler library and imports technology from either the input IC Compiler library or an existing user-specified Custom Compiler technology file.

The process completes by optionally loading display resources from the IC Compiler design.

6. Add access to this imported library by updating the library definition file through the Library Manager.

For details about object mappings, see [IC Compiler - Custom Compiler Library Mapping Files](#).

For instructions for creating target libraries, see [Creating Target Libraries for IC Compiler Import](#).

IC Compiler - Custom Compiler Library Mapping Files

For reference while importing or exporting libraries, use this section to view mapping files for IC Compiler libraries and Custom Compiler libraries.

See the tables that follow for details about the library import and export files.

[Table 2](#) lists cell map details.

Table 2 *cell.map*

Custom Compiler	Custom Compiler	Custom Compiler	IC Compiler	IC Compiler	IC Compiler
library name	cell name	view name	library name	cell name	view name
string	string	string	string	string	string

[Table 3](#) lists layer map details.

Table 3 *layer.map*

Custom Compiler	Custom Compiler	IC Compiler	IC Compiler	Custom Compiler	Custom Compiler	IC Compiler
layer name ¹	purpose name ¹	layer number ¹	dataType number ¹	material name	mask number	mask name
string	string	int	int	enum ²	int	string

Table 4 lists library map details.

Table 4 *lib.map*

Custom Compiler	IC Compiler
library name	<path>/library name
string	string

Table 5 lists object map details for these objects: oaAreaBlockage, oaAreaHalo, oaLayerBlockage, oaLayerHalo, oaAreaBoundary, oaClusterBoundary, oaPRBoundary, oaSnapBoundary.

Table 5 *object.map*

Custom Compiler	Custom Compiler	IC Compiler	IC Compiler
object type	layer	layer number	layer name
enum	string	int	string

Table 6 lists property mapping details.

Table 6 *property.map*

Custom Compiler	IC Compiler
property name	property name
string	string

Table 7 lists via mapping details.

1. Required
2. The possible values are: nWell, pWell, nDiff, pDiff, nImplant, pImplant, poly, cut, metal, contactlessMetal, diffusion, recognition, other. Use the option other when the material for the layer is not specified.

Table 7 *via.map*

Custom Compiler	IC Compiler
viaDef name	contactCode name
string	string

For instructions for creating target libraries, see [Creating Target Libraries for IC Compiler Import](#).

For a description of the mapping files that can be provided when using an attached OA Tech library, see [Using Separate OA Technology for an Opened IC Compiler Library](#).

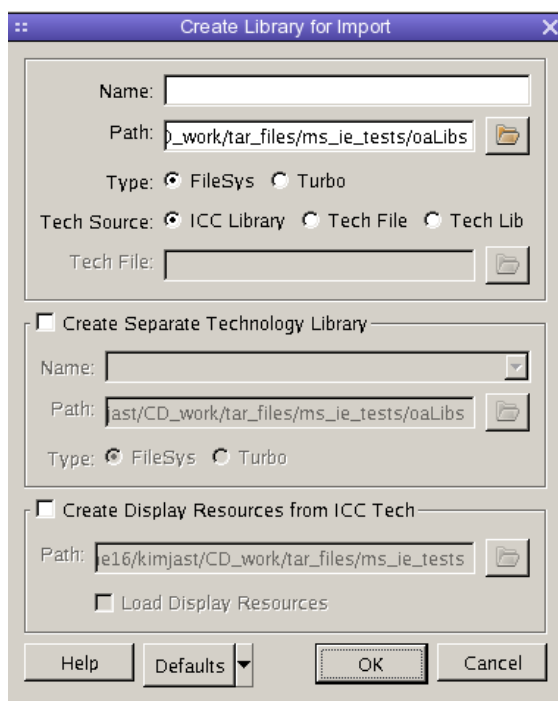
Creating Target Libraries for IC Compiler Import

If you have not created a target library to receive imported data from the IC Compiler tool, use the following procedure to create the library.

When the IC Compiler tool to Custom Compiler tool import operation completes, the system writes the converted data to a Custom Compiler library (the target library), overwriting any existing information.

To create a target library for IC Compiler import:

1. Choose **File > Import > From ICC** from the console to open the **Import from ICC** dialog box.
2. In the **Import from ICC** dialog box, click **Create**. The **Create Library for Import** dialog box opens.



3. Specify a library **Name**.

The system creates this library during the import operation.

4. Specify a **Path** to the library.

5. Accept the default **Type** (FileSys).

6. Select a **Tech Source** for the technology database for the new library.

- Selecting the **ICC Library** (default) option creates a Custom Compiler technology database based on the technology information contained in the IC Compiler library.
- If you select the **Tech File** option, also specify the path to the technology file.
- If you select the **Tech Lib** option, also choose an existing library from the **Tech Library** menu.

7. (Optional) If you want to create a second library to contain the imported technology, enable the **Create Separate Technology Library**, and specify the required information.

8. (Optional) If you want the system to create a Display Resources file (.drf or .tcl file), enable the option to **Create Display Resources from ICC Tech**, and specify the path name.

9. (Optional) If you want the Custom Compiler tool to load the display resources file after the import operation, enable the **Load Display Resources** option.
10. Click **OK**.

The system creates the library, the dialog box closes, and the new library name now appears in the Output region of the **Import from ICC** dialog box.

After you create the target library, you can continue the process of [Importing Design Data from the IC Compiler Tool](#).

For details about object mappings, see [IC Compiler - Custom Compiler Library Mapping Files](#).

For instructions for creating target libraries, see [Creating Target Libraries for IC Compiler Import](#).

Importing Individual Designs from the IC Compiler Tool

You can import individual designs from the IC Compiler tool after you set up the import process by performing it once. Follow the procedure for [Importing Design Data from the IC Compiler Tool](#).

After you import a design once, you can re-import it to discard edits and revert the design to its last imported state.

To import an individual design:

1. In any design window, choose **Tools > ICC Link** to add the ICC menu choice to the design menu.
2. Choose **ICC > Import** to open the **Import from ICC** dialog.

Importing Designs Containing Colored Shapes

The Custom Compiler environment supports IC Compiler designs that contain colored shapes for multiple patterning. To work with colored IC Compiler designs in the Custom Compiler environment:

- Make sure the IC Compiler technology that defines the color masks is translated to Custom Compiler technology. Use the `icc2cd_tech` executable. For more information and an example, see [Import From the IC Compiler Tool](#).
- Make sure you import the colored IC Compiler design using the **ICC > Import** command. The command uses a bridge process to correctly translate the design data. For more information about the import procedure, see [Importing Libraries](#) and [Importing Individual Designs from the IC Compiler Tool](#).

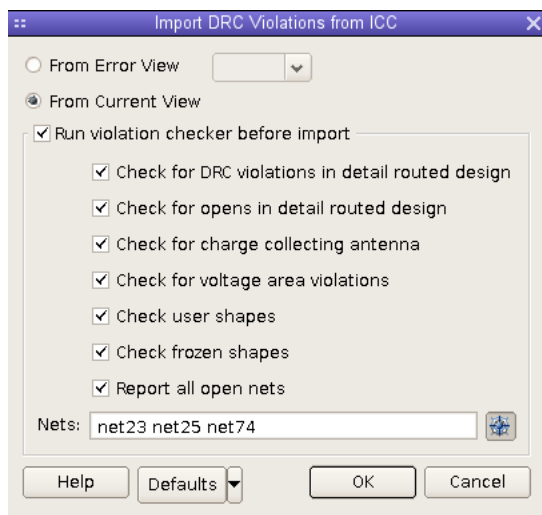
Importing Design Error Data from the IC Compiler Tool

You can use the **Import DRC Violations from ICC** dialog box to:

- Import IC Compiler DRC data as OpenAccess DRC markers. Importing the IC Compiler DRC data lets you resolve layout errors without having to leave the Custom Compiler environment. You can use the Autofix option to locate and fix the violations. Alternatively, you can manually edit the layout to correct the errors. For information about the DRD feature or instructions on how to edit the design, see the *Custom Compiler Design Checking and Physical Verification User Guide*.
- Run the IC Compiler `verify_zrt_route` command before the import process. You can select this option in the dialog box if the IC Compiler DRC data does not exist, or if you want to update the existing DRC data.

To import DRC data from the IC Compiler tool:

1. In the Layout Editor, choose **ICC > Import DRC Violations**. The **Import DRC Violations from ICC** dialog box opens.



2. Select one of the following:

Import Selection	Description
From Error View	Imports the DRC data from the IC Compiler error view. If you select From Error View , you must also choose the error view (error file) to import.
From Current View	Imports the DRC data from the IC Compiler layout view. If you select From Current View , you have the option of running the IC Compiler <code>verify_zrt_route</code> command. See step 3.

3. (Available only if you select **From Current View**) If you want to run the IC Compiler `verify_zrt_route` command before importing the DRC data, select **Run violation checker before import**, and then select one or more of the following:

DRC Checking Options	Description
Check for DRC violations in detail routed design	Checks for DRC violations of all nets. This option is equivalent to specifying the <code>verify_zrt_route -drc</code> argument.
Check for opens in detail routed design	Checks for open nets in the design. This option is equivalent to specifying the <code>verify_zrt_route -open_net</code> argument.
Check for charge collecting antenna	Checks for charge-collecting antenna rule violations on the nets. This option is equivalent to specifying the <code>verify_zrt_route -antenna</code> argument.
Check for voltage area violations	Checks for voltage-area rule violations on the nets. This option is equivalent to specifying the <code>verify_zrt_route -voltage_area</code> argument.
Check for user shapes	Checks for DRC violations that are caused by user-created shapes. This option is equivalent to specifying the <code>verify_zrt_route -check_from_user_shapes</code> argument.
Check for frozen shapes	Checks for DRC violations that are caused by shapes on frozen layers and frozen nets. This option is equivalent to specifying the <code>verify_zrt_route -check_from_frozen_shapes</code> argument.
Report all open nets	Reports all open nets found. If you do not select the option, the command reports a maximum of 200 open nets. This option is equivalent to specifying the <code>verify_zrt_route -report_all_open_nets</code> argument.
Nets	Specifies the nets to verify. During verification, the command checks the specified nets for the following: open nets (<code>-open_net</code>), antenna violations (<code>-antenna</code>), and voltage area violations (<code>-voltage_area</code>). The <code>verify</code> command ignores all other options. The Nets option is equivalent to specifying the <code>verify_zrt_route -nets</code> argument.

4. When you are finished, click **OK**.

The Custom Compiler tool saves the current design and makes it read-only and starts an `xtlImportDRC_#` job for the Job Monitor so that you can monitor the progress.

If you selected the option to run the IC Compiler `verify_zrt_route` command, the Custom Compiler tool creates a custom command file that applies the settings you specified in the **Import DRC Violations from ICC** dialog box and runs `verify_zrt_route` on the current design.

When the `verify_zrt_route` command is finished, the DRC data is imported into the design.

When the import process is finished, the list of DRC violations are displayed in the Error Viewer. For information about the Error Viewer, see the *Custom Compiler Environment User Guide*.

Note that the tool saves the following files in the `icbridge` directory by default:

- `zrt_route.tcl`: The Tcl script for the DRC and import process
- `zrt_route.log`: The log from the DRC and import process

Exporting Designs and Libraries to the IC Compiler Tool

When you are co-designing with the IC Compiler tool and the Custom Compiler environment, you can export designs for input to the IC Compiler tool in several ways.

When you have added timing shell black boxes to a design, you can export it as a VerilogICC netlist, either with single or multiple power supplies. For this option, see Exporting Design Data in Netlist Format and choose the VerilogICC netlist option.

You can export designs and their libraries, or save and export a design only. For either of these options, use one of the procedures in this section.

This topic describes the following:

- [About the Export Process](#)
- [Exporting Libraries to the IC Compiler Tool](#)
- [Creating IC Compiler Libraries for Export](#)
- [Exporting Individual Designs to the IC Compiler Tool](#)

About the Export Process

When exporting Custom Compiler (OA) designs to IC Compiler designs, the following occurs:

- Parameterized cells (PCells) are saved as non-parameterized cells in IC Compiler designs. When opening the exported IC Compiler design in the Layout Editor, one of the following occurs:
 - If the original PCell devices are in lib.defs, the non-parameterized cells are imported as PCell instances.
 - If the original PCell devices are not in lib.defs, the non-parameterized cells are imported as fixed layout (non-parameterized cells).
- Multipart paths and editable PCells are mapped and preserved in IC Compiler designs. The same design can then be imported back into the Custom Compiler Layout Editor with the multipart paths and editable PCells preserved in the design.
- Paths that have round end styles in OA designs are saved as polygons instead of IC Compiler paths with 1/2-width extensions. Saving the paths as polygons ensures that the shapes are interpreted in an identical way across the OA, Milkyway, and GDS databases.

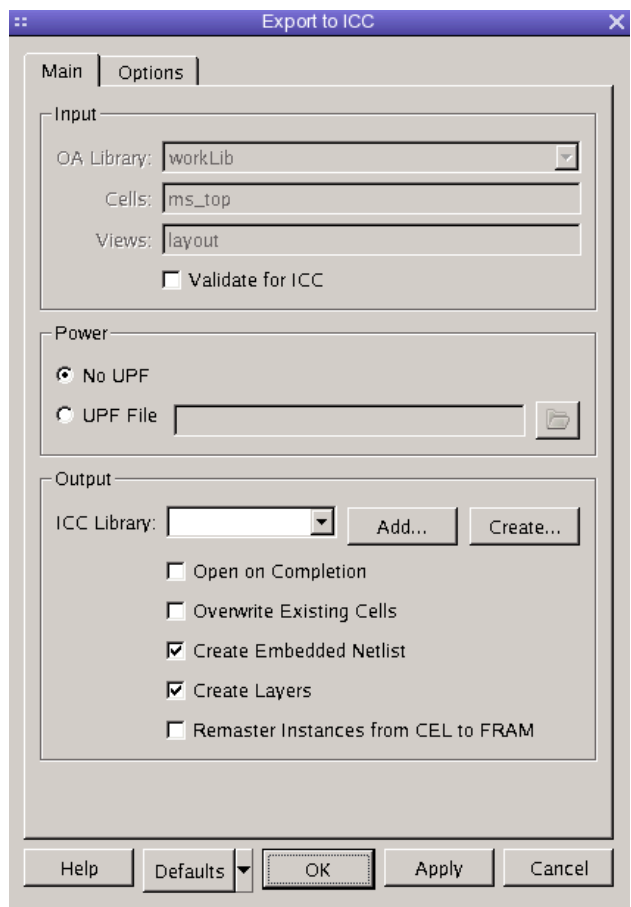
Exporting Libraries to the IC Compiler Tool

Export libraries to the IC Compiler tool for co-design using the Custom Compiler tool and the IC Compiler tool. When you want to use the IC Compiler tool, you can export the design and its libraries.

When you export designs to the IC Compiler tool, a bridge process between the two tools takes the source library containing the data you want to export, converts it for IC Compiler tool access, and adds it to a target library for your use with the IC Compiler tool.

To export a library to the IC Compiler tool:

1. From the Console, choose **File > Export > To ICC**, or use the Tcl command `xt::exportToICC`. The **Export to ICC** dialog box opens.

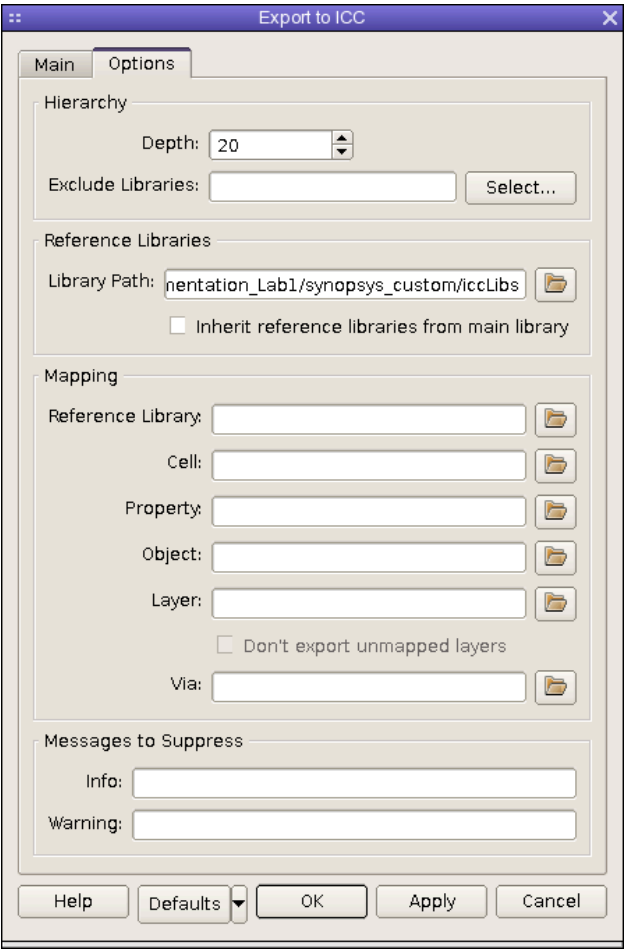


2. In the **Main** tab, specify source and destination details:

Setting	Description
Input	OA Library is the source library you want to translate. Cell(s) is the space-separated list of cells you want to translate. If you leave this field blank, the system translates all cells in the library. View(s) is the space-separated list of view types you want to translate. If you leave this field blank, the system translates all views.

Setting	Description
	Choose Validate for ICC to run the validation process for each design. If any of them fail, export is canceled. If there are only warnings in the design, the export process is not interrupted.
Power	Choose No UPF or browse to your UPF File .
Output	ICC Library is the destination IC Compiler library. To populate this field, click Add to open the Add Existing ICC Library dialog or Create to open the Create ICC Library for Export dialog (see Creating IC Compiler Libraries for Export). Or click the drop-down menu to enable the Other Library option and then browse to select it. Open on Completion opens the target IC Compiler design in the Custom Compiler window upon successful completion of the export and any optional post-processing script. Overwrite Existing Cells translates existing cells in the IC Compiler design. If you do not choose this, existing cells are not translated. Create Embedded Netlist the IC Compiler tool maintains both a flat and a hierarchical representation of the design for the implementation flows. The hierarchical representation is the logical netlist, which is stored as an internal data structure called hierarchy preservation. When this option is not selected, the physical representation of the design along with connectivity is exported to the IC Compiler tool. You can create a FRAM view for this block in the IC Compiler tool, but you cannot run a Place and Route flow. Choosing this option creates the logical netlist representation in the target IC Compiler design, which enables implementation flows on the IC Compiler design without requiring a separate Verilog Import process in IC Compiler. Contact Synopsys support for more information. Create Layers Exports shapes on layers not referenced by the technology. If this option is left unchecked, those shapes will be skipped. Remaster Instances from CEL to FRAM By default the instances of the target design are bound to the same view as the source design. Choosing this option forces all instances that are bound to a layout view to use the FRAM view as reference instead of CEL view. The FRAM view should exist for the instance's master. If the FRAM view does not exist, the instance will be bound to a CEL view. If you want to change the reference views for few instances, use the cell mapping file option.

- Click the **Options** tab in the **Export to ICC** dialog box.



4. In the **Options** tab, set the following:

Setting	Description
Hierarchy	Depth limits the amount of design information exported. Exclude Libraries if depth is greater than 0, you can choose to exclude libraries you enter here from the export. Caution: If the OA reference library and Milkyway reference library have the same name, the export process will overwrite the target Milkyway or OA reference library if the master cells from the library are exported. To prevent the overwrite, use the Exclude Libraries option to exclude the libraries that have the same name during the export process.

Setting	Description
Reference Libraries	Library Path browse to the directory where you want unmapped reference libraries to be created. You can choose whether or not to Inherit reference libraries from main library .
Mapping	Reference Library maps using the external reference <code>lib.map</code> file you browse to select. Cell maps using the <code>cell.map</code> file you browse to select. Property maps using the <code>property.map</code> file you browse to select. Object maps using the <code>object.map</code> file you browse to select. Layer maps using the <code>layer.map</code> file you browse to select. Select Don't export unmapped layers, if you want to exclude unmapped layers during the export process. Via maps using the <code>via.map</code> file you browse to select.
Messages to Suppress	Info Is a space-separated list of Information message IDs you want the tool to suppress Warning Is a space-separated list of Warning message IDs you want the tool to suppress

- The **Mapping** section of the **Options** tab explicitly specifies the library name mappings using full path names.

To view details about all mappings between the IC Compiler and Custom Compiler libraries, see [IC Compiler - Custom Compiler Library Mapping Files](#).

The tool reads the run directory map files, and uses them to perform translation. Duplicate mappings are ignored. Any mapping that conflicts with a previously read mapping generates a warning message describing the change the tool detects. When the tool detects a conflict, it uses the latest mapping found in the translation. Upon completion of the translation, the tool writes the full mapping back to the run directory, overwriting any currently saved files.

- Click **OK**.

The system sets up an `iccbridge` run directory at the path `$SYNOPSIS_CUSTOM_LOCAL/iccbridge`.

It saves the current preference settings to `prefs.xml` within the `iccbridge` directory.

If the target IC Compiler library is new, the system optionally creates a technology file, and creates a target IC Compiler library when it launches the export operation.

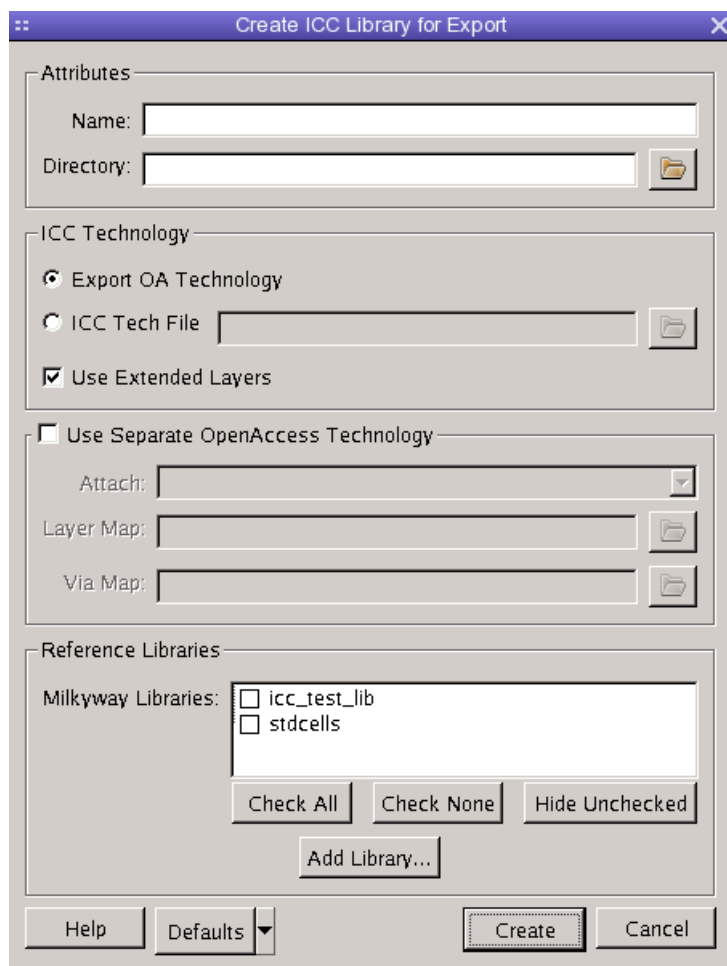
The system checks for updates made to the original library that might be open during the export process. If updates are detected, the system automatically adds the updates to the exported library.

The Custom Compiler tool creates hierarchy preservation within the exported design based on the source design.

Creating IC Compiler Libraries for Export

If you have not created an output IC Compiler library, do so from the **Export to ICC** dialog. To create a target library for export to the IC Compiler tool:

1. Choose **ICC > Export** to open the **Export to ICC** dialog box.
2. In the **Export to ICC** dialog box, click **Create**. The **Create ICC Library for Export** dialog box opens.



3. Specify a library **Name**.

The system creates this library during the export operation.

4. Specify or browse to a **Directory**.

5. In the ICC Technology section, **Export OA Technology** is on by default. **ICC Tech File** uses the technology file you browse to open. Check **Use Extended Layers** to enable use of extended layers.

6. (Optional) **Use Separate OpenAccess Technology** then choose a library to attach. **Attach** uses OpenAccess technology from the library you choose by attaching it, and does not enable two-way technology synchronization between the IC Compiler tool and OpenAccess technology representations.

These mapping files are used to map the layers and vias from the OA technology library to the corresponding layers and vias defined in the IC Compiler technology. This is needed if you supplied an IC Compiler technology file and that technology file has different layer names and numbers compared to the OA technology library that you want to use, too.

7. (Optional) The **Reference Libraries** section lists all open ICC libraries. Open additional libraries by clicking **Add Library**, which opens the **Add Existing ICC Library** dialog (see [Designs Originating with the IC Compiler Tool](#)). None of the libraries are checked by default.

The new library is not actually created upon clicking **Create**. Instead, the information is captured and the new library is listed in the **Milkyway Libraries** box in the **Create ICC Library for Export** dialog with the suffix "(new)" (such as "MyLibName (new)") to differentiate it from existing libraries. To open an existing ICC library, click **Add**, which brings up the **Add Existing ICC Library** dialog (see [Designs Originating with the IC Compiler Tool](#)). Upon dismissing this dialog, the list of ICC libraries is refreshed.

8. Click **OK**.

The system creates the library, the dialog box closes, and the new library name now appears in the **Output** region of the **Export to ICC** dialog box.

After you create the target library, you can continue the process of [Importing Design Data from the IC Compiler Tool](#).

For details about object mappings, see [IC Compiler - Custom Compiler Library Mapping Files](#)

For a description of the mapping files that can be provided when using an attached OA Tech library, see [Using Separate OA Technology for an Opened IC Compiler Library](#).

Exporting Individual Designs to the IC Compiler Tool

You can export individual designs to the IC Compiler tool after you set up the export process by performing it once. Follow the procedure for [Exporting Libraries to the IC Compiler Tool](#).

This provides flexibility to export the current design to the IC Compiler tool.

To export an individual design:

1. In any design window, choose **Tools > ICC Link** to add the ICC menu choice to the design menu.
2. Choose **ICC > Export** to open the **Export to ICC** dialog.

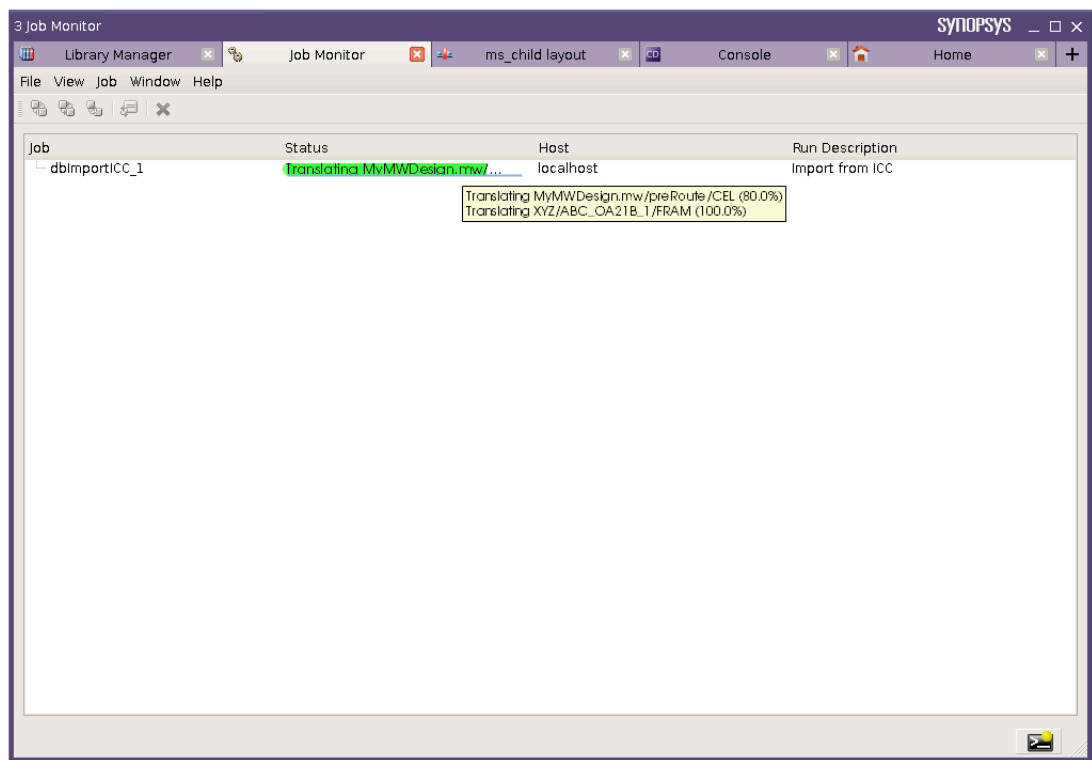
Alternatively, to save and export an individual design:

- Choose **ICC > Save and Export** to open the **Export to ICC** dialog.

Monitoring the Import and Export Progress

Because IC Compiler designs can be large, the import and export processes can take some time. You can use the Job Monitor to track the progress of the processes. To display the Job Monitor, see the *Custom Compiler Environment User Guide*.

During the import or export process, the Job Monitor displays the following information:



Status Information	Description
Job name	The name of the job that is currently running.
Status text	The text in the Status column indicates the design being imported or exported, and the object that is currently being processed.
Green progress bar	Overlaying the status text is a green progress bar that indicates the overall progress of the job.
Blue line beneath the green progress bar	At the bottom of the green progress bar is a thin blue line that indicates the progress of the current phase of the translation.
Tooltip	If you rest the cursor on the progress bar, a tooltip appears and displays the percent completion for the overall import or export operation and the percent completion for the current object being processed.

At the end of the operation, the Status column lets you know if the job is finished or it has failed. For information about the common status messages, see the *Custom Compiler Environment User Guide*.

4

Designing

This chapter provides information about opening designs and deciding which editor to use.

For detailed information, see General Designing.

These sections cover:

- [Opening a Design From the Library Manager](#)
- [View Mapping between IC Compiler and Custom Compiler](#)
- [Working on Schematic Designs](#)
- [Working on Layout Designs](#)
- [Closing Designs](#)

Opening a Design From the Library Manager

Open a design cellview to view or edit it.

To open a design from the Library Manager:

1. In the Library Manager, specify the names of the library, category (optional), cell and view.

Entering text in an input field at the top of a column filters the set of names offered. To return to the full list, delete text in the input field.

2. On the view name, either double-click, or use the alternate mouse button and choose **Open**.

The design opens in the editor for this view.

Preference options apply. Active cellview level preference settings also apply, and take precedence.

The first time an IC Compiler design is opened in the Custom Compiler environment, the design information is imported to the custom environment. The duration of this step depends on the size of the design, the number of shapes in the design, and how many

cell masters are instantiated. If the IC Compiler design is not modified, subsequent design opens are faster.

When a colored IC Compiler design is opened in the Custom Compiler environment, ICC-Custom Compiler link automatically maps the IC Compiler color mapping information to Custom Compiler color classes. For information about color classes (the `colorClass` layer rule), see the *Custom Compiler Technology and Display Resource File Reference Manual*. For information about assigning colors to shapes and the color propagation rules in the Layout Editor, see the *Custom Compiler Layout Editor User Guide*.

Opening an IC Compiler design on the Custom Compiler environment places a lock on the design so that it cannot be modified in both environments at the same time.

View Mapping between IC Compiler and Custom Compiler

The following table shows layout view mapping between the IC Compiler and Custom Compiler environments.

Table 8 *View Mapping between IC Compiler and Custom Compiler*

IC Compiler Design viewName	Custom Compiler View Type
layout	.CEL
abstract	.FRAM
fill	.FILL
smash	.SMASH
ILM	.ILM

The view you open determines the editor you will use. All of the views in [Table 8](#) use the Custom Compiler Layout Editor.

Opening an IC Compiler design that has the instances bound to a FRAM view in IC Compiler automatically binds the instances to an abstract view in the Custom Compiler environment (the equivalent of the FRAM view). To swap the views for any instance, run the `change_macro_view` command in IC Compiler to change the reference for the desired instances. Save the design in IC Compiler and then reopen the design in the Custom Compiler window.

Working on Schematic Designs

For detailed information on working with schematic designs, see Schematic Editor Description.

Working on Layout Designs

For detailed information on working with layout designs, see Layout Editor Description.

Closing Designs

Once your editing steps are completed in the Custom Compiler environment, simply save the design to save these edits back to the IC Compiler view.

After you draw routes, you can use the IC Compiler tool to perform place and route on the complete design. You can then either close the design in the Custom Compiler window or make it read-only to use the IC Compiler tool to perform further design implementation tasks such as floor plan, placement, routing, and design optimizations. You can go back and forth between the two design environments as many times as needed to complete both the custom/analog layout and digital layout implementation.

5

Preparing Libraries for Co-Design

This chapter describes how to prepare libraries for co-designing in the Custom Compiler environment.

You must prepare libraries for co-designing with the IC Compiler tool in the Custom Compiler environment before designing analog blocks or implementing digital blocks.

- If no IC Compiler library exists for your design, create a library of the IC Compiler type in the Custom Compiler environment.

For instructions, see [Designs Originating in the Custom Compiler Environment](#).

- If a IC Compiler library does exist for your design, make it available to the Custom Compiler environment by adding its library.

For instructions, see [Designs Originating with the IC Compiler Tool](#).

- If no timing library exists for your custom design, and you are planning to run place and route with multi-voltage power sources, you need to create one.

For instructions, see [Creating Timing Libraries for Multi-Voltage Power in Place and Route](#).

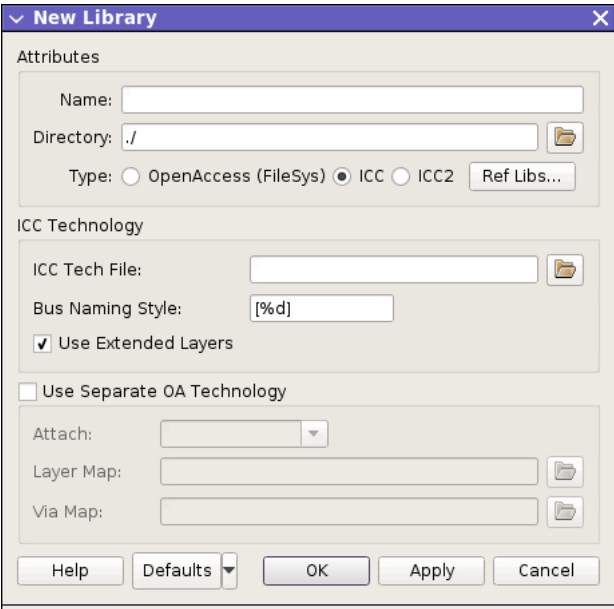
Designs Originating in the Custom Compiler Environment

When you have a design originating in the Custom Compiler Environment, and you want to transfer it to the IC Compiler tool for digital block place and route, create an IC Compiler library for it.

(If this library already exists, see instead [Designs Originating with the IC Compiler Tool](#).)

To create an IC Compiler design library:

1. From the Custom Compiler environment, display the Library Manager.
2. In the Library Manager, choose **File > New > Library**. The **New Library** dialog box opens.



- 3. Select the library **Type: ICC**, and click **Ref Libs** to select the reference libraries that this library will have as reference libraries. You can choose from the list of reference libraries, or you can open more libraries to consider for use as reference libraries.
- 4. Set the ICC library options:

Settings	Description
ICC Technology	ICC Tech File – Uses the technology file you browse to open. Bus Naming Style – Choose the bus naming style.
Use Separate OA Technology	Attach – Uses OpenAccess technology from the library you choose by attaching it, and does not enable two-way technology synchronization between IC Compiler and OpenAccess technology representations. Optional, in case an IC Compiler technology file is not available. Browse to your Layer Map and Via Map files to map the OA layers from the OA technology data to different IC Compiler layers in the IC Compiler database.

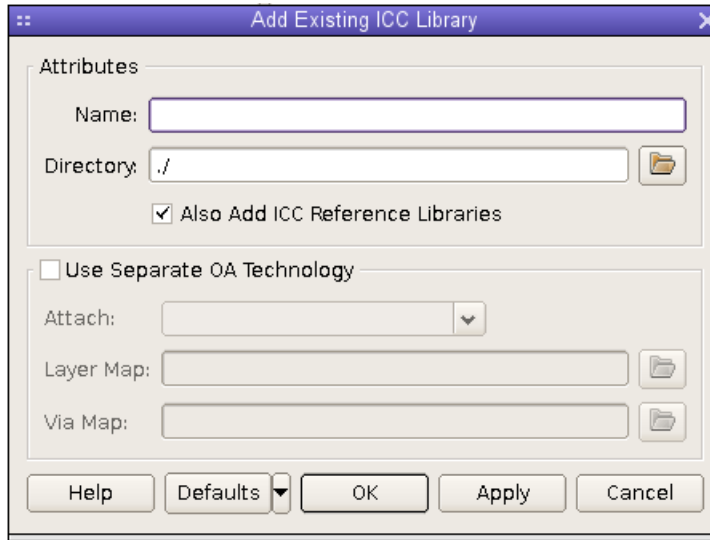
- 5. Click **OK**.

Designs Originating with the IC Compiler Tool

When you have a design originating with the IC Compiler tool, and you want to develop it in the Custom Compiler environment, add an IC Compiler design library for it.

To add an IC Compiler design library:

1. In the Custom Compiler Library Manager, either choose **File > Add ICC Library** or right-click anywhere in the **Libraries** list, and choose **Add**. The **Add Existing ICC Library** dialog box opens.



2. Leave the **Name** field blank. This field automatically populates when you select the IC Compiler library in the **Directory** field.
3. In the **Directory** box, type the path to the IC Compiler library, or click  to select the IC Compiler design library for this design.
4. (Optional) Select **Also Add ICC Reference Libraries** if you want Custom Compiler to add all reference libraries stored within the specified IC Compiler library to the library definitions. When the design changes, the libraries referenced by this design are updated. By default, the option is selected.

When the tool encounters duplicate names for reference libraries, it appends **_ICC** to the duplicate, and adds a number to increment each duplicate name.

5. Select **Use Separate OA Technology** if you want to use a separate OA technology file from your library.

- In the Attach list, choose an OA technology file from your library. This OA technology file will be used with OA applications but not the IC Compiler tool.
- In the Layer Map box, type the path to the layer mapping file, or click  to locate and select the layer mapping file that you want to use to map the IC Compiler layers to the corresponding OA layers defined in the associated OA library.
- In the Via Map box, type the path to the via mapping file, or click  to locate and select the via mapping file that you want to use to map the IC Compiler vias to the corresponding OA vias defined in the associated OA library.

Note:

Using separate OA technology requires write access to the IC Compiler library.

6. Click **OK**.

The tool adds a DEFINE statement with the path to this library to the current `lib.defs` file for automatic access on next startup.

Creating Timing Libraries for Multi-Voltage Power in Place and Route

You can create timing libraries when you want to run place and route on designs with multiple power supplies using power supply definitions or a UPF file.

To create a timing library:

1. In the Custom Compiler environment, export the design in Liberty library file format. See Exporting Design Data in Liberty Library File Format.
2. Convert the Liberty format file from `<cellName>.lib` to `<cellName>.db` using one of these two methods:
 - Use the IC Compiler tool to make this conversion, using the IC Compiler functions `read_lib` and `write_lib`.
 - Set the `leICCGenerateBlackBoxTimingFiles` preference to `true` for the place and route process.

When this preference is set to `true`, the tool searches for a `<blackbox>.lib` file in the Linux paths specified in the search path preference `leICCSearchPath`. If it finds the a `.lib` file, it converts this file to a `<blackBox>.db` file.

3. Supply this file as input to the place and route process.

See [Running IC Compiler P & R from Design Schematics or Verilog Netlists](#).

6

Editing IC Compiler Designs in the Custom Compiler Environment

This chapter describes how to edit IC Compiler designs in the Custom Compiler environment.

Analog content in IC Compiler designs is increasing and designers have a need to use a full custom layout editor and custom router to implement efficiently the analog nets in those designs. You can develop analog nets with co-design using the Custom Compiler tool and the IC Compiler tool.

You can implement analog routes, either automatically or interactively,

After you create routes, you can use the IC Compiler tool to perform place and route on the complete design.

These sections cover:

- [Custom Compiler Tools Used in IC Compiler Co-design Flows](#)
- [Implementing Analog Nets](#)
- [Performing IC Compiler Operations on Design Layouts](#)
- [Use Model Example](#)

Custom Compiler Tools Used in IC Compiler Co-design Flows

The following sections briefly describe some of the tasks and the Custom Compiler products and features you might use in an ICC co-design flow.

- [View Nets, Instances, and Terminals](#)
- [Set Design-Specific Constraints](#)
- [Adjust Components](#)
- [Create Interconnect](#)
- [Create Routes](#)

- [Check Metal Density](#)
- [Check EM and Check Resistance](#)

View Nets, Instances, and Terminals

In the Layout Editor, you can use the Design Navigator to view layout information such as nets, instances, instTerms, Terms, net groups, and figure groups from the IC Compiler design data. Constraints can be created using the Constraint Manager on nets selected in the Design Navigator.

To display the Design Navigator, choose **Window > Assistants > Design Navigator** in the Layout Editor.

Set Design-Specific Constraints

By default, the constraint parameter values are set to the values in the technology library. You can override the values in the technology library by supplying the desired values in the Constraint Editor. Some design-specific constraints also supplement the process rules, such as placement constraints.

To display the Constraint Editor, choose **Window > Assistants > Constraint Editor** in the Layout Editor or Schematic Editor windows. For placement constraints, see the *Custom Compiler SDL and Visually Assisted Automation Guide*. For routing constraints, see the *Custom Compiler Custom Router User Guide*.

If you create placement constraints, you must export and store the constraint data with the design using the `cm::exportIntents` Tcl command. When you re-open the design in the Custom Compiler environment, you must use the `cm::importIntents` Tcl command to restore the placement constraints.

Adjust Components

In the Layout Editor, you can use the Property Editor assistant to adjust the attribute or property values of components. For example, you can modify the number of striped wires, the dimensions and orientation of a bounding box, and the location and spacing of route guides.

To display the Property Editor, choose **Window > Assistants > Property Editor** in the Layout Editor. For more information, see the *Custom Compiler Environment User Guide*.

Create Interconnect

IC Compiler designs have wire tracks created automatically as part of the floorplan setup. The wire tracks are used by the IC Compiler auto router ZRoute to route the design

efficiently. When manually editing IC Compiler designs, it is preferable to keep the routes on track to keep the efficiency of the layout, leave tracks available for other routes as far as possible, and also as an aid to keep the layout DRC clean. The Custom Compiler Track Pattern assistant supports displaying the wire tracks, using the option to support snapping routes to tracks during the **Create > Interconnect** command. Open the Track Pattern assistant using **Window > Assistants > Track Pattern Assistant**. In the Track Pattern assistant, enabling the Active setting for a layer sets the track pattern to be active. For an active track pattern, mouse movements snap to the track during creation. Copy, move, and stretch operations also snap to the track.

The IC Compiler tool prefers to deal with HWIRE and VWIRE objects, for instance for ECO routing. For more flexibility in the interaction between the IC Compiler and Custom Compiler environments, you can set the default objects that will be created by the custom router, Interactive Router, and **Create > Interconnect** commands to be `oaPathSeg`. To do so, set the following preference:

```
db::setPrefValue leUsePathSeg -value true
```

The preference value can be changed in the **Options > Design** dialog box, **Command** tab.

See [Setting Layout Design Options and Creating and Editing Interconnects](#).

Create Routes

You can use the Custom Router to automatically or interactively create interconnects (or routes) for digital and mixed signal nets at all levels of the chip design. The Custom Router integrates both electrical and topological rules as part of the routing process. For digital IC designs, you can pre-route sensitive nets using a set of custom routing options and continue with the IC Compiler tool to complete the physical implementation.

You can also use the Custom Router to add shielding during the routing process or post route. You can choose from different shielding styles. In addition, you can set up shields such that they are shared between two adjacent signal routes.

To use the Custom Router, in the Layout Editor, choose **Tools > Routing**. For instructions on how to use the router, see [Net-Based Routing Workflow](#).

Routes (shapes) created by the Custom Router (Auto Router, Interactive Router, or Pattern Route Assistant) have the **Custom Route** Route Type property. You can view the property of the shapes in the Property Editor. You can confirm the property values when you review the design in the IC Compiler II tool.

Check Metal Density

You can use the Layout Editor metal density analyzer to check the metal density of your design. Checking the metal density ensures uniform distribution of components across metal layers, thereby improving manufacturability. For more information, see the *Custom Compiler* Design Checking and Physical Verification User Guide.

Check EM and Check Resistance

You can use the Design Navigator to check electromigration and resistance for selected nets or all the nets in a design. For more details, see the Checking and Reporting Electromigration and Checking and Reporting Resistance topics.

Implementing Analog Nets

Before you perform place and route for an entire design with the IC Compiler tool, you can choose to create individual routes for analog nets in the Custom Compiler environment. This includes nets with special shielding or contours.

You can automatically or interactively create routes for individual nets.

Route guides defined in IC Compiler designs define regions where routes are not permitted. You can optionally edit the route guides before creating routes.

Foundry rules are applied in an equivalent manner in both tools.

These sections cover:

- [Safe Editing](#)
- [Namespace Mapping](#)
- [Object Mapping](#)
- [Adjusting Placement of Cells](#)
- [Editing Route Guides](#)
- [Creating Routes Automatically](#)
- [Creating Routes Interactively](#)
- [Adding Shielding](#)

Safe Editing

The Custom Compiler safe-editing mode prevents you from modifying the logical elements of the design that would affect the IC Compiler view consistency check. It ensures that you can safely continue to use the IC Compiler for the design implementation after editing it in the Custom Compiler environment.

When working with IC Compiler designs, the Custom Compiler safe-editing mode is on by default.

Namespace Mapping

When IC Compiler designs open in the Custom Compiler environment, IC Compiler tool instance and net names map to Custom Compiler naming conventions, which use OpenAccess notation.

The following table shows the character mapping between the IC Compiler tool and Custom Compiler environments. Unless otherwise indicated, the information below is true for both instance and net names.

IC Compiler	Custom Compiler
	/
#2f and /	³ (see Note)
[	(for instances < for nets
]	) for instances > for nets
<	#3c
>	#3e
\	#5c

Object Mapping

The following table shows object mapping between the IC Compiler tool and Custom Compiler environments.

- Both #2f and / map from the IC Compiler tool to the Custom Compiler environment as |, but from the Custom Compiler environment to the IC Compiler tool, | only maps to /.

Note:

Objects labeled cd* are custom objects and are warehoused in the IC Compiler tool for roundtrip.

IC Compiler	Custom Compiler
Cell	oaInst
Pin	oaPin
Port	oaBitTerm
Net	oaNet
Metal Layer Blockages	oaLayerBlockage of type routing
System Metal Layer Blockages ⁴	oaLayerBlockage of type routing
Placement Blockages	oaAreaBlockage
Rectangle	oaRect
Polygon	oaPolygon
Trapezoid	oaPolygon
Text	oaText
Path ⁵	oaPath
HWIRE/VWIRE	oaPathSeg ⁶
Parameterized contact codes	oaStdVia
Rectangle-based contact codes	oaStdVia/oaCustomVia
Design-specific via master	oaStdVia/oaCustomVia
Voltage Area	leVoltageArea ⁶
Routing Corridors	leRouteCorridor ⁷

4. IC Compiler via layer blockages in FRAM view are mapped to via layers in the Custom Compiler environment.

5. Shield association to nets is not maintained in the IC Compiler tool when path objects are used.

6. The IC Compiler tool prefers to deal with HWIRE and VWIRE objects, for instance for eco routing. For more flexibility in the interaction between the IC Compiler and Custom Compiler environments, you can set the default objects that will be created by the custom router, Interactive Router, and Create > Interconnect commands to be oaPathSeg. To do so, set the following preference: db::setPrefValue leUsePathSeg -value true. The preference value can be changed in the Options > Design dialog, Command tab. The Custom Compiler Layout Editor does not display the power domain names associated with a single voltage area in the Property Editor or in the Object Information toolbar. It does display the power domain name if only one power domain name is associated with a voltage area.

7. A new semantic object in the Custom Compiler environment.

IC Compiler	Custom Compiler
Wire Tracks	leTrackPatternDef/leTrackPattern
cdTrackPatternDef	leTrackPatternDef/leTrackPattern originating from the Custom Compiler environment
Core Area	oaGroup ⁸
Cell Row (Site Row)	oaRow
Cell Boundary	oaPRBoundary
PR Boundary	oaSnapBoundary
Route Guides	oaAreaBoundary
cdAreaBoundary	oaAreaBoundary originating from the Custom Compiler environment
Custom Compiler-Specific Objects	
cdSteiner	Steiners
Group	oaRoutes
Edit Group	Striped Wires
Edit Group	Synchronous Fig Groups
Edit Group	Multi-Part Paths
Polygon	oaEllipse ⁹

Adjusting Placement of Cells

You can move standard cells in an IC Compiler design and the tool snaps the standard cells to the rows in the design, at proper multiple locations of the unit tile.

8. The oaGroup mapped to the core area uses the oaFlatGroupDef named 'ICC_BASE_ARRAY', which identifies the group semantically and only allows oaRow objects to be members of the group.
9. The IC Compiler tool does not support polygons, so oaEllipse(s) are translated to 32-point IC Compiler polygons. Ellipses are recovered in Custom Compiler environment when opening the IC Compiler design even if they are stored as polygons in the Milkyway database.

Figure 1, Figure 2, and Figure 3 illustrate the move and snap to row/site locations.

Figure 1 *Selecting the Instance to Move*

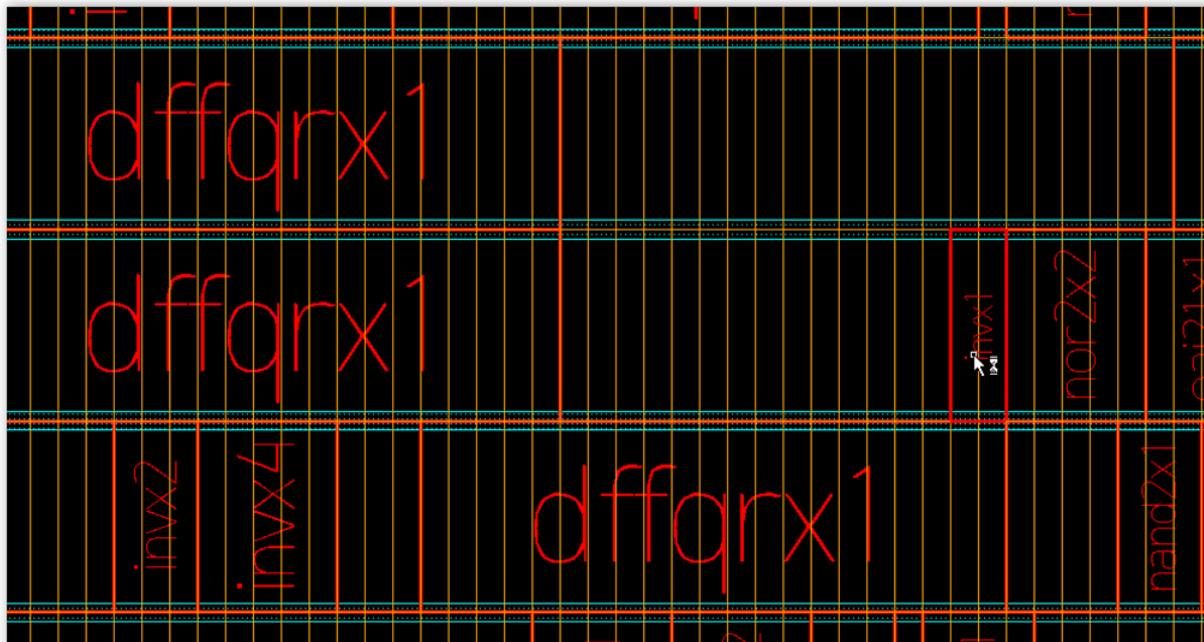


Figure 2 *The Instance Snaps to Row Sites during the Move Command*

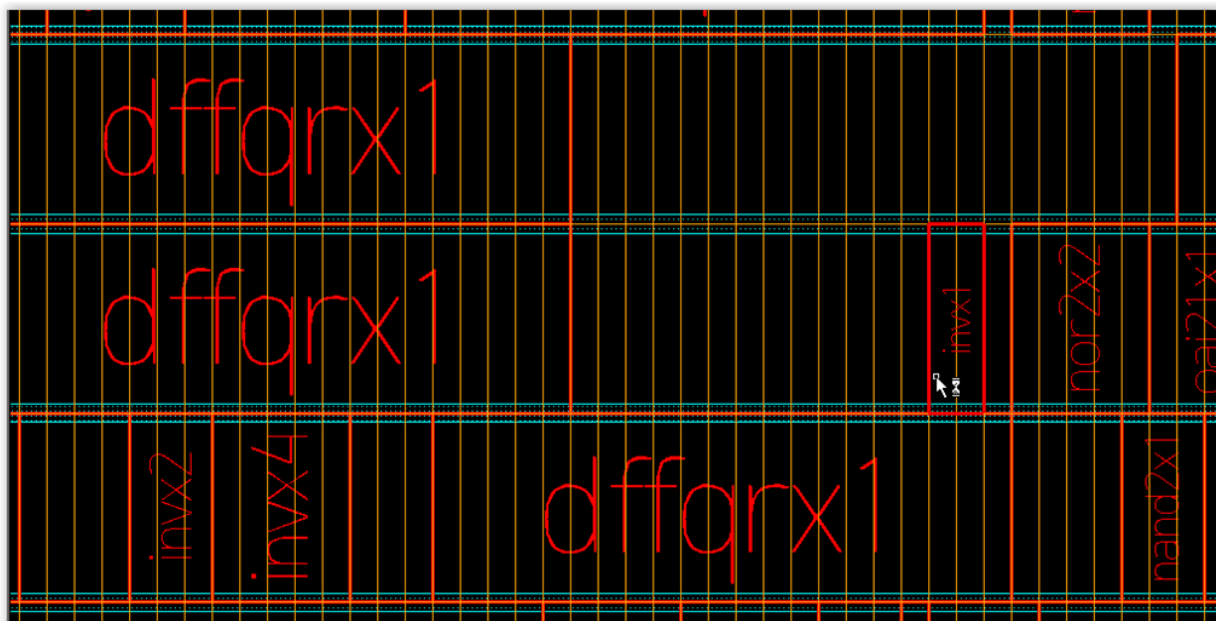
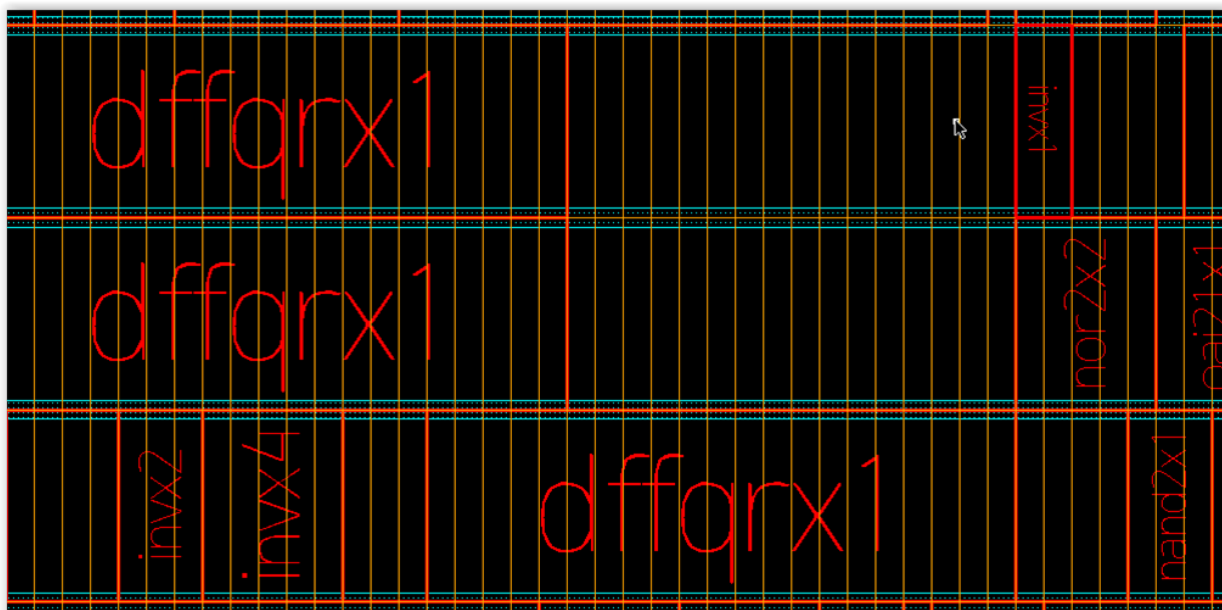


Figure 3 *The Instance Orientation Adjusts when Moved to the Next Row in a Double Back Row Configuration*



In the figures above, rows are separated by the blue m1 layer power and ground straps. Site locations are identified with the orange lines.

Cells match the required orientation of the row, as shown in [Figure 3](#), where the cell has been flipped (as seen by the location of the beveled corner). Make sure that cells do not overlap when you move them, because the tool allows cells to be moved over other ones, resulting in overlaps, even though cell overlaps are not legal in the IC Compiler tool. After you move cells to the target location, lock their placement by changing the placement status in the Property Editor assistant. When the cell placement status is locked, neither Custom Compiler or IC Compiler tools allow moving the cell either manually or automatically. Macro cells are not bound to respect the row site locations.

Editing Route Guides

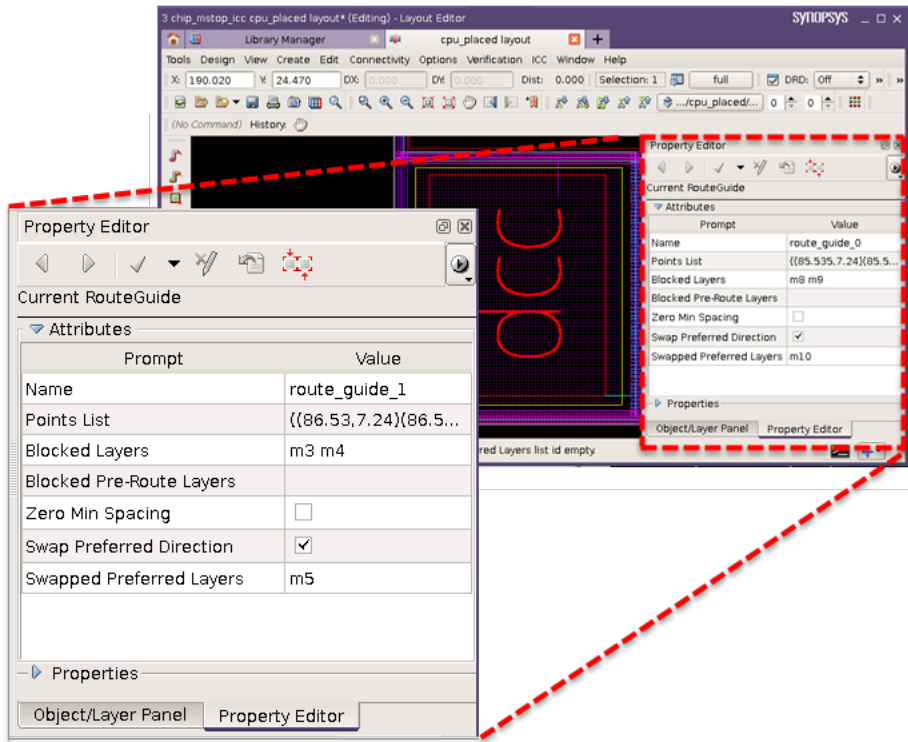
You can edit route guides to further refine the design before adding routes. Route guides are rectangular shapes created in IC Compiler designs to provide routing directives. For example, you can use route guides to define regions where routes are not permitted. Route guides appear as area boundaries in the Custom Compiler Layout Editor. For more information about route guides, see the [IC Compiler user documentation](#) available on SolvNetPlus.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

To modify route guides in the Layout Editor:

1. Open the IC Compiler design in the Layout Editor. For instructions, see [Opening a Design From the Library Manager](#).
2. Display the Property Editor.
3. On the canvas, locate and select the route guide you want to modify. The attributes of the selected route guide appear in the Property Editor. By default, the **Attributes** section is expanded.



4. Modify the desired route guide **Attributes**.

Attribute	Description
Name	Identifies the selected route guide. By default, the name assigned in the IC Compiler tool is displayed. To change the name, double-click the box, and then type a unique name.

Attribute	Description
Points List	Specifies the location of the route guide. To change the location, click  . In the Edit Points dialog box, update the coordinates of route guide vertices.
Blocked Layers	Specifies the layers that cannot be used for routing. To change the layers, click  , and then select the layers in the Layers dialog box.
Blocked Pre-Route Layers	Specifies the layers that cannot be used for pre-routes. This attribute is equivalent to the <code>no_preroute_layers</code> property in the IC Compiler tool. For more information, see the IC Compiler user documentation. The Router will not use this attribute during routing. However, you can change the value and the updated value will be available when you open the design in the IC Compiler tool.
Zero Min Spacing	Permits routes within the keepout margin of the route guide.
Swap Preferred Direction	Switches the preferred routing direction for specified layers within the route boundary. Select this attribute to reduce congestion in designs that contain many detour routes. This attribute is equivalent to the <code>switch_preferred_direction</code> property in the IC Compiler tool. For more information, see the IC Compiler user documentation. The Router will not use this attribute during routing. However, you can change the value and the updated value will be available when you open the design in the IC Compiler tool.

Attribute	Description
Swapped Preferred Layers	Specifies the layers where the preferred routing direction can be switched. This attribute is equivalent to the <code>switch_preferred_direction</code> property in the IC Compiler tool. For more information, see the IC Compiler user documentation. This field is available only if you selected Swap Preferred Direction. The Router will not use this attribute during routing. However, you can change the value and the updated value will be available when you open the design in the IC Compiler tool.

5. When you are finished, click **Apply Changes** () to save the modifications.

Creating Routes Automatically

You can create net routes automatically when you are developing analog or mixed signal regions, and you want to allow the Custom Router to control the route path.

To create routes automatically:

1. Add the IC Compiler design to the Custom Compiler environment. See [Designs Originating with the IC Compiler Tool](#) for details.

2. If you have not yet done so, close the design in the IC Compiler tool.

3. Open the design in the Custom Compiler environment.

The Custom Compiler environment locks the IC Compiler version of this design.

By default, the Object/Layer Panel opens.

4. Display the Design Navigator by choosing **Window > Assistants > Design Navigator**.

5. In the Design Navigator, locate and select a net.

Before creating net routes, you can optionally use the Constraint Editor to create routing constraints.

You can use the Design Navigator to set flight line colors. See Design Navigator Assistant.

6. In the Layout Editor, choose **Tools > Routing**. The **Routing** menu appears on the menu bar.

7. Automatically create the selected net route:

- a. Choose **Routing > Auto Route**.
- b. On the **Auto Route** toolbar, choose the **Selected** option to route the selected net.
- c. (Optional) In the Design Navigator, turn on  (**Shadow Mode**) to dim the design surrounding the selected net.
- d. On the **Auto Route** toolbar, click  to start routing.

For additional routing instructions, see Custom Router Description.

8. View the results. With shadow mode enabled, the net routes appear visually brighter than the surrounding design.

9. (Optional) Save the design at any point.

Saving the design commits all changes to the Milkyway database.

From this point, you can optionally open the design in the IC Compiler tool, perform the next place and route operation, and then optionally return the design to the Custom Compiler environment to continue designing.

Before you can open the design for writing in the IC Compiler tool, it has to be either closed or changed to read-only in the Custom Compiler environment.

Creating Routes Interactively

You can create net routes interactively when you are developing analog or mixed signal regions, and you control the route path.

This is an IC Compiler tool and Custom Compiler environment co-design flow.

To interactively route designs:

1. Begin by [Preparing Libraries for Co-Design](#).
2. If you have not yet done so, close the design in the IC Compiler tool.
3. Open the design in the Custom Compiler environment.
4. Display the Design Navigator by choosing **Window > Assistants > Design Navigator**.
5. In the Design Navigator, locate, and select a net to route.

Before creating net routes, you can optionally use the Constraint Editor to create routing constraints.

You can use the Design Navigator to set flight line colors. See Design Navigator Assistant.

6. In the Layout Editor, choose **Tools > Routing**. The **Router** menu appears on the menu bar.
7. Choose **Routing > Interactive Route** to create routes using one of the two modes:

Follow-the-Cursor Mode

- a. On the **Route** toolbar, deselect the default **Point to Point** option to enable follow-the-cursor mode.
- b. Specify **Multilayer** or **Single Layer**.
- c. On the canvas, click the source pin and create the route, using the dynamically displayed flight line as a guide to the destination pin.
- d. To complete the path, optionally move the cursor directly over the destination pin or right-click the canvas near the destination pin, and choose **Finish Route**.

Point-to-Point Mode

- a. On the **Route** toolbar, choose the default **Point to Point** option, if it is not now selected.
- b. Specify **Multilayer** or **Single Layer**.
- c. On the canvas, click the pin, and then create the route, clicking again in locations where you want to set intermediate route points.

The router finds the best path along the intermediate route points.

- d. (Optional) Adjust constraint values, and recreate the route.

Use the Constraint Editor assistant to adjust constraints.

8. View the results. With shadow mode enabled, the routed nets appear visually brighter than the surrounding design.
9. (Optional) Save the design at any point.

Saving the design commits all changes to the Milkyway database.

From this point, you can optionally open the design in the IC Compiler tool, perform the next place and route operation, and then optionally return the design to the Custom Compiler environment to continue designing.

Before you can open the design for writing in the IC Compiler tool, it has to be either closed or changed to read-only in the Custom Compiler environment.

Adding Shielding

If you enabled the shielding options, shielding is automatically added based on your specifications during the auto routing and interactive routing operations. However, you can choose to disable shielding during routing and add shielding after the routing operation is finished. Post-route shields are wrapped around existing routes without modifying the route.

To add shielding during routing:

- *Auto Routing*: Use the options in the **Shield** tab of the **Auto Route** dialog box.
- *Interactive Routing*: Set the shield options on the **Route** toolbar.

To add shielding post-route, choose **Routing > Add Shield**, or choose **Routing > P2P Shield**.

For information about adding shielding, see Adding Post-Route Shielding and Shield Tab.

After you add shielding, you can check the shield coverage to ensure that it is optimal. For more information, see Reporting and Checking Shield Coverage.

Shielding that is created in the IC Compiler tool are preserved and not modified in the Custom Compiler environment. Shielding that is created using the Custom Router are preserved and not modified in the IC Compiler tool.

Performing IC Compiler Operations on Design Layouts

After you run place and route on a design schematic to implement the block, you can continue to use the IC Compiler tool to work with the block.

You can perform IC Compiler operations on the IC Compiler design (opened in the Custom Compiler Layout Editor) using Tcl scripts, or using Console command input that executes IC Compiler commands. Console command input can include both individual IC Compiler commands and Tcl scripts.

These sections cover:

- [Running the IC Compiler Tool Using Tcl Scripts for Input](#)
- [Running the IC Compiler Tool Using Console Commands](#)
- [Setting IC Compiler Shortcut Keys for the Custom Compiler Environment](#)

Running the IC Compiler Tool Using Tcl Scripts for Input

You can run IC Compiler tool operations on design layouts and use a Tcl script to determine the input settings.

To run the IC Compiler tool using a Tcl script:

1. In the Custom Compiler environment, from the Layout Editor, open the IC Compiler design.
2. In the Layout Editor, choose **ICC > Run ICC**.

This menu option is available only when the IC Compiler design is open.

3. Browse to select the Tcl script file for the system to use.

By default, the tool looks in the current working directory first, unless the `xtICCScript` preference sets it to a different directory location.

4. Click **Open**.

The system evaluates the script in the IC Compiler tool.

Once you choose **Apply** or **OK** in the dialog, the Layout Editor saves the current design, makes it read-only, and launches the IC Compiler tool with the IC Compiler-selected script. Then, upon completion of the IC Compiler run, the design is refreshed in the Layout Editor.

See [Tracking Progress and Viewing Output Files for IC Compiler Operations](#) to continue.

Running the IC Compiler Tool Using Console Commands

You can run IC Compiler tool operations on design layouts using command input from the Custom Compiler console.

This example executes the IC Compiler `route` command on the active design, using current settings:

```
xt::execICC route_zrt_auto -cellView [de::getActiveContext]
```

This example executes the script `myScript.tcl` and passes it as input to the IC Compiler tool. In the example, `myScript.tcl` is a script you create containing IC Compiler commands.

```
xt::execICC {source myScript.tcl}
```

See the *Custom Compiler* Tcl Reference Manual for command descriptions.

When you run the Tcl script, the Custom Compiler tool saves the current design, makes it read-only, and launches IC Compiler tool with the IC Compiler-selected script. Then, upon completion of the IC Compiler run, the design is refreshed in the Custom Compiler window.

See [Tracking Progress and Viewing Output Files for IC Compiler Operations](#) to continue.

Setting IC Compiler Shortcut Keys for the Custom Compiler Environment

You can switch between the IC Compiler tool and Custom Compiler keyboard shortcuts. From the console window, choose **Options > General**. In the dialog, select IC Compiler as the **Active Binding Set**.

Not all IC Compiler keyboard shortcuts are supported. Refer to the table of shortcut keys for the available IC Compiler set below.

IC Compiler Function	IC Compiler Shortcut Keys	Custom Compiler Shortcut keys
Edit > Copy	C	Edit > Copy
Edit > Create > Text	Shift+T	Create > text > Label
Edit > Create > Via	Shift+C	Create > Via
Edit > Delete	D	Edit > Delete
Edit > Merge Net Shapes	Shift+M	Edit > Merge
Edit > Move/Resize	M	Edit > Move
Edit > Properties	Ctrl+R	Property Editor Assistant
Edit > Redo	Ctrl+Y	Edit > Redo
Edit > Stretch Wire	Shift+W	Edit > Stretch
Edit > Undo	Ctrl+Z	Edit > Undo
Highlight > Clear All	Ctrl+M	Highlight > Clear
Select > By Name	Ctrl+B	Ctrl+B
Select > Clear	Ctrl+D	Ctrl+D
Verification > Previous Error	Shift+E	View > Zoom > In

IC Compiler Function	IC Compiler Shortcut Keys	Custom Compiler Shortcut keys
View > Mouse Tools > Ruler Tool	Ctrl+U	Create > Ruler
View > Mouse Tools > Smart Selection Mode	P	Edit > Select > By Region
View > Zoom > Zoom Fit All	F	View > Fit > view
View > Zoom > Zoom Fit Selection	Ctrl+T	View > Fit > Selected
View > Zoom > Zoom In	+	View > Zoom > In
View > Zoom > Zoom In	=	NA
View > Zoom > Zoom Out	-	View > Zoom > Out
View > Zoom > Zoom Out	Shift+Z	Shift+Z
View > Zoom > Zoom To	T	View > pan
Window > Close View	Ctrl+W	Design > Close
Remove point during editing command.	Backspace	Backspace
Abort the current active command.	Esc	Esc
In the console window, move backward a character.	Left	Left
In the console window, move forward a character.	Right	Right
In the console window, move to the start of the current line.	Home	Home
In the console window, move to the end of the current line.	End	End
In the console window, select next event in history.	Up	Up
In the console window, select previous event in history.	Down	Down
Set the selectability and visibility of layer on metal1 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+1	Ctrl+1
Set the selectability and visibility of layer on metal2 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+2	Ctrl+2

IC Compiler Function	IC Compiler Shortcut Keys	Custom Compiler Shortcut keys
Set the selectability and visibility of layer on metal3 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+3	Ctrl+3
Set the selectability and visibility of layer on metal4 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+4	Ctrl+4
Set the selectability and visibility of layer on metal5 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+5	Ctrl+5
Set the selectability and visibility of layer on metal6 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+6	Ctrl+6
Set the selectability and visibility of layer on metal7 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+7	Ctrl+7
Set the selectability and visibility of layer on metal8 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+8	Ctrl+8
Set the selectability and visibility of layer on metal9 purpose drawing and turn off selectability/visibility for all the other LPPs.	Ctrl+9	Ctrl+9
Toggle selectability and visibility of LPP metal1 drawing.	1	1
Toggle selectability and visibility of LPP metal2 drawing.	2	2
Toggle selectability and visibility of LPP metal3 drawing.	3	3
Toggle selectability and visibility of LPP metal4 drawing.	4	4
Toggle selectability and visibility of LPP metal5 drawing.	5	5
Toggle selectability and visibility of LPP metal6 drawing.	6	6
Toggle selectability and visibility of LPP metal7 drawing.	7	7

IC Compiler Function	IC Compiler Shortcut Keys	Custom Compiler Shortcut keys
Toggle selectability and visibility of LPP metal8 drawing.	8	8
Toggle selectability and visibility of LPP metal9 drawing.	9	9

Use Model Example

The following sections describe common use models for the IC Compiler tool and Custom Compiler co-designing:

- [Creating Analog Pre-Routes Targeting a Specific Resistance Value](#)
- [Cleaning Up the Layout after Routing](#)
- [Working with Routing Objects in an ICC-Custom Compiler Flow](#)

Creating Analog Pre-Routes Targeting a Specific Resistance Value

You might have to implement analog signal nets or analog power nets that target specific resistance values to connect high-speed IP blocks. Custom Compiler tools can help you with such implementation by providing key features such as the Resistance Checker and the Stripe Wire commands.

First, estimate which layers to use with what width to achieve the resistance target for your critical net. Layer and width constraints can be set for that net in the Constraint Editor.

To create the wires that achieve your estimated resistance, use **Create > Interconnect**, **Routing > Interactive Route**, and **Routing > Auto Route**. You can use the Resistance Checker to confirm that the resistance of the route that you created matches the resistance constraint set for that net using the Constraint Editor. If needed, widen the segments of the route to decrease the overall effective resistance. To iterate on the net implementation, you can delete all the shapes on the net by right-clicking the net and selecting the menu command in the Design Navigator. (For more details on routing commands, see Routing Methods. For more information on checking resistance, see Reporting and Checking Resistance.)

The route width might end up being larger than the foundry maximum width process rule for the layers used for the routes. The resistance checker assumes that the net route will be split in stripes and takes that into consideration for calculating the resistance value. You then need to use the **Stripe Wire** command (**Edit > Stripe Wire** from the layout window),

which will automatically split the fat wire into multiple parallel stripes. Stripe wire objects have stripe attributes that can be modified in the attribute dialog. See *Striping Wires*).

Cleaning Up the Layout after Routing

Once routing is completed in the design, whether partially or fully, you can run a sign off physical verification tool to validate the layout compliance with foundry rules. This is a required step before sending the design to the foundry. Custom Compiler tools have interfaces to several physical verification tools, such as IC Validator. Once the physical verification job is set up and executed, the debugging environment for the same tool (ICV VUE for instance) is launched, in case there are violations to review and fix. The debugging window can be used to cross-probe to specific locations in the layout and highlight the violations.

The Custom Compiler environment includes a DRD Autofix feature, which you can access using **Window > Toolbars > DRD** from the layout window. By drawing a region over the area that has the violations, DRD Autofix detects the potential violations and attempts to fix them with minimal changes to the layout.

You can also use the whole range of Custom Compiler manual editing capabilities to correct the DRC.

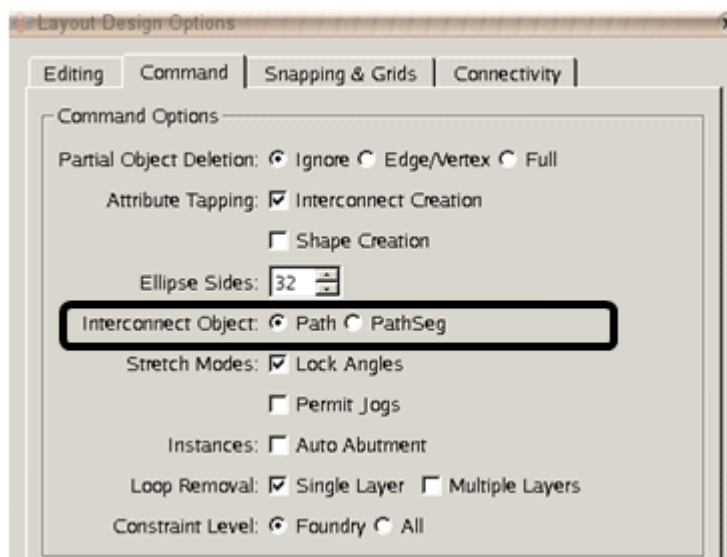
Working with Routing Objects in an ICC-Custom Compiler Flow

When using Paths or Rectangles as routing objects, you might encounter some issues. Therefore, you should use PathSeg as the routing object for the ICC-Custom Compiler flow using Custom Router (Router) applications whenever possible.

Custom Compiler routing supports an option to choose **Path** or **PathSeg** as the routing objects for various route creation commands such as **Create > Interconnect** and **Routing > Interactive Route**.

To set Path or PathSeg objects, choose **Options > Design** in the layout window to bring up the **Layout Design Options** dialog box (Figure 4).

Figure 4 Setting Path/PathSeg



In the **Command** tab, choose **Path** as the **Interconnect Object** (shown in Figure 4).

Paths

Note that when Path is used as the routing object, you might encounter the following issues:

- *Known Issue: False DRC violations*

For nets in an IC Compiler design, shielding is performed in the Custom Compiler environment and the design is verified in the IC Compiler tool using `verify_zrt_route -check_from_user shapes true`. The tool flags "Diff net var rule" spacing violations between shield and signal routing.

These violations are false, because the IC Compiler tool does not honor shield attributes on the Path objects.

Solution: Use PathSeg as the routing object to avoid false DRC violations.

- *Known Issue: Adding buffer on long routes*

The IC Compiler `add_buffer_on_route` command does not support routes created using Path objects.

Solution: Use PathSeg as the routing object.

Rectangles

Rectangles might be created on the design to address various design needs, such as fixing DRC errors. However, the IC Compiler tool does not support rectangles as part of routing, so it is necessary to convert Rectangles to Wires before routing. This can help avoid issues in later ICC applications and commands; for example, if the nets associated with routing have to undergo an ECO flow in the IC Compiler tool, the IC Compiler command `remove_net` will fail if there are Rectangle objects.

To convert Rectangles to Wires, use the script found in SolvNetPlus article #000025968, ["Script to Convert Rectangle Shapes to Wires."](#)

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

7

Digital Timing Shells for Analog Blocks

This chapter describes how to create digital timing shells for analog blocks to export them to the IC Compiler tool.

You can create timing shells for running IC Compiler place and route. Digital timing shells are created for analog blocks to export the analog block to the IC Compiler tool with a timing model to use the block in the next level of hierarchy for top-level implementation in the IC Compiler tool.

Using timing shell methodology, an analog block is wrapped with a thin digital logic. The advantages to this method are that

- no layout is required
- analog can be a black box
- no SPICE simulation needed
- covers all operating conditions
- automated support for multi-voltage
- model view is not dependent on analog cell changes

The IC Compiler tool is used for placement and routing of the timing shell. This includes placement of standard cells, black boxes (size and floor plan constraints are defined in Custom Compiler tools), and pins. The output of the place-and-route flow is a placed-and-routed Milkyway representation of the timing shell, with a power/ground network created using UPF annotation.

A block abstraction can also be generated for the timing shell. This block abstraction is the timing model and helps to perform an accurate timing analysis at the next level up in the hierarchy where the block will be instantiated. The black box can then be edited in the Custom Compiler environment to perform the layout of the analog content.

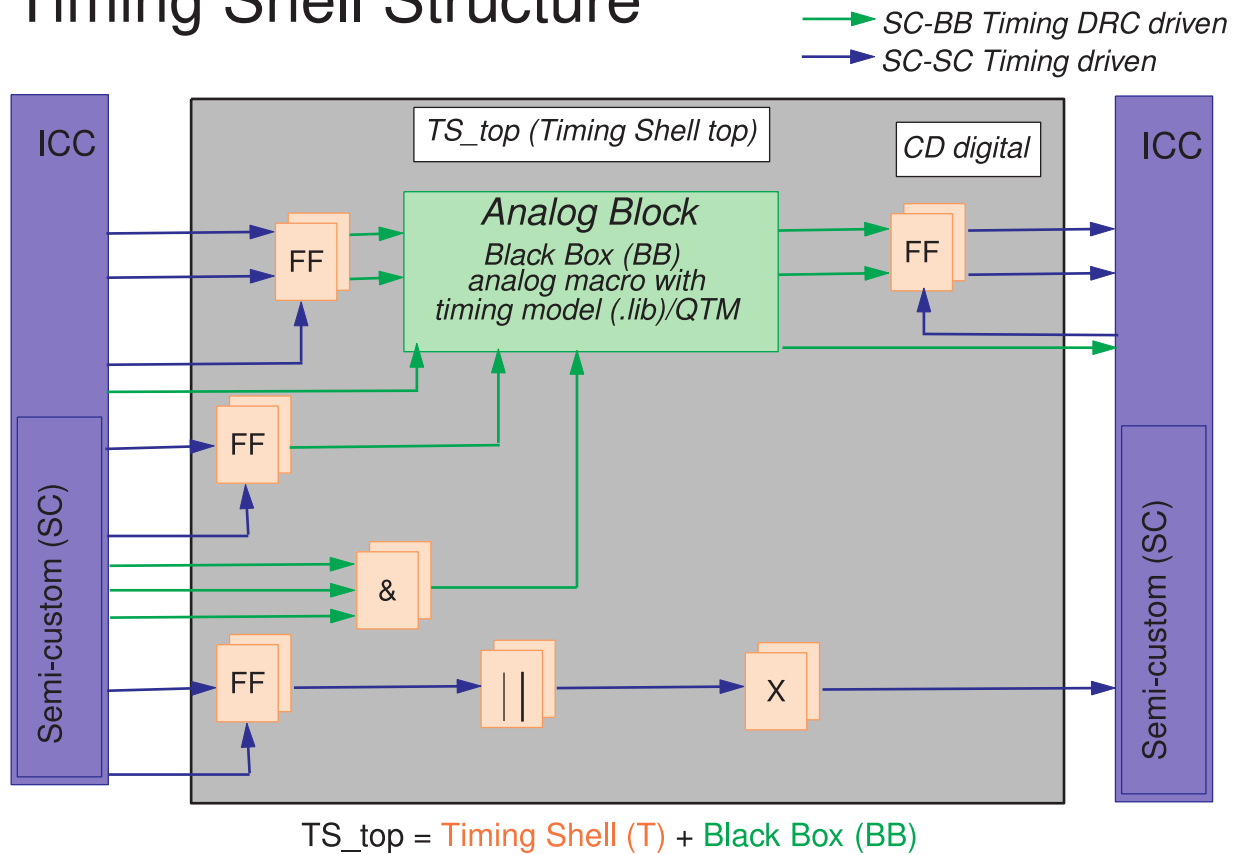
These sections cover:

- [Timing Shells](#)
- [Creating Timing Shells for Analog Blocks](#)

Timing Shells

A timing shell represents a mixed-signal design with one or more analog blocks (black boxes) and a semi-custom digital shell wrapped around those full-custom blocks. A digital timing shell mainly consists of flip-flops, which enables creation of a block abstraction in the IC Compiler tool. At the chip level, the IC Compiler tool performs an accurate timing analysis using the block abstraction of the timing shell block.

Timing Shell Structure



In Timing Shell Structure shown, the green box contains the analog block within a black box. The gray region is the digital timing shell, which can contain:

- semi-custom components (e.g. gates, interconnects, pins)
- instances of black boxes (BB)

The digital timing shell cannot contain non-semi-custom components, such as PCells.

All timing paths end in the timing shell in registered in- and outputs. No timing path can end on black-box pins (design-rule driven only).

Creating Timing Shells for Analog Blocks

Before you perform place and route for digital blocks with the IC Compiler tool, you can create digital timing shells for analog blocks. The analog block is treated or defined as a black box during place and route. Only the analog block's power, ground, and signal pin connections interact with the digital design during place and route.

Define black boxes by calculating their dimensions and specifying pin connections for the power and ground pins that will connect the analog block with the surrounding design. You can define black boxes with single power and ground connections, or UPF multiple power and ground connections.

You can use this procedure both to define a new black box, and to edit an existing black box.

The following sections describe an IC Compiler and Custom Compiler Co-Design flow that begins with a design schematic. The flow consists of the following steps:

- [Preparing to Create a Timing Shell](#)
- [Creating Connection Definitions for the Timing Shell](#)
- [Defining the IC Compiler Black Box](#)

Preparing to Create a Timing Shell

Before you create a timing shell, you might want to create timing libraries and/or modify the Place and Route command file to create a block abstraction. The section [Customizing the IC Compiler Place and Route Command File](#) describes how to modify the Place and Route command file.

1. (Optional) If you plan to use multiple power supplies using power supply definitions or a UPF file, or you want to create a block abstraction, set up timing libraries.
 - If timing libraries now exist, use the preferences in [Table 9](#) to specify them.
 - If timing libraries do not exist, you can create them. See [Creating Timing Libraries for Multi-Voltage Power in Place and Route](#).

[Table 9](#) lists and describes preferences for setting up existing timing libraries. For additional assistance in using these preferences, contact your Synopsys Support representative.

Table 9 *Preferences for Setting Up Timing Libraries*

Preference	Description
<code>leICCSearchPath</code>	Sets the value of <code>ICC_SEARCH_PATH</code> in the IC Compiler template file, which is in turn appended to the IC Compiler variable <code>search_path</code> . The IC Compiler tool uses it to resolve relative <code>.db</code> file names in the link/target_library.
<code>leICCLinkLibrary</code>	Tcl list of <code>.db</code> file names. The list should contain the <code>.db</code> files for standard cell libraries used in the design. Sets the <code>ICC_LINK_LIBRARY</code> variable in the IC Compiler template file.
<code>leICCTargetLibrary</code>	Tcl list of <code>.db</code> file names. The list should contain the <code>.db</code> files for standard cell libraries used in the design. Sets the <code>ICC_TARGET_LIBRARY</code> variable in the IC Compiler template file.

2. (Optional) If you want the system to create a block abstraction after place and route, set the `ICC_CREATE_TIMING_MODEL` variable to true in the IC Compiler Place and Route command file.

Then specify the values for the `TLUplus` files.

[Table 10](#) lists and describes the preferences you need to use to set up block abstractions.

Table 10 *Preferences for Setting up Block Abstraction*

Preference	Description
<code>leICCTLUPlusMapFile</code>	Sets the value of the <code>TLUPLUS_MAP_FILE</code> in the IC Compiler template. Specifies the name of the file that defines the layer mapping between the IC Compiler technology file and the Interconnect Technology Format (ITF) file.
<code>leICCTLUPlusMinFile</code>	Sets the value of the <code>TLUPLUS_MIN_FILE</code> in the IC Compiler template. The IC Compiler tool uses this <code>TLUplus</code> file for minimum condition calculation of resistance and capacitance.
<code>leICCTLUPlusMaxFile</code>	Sets the value of the <code>TLUPLUS_MAX_FILE</code> in the IC Compiler template. The IC Compiler tool uses this for maximum condition calculation of resistance and capacitance.

Creating Connection Definitions for the Timing Shell

Connection definitions enable the use of IP from different sources using multiple power supplies by mapping to power domains and connecting supply statements. For a multi-power-supply flow that uses UPF file generation, the schematic needs to have connection definitions.

After preparing your timing libraries and creating the necessary views by modifying the IC Compiler Place and Route command file, you can create connection definitions.

1. Open the design schematic you want to use.
2. In the Schematic Editor, choose **Tools > ICC Link**.
3. Follow the instructions for Creating Connection Definitions in the *Custom Compiler Schematic Editor User Guide*.

See [Table 11](#) for details about connection definition mapping.

Table 11 How Connection Definitions Map to UPF Power Domains and Supply Nets

Connection Definitions	UPF Mapping	Example
All connection definitions	Supply nets/ports are created Note: ! is removed from the names	<pre>create_supply_port vdd create_supply_net vdd connect_supply_net vdd -ports vdd</pre>
stdcells connection definitions	Primary Power net Ground net Top Power Domain is created with these nets	<pre>set_domain_supply_net Top_PD -primary_power_net vdd -primary_ground_net vss</pre>
BlackBox with connection definitions	Supply nets are created and black box ports are connected to the supply nets	<pre>create_supply_port vdda create_supply_net vdda connect_supply_net vdd -ports vdda connect_supply_net vdda -ports {BB1/vdda}</pre>

Defining the IC Compiler Black Box

Optionally, you can create an IC Compiler black box. The black box is intended for use with an analog instance whose layout is not available yet. The timing shell also applies to a cell that has an analog cell and a digital interface around it.

Note:

Currently the flow does not support UPF generation for an analog cell that already has a completed layout.

1. (Optional.) Choose **ICC > Define BlackBox**, and select the instance you want to use.
2. In the **Create ICC Black Box** dialog box, begin to define the black box as described in [Table 12](#).

If you are editing an existing black box, the dialog box title shown is **Edit ICC Black Box**.

[Table 12](#) lists and describes the settings required to create or edit an ICC black box.

Table 12 *Create or Edit ICC Blackbox (top section of the dialog box)*

Setting	Description
Library	The library where the design resides, from the selected instance.
Cell	BlackBox (default), from the selected instance.
View	blackbox (default)
Technology	Adds technology details that supply layer information, for example, layer constraints for signal pins, and the layer used for power/ground pin creation. The system also uses the technology to determine the <code>minWidth</code> of a layer when you specify the Use Default (3x minWidth) setting in the Add/Edit pin dialog box.
Size (Option provided by the IC Compiler tool)	Width/Height – Calculates the black box dimensions in microns from the width and height you enter here. For example: 10 x 10. You can use this option when you want the black box to be a rectangle and you know the size. Gate Count – Calculates the black box dimensions from the Gate Count and Utilization you enter here. You can use this option if you know the approximate size in numbers of gates. Boundary – Calculates the black box dimensions, in microns, from the coordinates you enter here. For example: {{0 0} {0 200} {230 200} {230 0}}. You can use this option when you have a non-rectangular boundary already defined in the layout.

3. In the **Create ICC Blackbox** dialog box under **Pins**, continue by designating supply pins.

- If you are creating a new blackbox cellview, the signal pins are copied from the placed master of the selected instance. The power/ground pins are copied from the inherited connections of the schematic hierarchy below the selected instance.
- If you are editing an existing blackbox definition, the pins are loaded from the existing blackbox cellview. If the schematic design does not contain a full schematic implementation of the blackbox cellview, you might need to explicitly add power/ground pins (see the step that follows).

Alternatively, you can copy the pins from an existing cellview. To copy pins, click **Extract**, enter details about the source view as described in [Table 13](#), and click **OK**.

Table 13 *Extract and Copy Pins for a Black Box (middle section of the dialog box)*

Setting	Description
Library	The library that contains the source design view.
Cell	The cell of the source design view.
View	The view of the source design.
Hierarchy	View Search List – Defines the list of views to search to find the master of the source design. Separate each view name with a space. Stop List – Defines the view that stops the search where the power supply is defined. This view must have either a terminal or net connection definition.

- In the **Create ICC Blackbox** dialog box under **Pins**, optionally add, edit, and remove power and ground pins.
 - To add pins, click **Add**, enter details as described in [Table 14](#), and click **OK**.
 - To edit details for a power or ground pin in the list, select the pin name, click **Edit**, adjust details as described in [Table 14](#), and click **OK**.
 - To remove a pin from the list, select it by name in the list, and then choose **Remove**.

Table 14 *Add or Edit Pin Connections for a Black Box (middle section of the dialog box)*

Setting	Description
Name	The name of the pin.
Type	Power – Specifies that this pin is a power pin.

Setting	Description
	Ground – Specifies that this pin is a ground pin.
	Layer – The one you choose from the alphanumerically sorted list of metal layers in the drop down menu.
Pin Size	Use Default (3x minWidth) – Applies the default size shown.
	Width – Applies a non-default size to the pin with the width you specify.
	Height – Applies a non-default size to the pin with the height you specify.
Connection Assignment	Name – The name of the connection definition created for the new terminal.
	Default Net – The name of the default net for the connection definition.

5. In the **Create ICC Blackbox** dialog box under **Pin Constraints**, complete the black box definition as described in [Table 15](#).

Table 15 Create or Edit ICC Blackbox (lower section of the dialog box)

Setting	Description
Pin Constraints	Allowed Layers – The layers you want the IC Compiler tool to use for creating signal pins. The system finds these in the technology library. Layers listed here correspond to the alphabetically sorted list of all OpenAccess physical layers for which oaMaterial=metal. Preroute Spacing – Designates the spacing between pins in the black box. Can be 0 or greater. No Stacking – Controls signal pin stacking. Options include no stacking for all_pins, stacking for pg_pins_only or signal_pins_only, or stacking_allowed. Bus Ordering – Designates the bus order for the IC Compiler tool to recognize. Choices are lsb_to_msb, msb_to_lsb, scrambled, or consistent_wirelengths. Keep Buses Together – Specifies that buses be kept together

6. In the **Create ICC Blackbox** dialog box that shows details for the black box you have defined, click **OK**.

The system creates the black box view with this definition.

If you edited a black box view, the system updates its definition.

7. Run Place and Route.

Continue by [Developing Digital Blocks with Co-design](#).

8

Developing Digital Blocks with Co-design

This chapter describes how to develop digital blocks.

You can implement and develop digital blocks in your mixed signal designs. To do this, use the IC Compiler tool to place and route the blocks through the tool's interface with the Custom Compiler environment.

You can optionally choose to create a block abstraction during the place and route process.

You can create a custom process flow with these additional procedures:

- You can add floorplanning conditions when you want to cause the IC Compiler tool to recognize and respect them during place and route. See [Adding Floorplanning Conditions for Digital Blocks](#).
- After you run place and route, you can customize IC Compiler processing by editing the IC Compiler command file. [Customizing the IC Compiler Place and Route Command File](#).

These sections cover:

- [Adding Floorplanning Conditions for Digital Blocks](#)
- [Implementing Digital Blocks](#)
- [Customizing the IC Compiler Place and Route Command File](#)

Adding Floorplanning Conditions for Digital Blocks

From the Custom Compiler environment, you can perform IC Compiler place and route using the created layout with floorplanning conditions as a template.

This is an IC Compiler tool and Custom Compiler environment Co-Design flow.

To add floorplanning conditions:

1. In the Custom Compiler environment, open the design layout.
2. Create floorplanning conditions.

Floorplanning conditions can include, for example, rectilinear boundaries to physically constrain the digital block placement area, straps for pre-routing the instances, and pin placement to constrain pin locations. Straps are paths or path segments, which can be connected to power or ground.

3. From the Custom Compiler environment, perform IC Compiler place and route. See [Implementing Digital Blocks](#).

Implementing Digital Blocks

You can implement digital blocks by using the IC Compiler tool to run place and route, and to perform other operations on mixed signal designs.

These sections cover:

- [Running IC Compiler P & R from Design Schematics or Verilog Netlists](#)
- [Tracking Progress and Viewing Output Files for IC Compiler Operations](#)
- [Designing with Place and Route Results](#)

Running IC Compiler P & R from Design Schematics or Verilog Netlists

You can run place and route with the IC Compiler tool using either a design schematic or a Verilog netlist for the design source.

This process produces a new block based on the schematic, and uses, when available, layout shapes to define floorplanning conditions, as imported from the design layout.

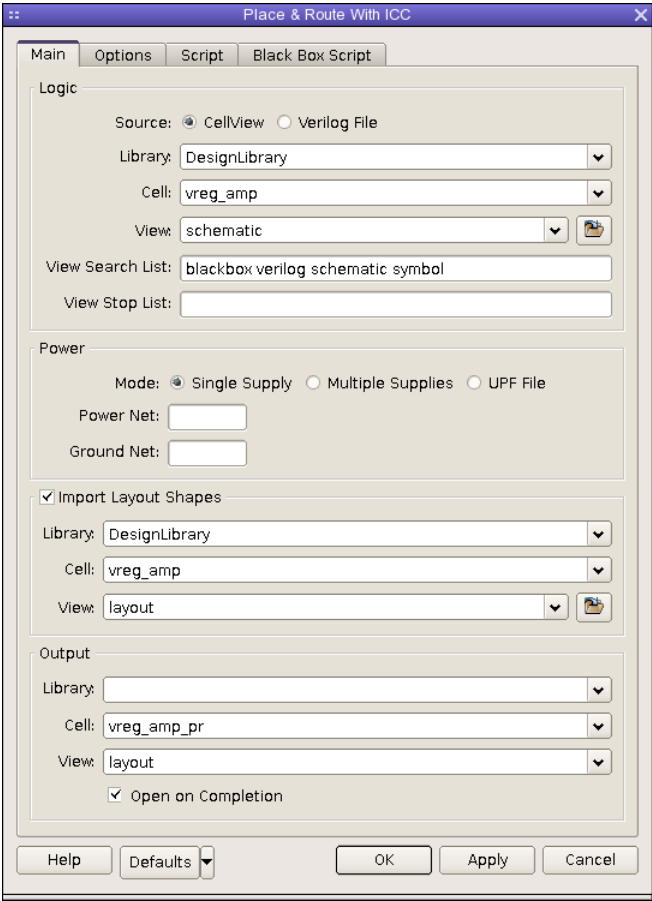
Prepare for place and route by [Preparing Libraries for Co-Design](#).

If you want to specify multiple or UPF power supplies, create a timing shell before running place and route. (See [Creating Timing Shells for Analog Blocks](#).)

If you want to include floorplanning conditions, you can add these to your design layout before running place and route. (See [Adding Floorplanning Conditions for Digital Blocks](#).)

To run place and route:

1. In the Custom Compiler environment, open the schematic view of the design.
2. Initiate the IC Compiler place and route process:
 - a. Choose **Tools > ICC Link**.
 - b. Choose **ICC > Place and Route**. The **Place & Route with ICC** dialog box opens.



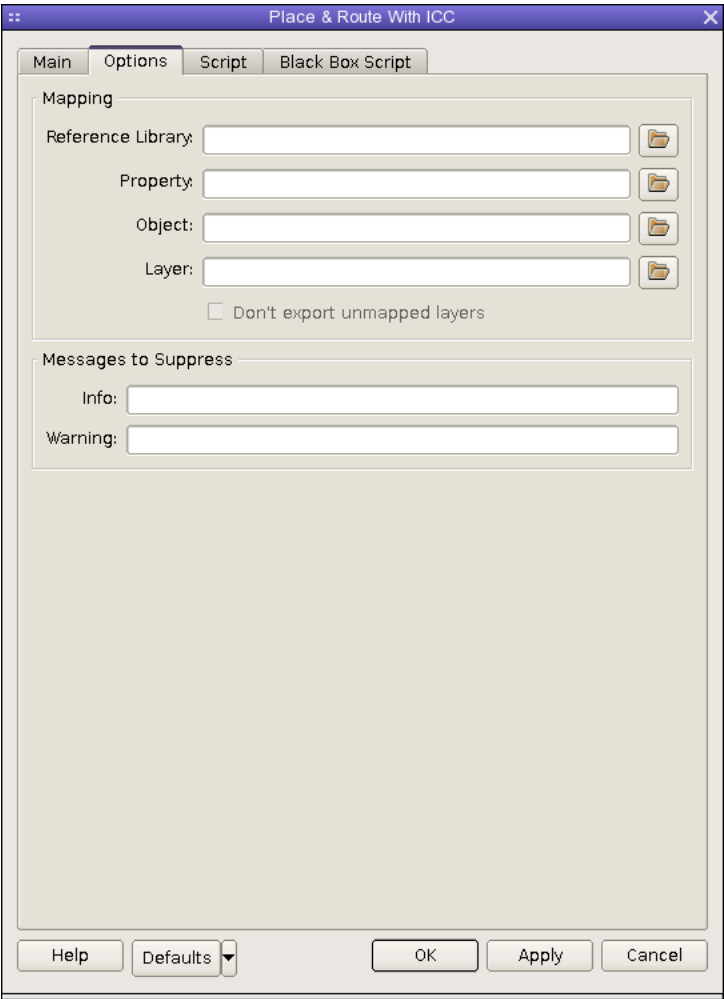
3. Enter settings on the **Main** tab.

Settings	Description
Logic	Specify the source type. If you have a cellview source, specify the Library , Cell , and View . Additionally specify a View Search List , and View Stop List , or accept the default lists.

Settings	Description
Power	If you have a Verilog File source, browse to select the file, and enter the Top Module name. Single Supply: Specifies a single power source with the Power Net and Ground Net you enter here. If your design has a single supply net without a black box, choose this option. If your design has a single supply net, and one or more black boxes, you can choose this option if the black boxes do not have multiple power and ground terminals. Multiple Supplies: Extracts the power network information from this design for outputting in UPF format. Running place and route once with multiple supplies generates a UPF file you can use to rerun place and route with unified power format. The Multiple Supplies - Domains tab shows the common power and ground connections shared by the gates in the design. The Multiple Supplies - Black Boxes tab shows the power and ground details for any black boxes. The Multiple Supplies - Supply Nets tab shows all power and ground details for the design. If your design has multiple supplies, with or without one or more black boxes, you must choose this option. When your design has inherited connections to define the supplies, you must choose this option to make sure inherited connections for power and ground nets are honored. UPF File: Uses the Unified Power Format (UPF) file you browse for and select here to define power and ground settings.
Import Layout Shapes	Specifies a Library, Cell, and View to use as a layout template for layout shapes. This setting is enabled by default, with the current design layout used as the template. See Adding Floorplanning Conditions for Digital Blocks.
Output	Specifies the IC Compiler target Library location for running the place and route process, and the Cell and View to produce as output. The tool appends the characters _pr to the cell name to prevent a naming conflict with the place and route layout template. If you are rerunning place and route, for example, with a UPF file generated by a previous run with multiple supplies, you can choose to rename the output cellview to retain the results of the previous run. Open on Completion optionally opens the design after the place and route process completes.

4. (Optional) Enter settings in the **Options** tab.

Setting	Description
Mapping	Specify the mapping files when the IC Compiler and Custom Compiler technologies are different, such as when the metal layers use different names and numbers. You can specify the mapping files for reference libraries, properties, objects, and layers.
Message to Suppress	Specifies the types of messages to hide: Info, and Warning.



5. On the **Script** tab, optionally view the default Tcl script template shipped with the tool, `icc-place_route-template.tcl`.

If you have created a custom Tcl script, you can browse to select it for this place and route job.

- 6. If this design contains a black box, on the **Black Box script** tab, optionally view the default black box Tcl script template shipped with the tool, `BlackBox.tcl`.
- 7. Click **OK**.

The tool extracts a Verilog netlist and then performs place and route on the design. See [Tracking Progress and Viewing Output Files for IC Compiler Operations](#) to continue.

Tracking Progress and Viewing Output Files for IC Compiler Operations

You can track progress, and view job output files, for all IC Compiler operations executed from the Custom Compiler environment.

To track progress and view output files:

- 1. Open the Job Monitor by choosing **Tools > Job Monitor** in the Layout Editor.
- 2. Select the job by name.
- 3. Right-click and choose **View Output**.

If you chose to open the design on completion when you set up the job, it opens in the Layout Editor.

If you did not choose to open it on completion, you can open the file `design_pr` to see the place and route results.

- 4. In the Job Monitor, you can view output files by clicking the tabs.

During place and route, the system creates a custom command file from the command template by applying the current settings. It netlists the design, runs place and route, and produces output files.

The tool stores the output files in the `iccbridge/place_route` directory by default.

[Table 16](#) lists and describes the output files.

Table 16 IC Compiler Place and Route Output Files

File	Description
<code>icc-place_route-template.tcl</code>	Supplied template for command file.
<code>icc-place_route.tcl</code>	Tcl script for this run.
<code>icc-place_route.log</code>	Log file for this run.

File	Description
<code><cellName>.v</code>	Verilog netlist for this run. If multiple power supply UPF power output was selected, power and ground net information are not included.
<code><cellName>.upf</code>	Netlist for multiple power supply output in UPF format for this run. Contains power and ground nets. You can use this file as input for rerunning UPF format place and route.

For timing shell black boxes, the system will create a Tcl file for each black box in the hierarchy. These black box Tcl files communicate boundary and pin constraints and initial pin shapes for power/ground and signal nets to the IC Compiler tool.

If you have a layout with floorplanning conditions set, the IC Compiler tool has recognized them during place and route.

If you defined a black box, the IC Compiler tool honors its boundaries, and recognizes its power and ground connections. To see the power and ground pins, descend into the black box instance.

5. (Optional) View the improved design. See [Designing with Place and Route Results](#) to continue.

Designing with Place and Route Results

You can design with place and route results by using both the Custom Compiler Layout Editor and the IC Compiler tool for viewing, editing, and optionally rerunning the place and route process.

To design with place and route results:

1. In the design layout on the Custom Compiler environment, view the initial routing results.

To obtain place and route results, see [Running IC Compiler P & R from Design Schematics or Verilog Netlists](#).

If you chose to open the design on completion when you set up the job, it opens automatically in the Layout Editor.

If you did not choose to open it on completion, you can open the file `design_pr` to see the place and route results.

2. (Optional) In the Layout Editor, edit the design.

3. When you are ready, choose **ICC > Open in ICC** to view the results in the IC Compiler tool.

Note:

This menu option is available only when an IC Compiler tool design is opened in LE.

If you edited the design in the Layout Editor, save changes in preparation for allowing the LE version to enter *read-only* mode.

The design opens in an IC Compiler tool window in *write* mode.

4. Continue to develop the design with the IC Compiler tool.

See the Synopsys IC Compiler tool documentation for instructions.

5. When you are ready, save changes and close the IC Compiler tool.

The Custom Compiler Layout Editor opens the design in *editing* mode, showing the recent changes.

6. Continue to develop the design with the Layout Editor.

Customizing the IC Compiler Place and Route Command File

This topic describes the following:

- [Workflow for Customizing the Command File](#)
- [Tcl Variables to Use in the Command File](#)

Workflow for Customizing the Command File

You can customize the IC Compiler command file to add execution of specific operations during place and route processes you run from the Custom Compiler environment.

After running place and route once, you can edit the command file, and run place and route again with custom settings.

For example, where your design has standard cells with instance placement already defined, you can edit the command file to perform route drawing only.

You can also enable IC Compiler to create a FRAM view from the block.

This is an IC Compiler and Custom Compiler Co-Design flow.

To use a custom command file to control place and route:

1. Begin by [Running IC Compiler P & R from Design Schematics or Verilog Netlists](#).
2. After running the place and route process once, open the command file at this location:
`iccbbridge/place_route/icc-place_route.tcl`
3. Copy this command to a file with a new name.
4. Edit the file.

Under the section **Operations to Perform**, set listed operations to true or false.

(Optional) Add IC Compiler commands for additional operations.

5. Save the file in the same directory to make it available from the **Commands** tab in the **Place and Route with ICC** dialog box.

For future place and route processes, you can browse to select the custom file from the **Commands** tab.

Tcl Variables to Use in the Command File

[Table 17](#) lists the variables you can use in the supplied place-and-route command file. The first two columns list the design operations and the IC Compiler commands you would use for those operations. The third and fourth columns list the equivalent Tcl variables you would use in the Tcl command file.

Table 17 IC Compiler Commands and Equivalent Tcl Variables

Operation: IC Compiler Command	IC Compiler II Command Options	Equivalent Tcl Variable	Equivalent Tcl Options
Floorplanning: initialize_floorplan	core_utilization	ICC2_RUN_FLOORPLAN	ICC2_UTILIZATION
	control_type		ICC2_CONTROL_TYPE
	core_offset		ICC2_CORE_LEFT ICC2_CORE_RIGHT ICC2_CORE_TOP ICC2_BOTTOM
	shape		ICC2_FLOORPLAN_SHAPE_TYPE
	side_length		ICC2_SIDE_LENGTH
Placement: create_placement, legalize_placement	—	ICC2_RUN_PLACEMENT	—

Table 17 IC Compiler Commands and Equivalent Tcl Variables (Continued)

Operation: IC Compiler Command	IC Compiler II Command Options	Equivalent Tcl Variable	Equivalent Tcl Options
Create PG rings: create_pg_ring_pattern	—	ICC2_CREATE_PG_RINGS*	—
Add Filler Cells: create_stdcell_filler	lib_cell prefix lib_cell prefix	ICC2_ADD_FILLER_CELL	ICC2_CHIP_FINISH_METAL_FILLER_LIB_CELL_LIST ICC2_CHIP_FINISH_METAL_FILLER_PREFIX ICC2_CHIP_FINISH_NON_METAL_FILLER_LIB_CELL_LIST ICC2_CHIP_FINISH_NON_METAL_FILLER_PREFIX
Add tap cells: create_tap_cells	lib_cell distance no_tap_cell_under_layers	ICC2_ADD_TAP_CELL	ICC2_TAP_MASTER_CELL ICC2_TAP_CELL_DISTANCE ICC2_TAP_AVOID_LAYERS
Create PG standard cells: create_pg_std_cell_conn_pattern set_pg_strategy compile_pg	—	ICC2_RUN_ROUTING	—
Run Routing: set_ignored_layers route_auto route_detail	min_routing_layer max_routing_layer max_number_iterations	ICC2_RUN_ROUTING	ICC2_MIN_ROUTING_LAYER ICC2_MAX_ROUTING_LAYER ICC2_SEARCH_REPAIR_ITERATIONS
Run DRC/LVS: check_routes check_lvs	—	ICC2_RUN_VERIFICATION	—

* By default, the Custom Compiler environment derives and uses the metal1 and metal2 layers and their default widths for creating PG rings.

9

Tutorial: Constraint Rules Mapping

This chapter provides a tutorial introduction to the IC Compiler to Custom Compiler round-trip flows.

The design used in this tutorial is a mixed-signal design where the top design has a completed floor plan and initial block placement using the IC Compiler tool. The design contains these design elements:

- dcc -Duty Cycle Converter, an analog block implemented in the Custom Compiler environment using reference40nm PDK
- I106 - mixed signal block
- cpu|interrupt - digital block

As a part of the tutorial, you will:

- Review existing non-default rules in IC Compiler design.
- Create new constraints and rules in the Custom Compiler environment.
- Save the design back to the IC Compiler database and review the rules in the IC Compiler tool.

This tutorial instructs you to do the following:

- [Before You Start](#)
- [Task 1: Analyze the Design Using the IC Compiler Tool](#)
- [Task 2: Add IC Compiler Libraries to the Custom Compiler Environment](#)
- [Task 3: Open the Design Layout](#)
- [Task 4: Review NDRs with the Custom Compiler Layout Editor](#)
- [Task 5: Edit and Create NDRs and Shield Constraints](#)
- [Task 6: Evaluate Edits with the IC Compiler Tool](#)
- [Task 7: Edit Routing Rules in the IC Compiler Tool](#)

- [Summary](#)
- [Limitations](#)

Before You Start

Make sure you are using the following product versions for this tutorial:

Product	Version
Custom Compiler	M-2017.03 or later
IC Compiler	K-2016.03 or later

Task 1: Analyze the Design Using the IC Compiler Tool

In this task you will analyze the IC Compiler version of the design to understand the objects in the design.

1. Launch the IC Compiler tool:

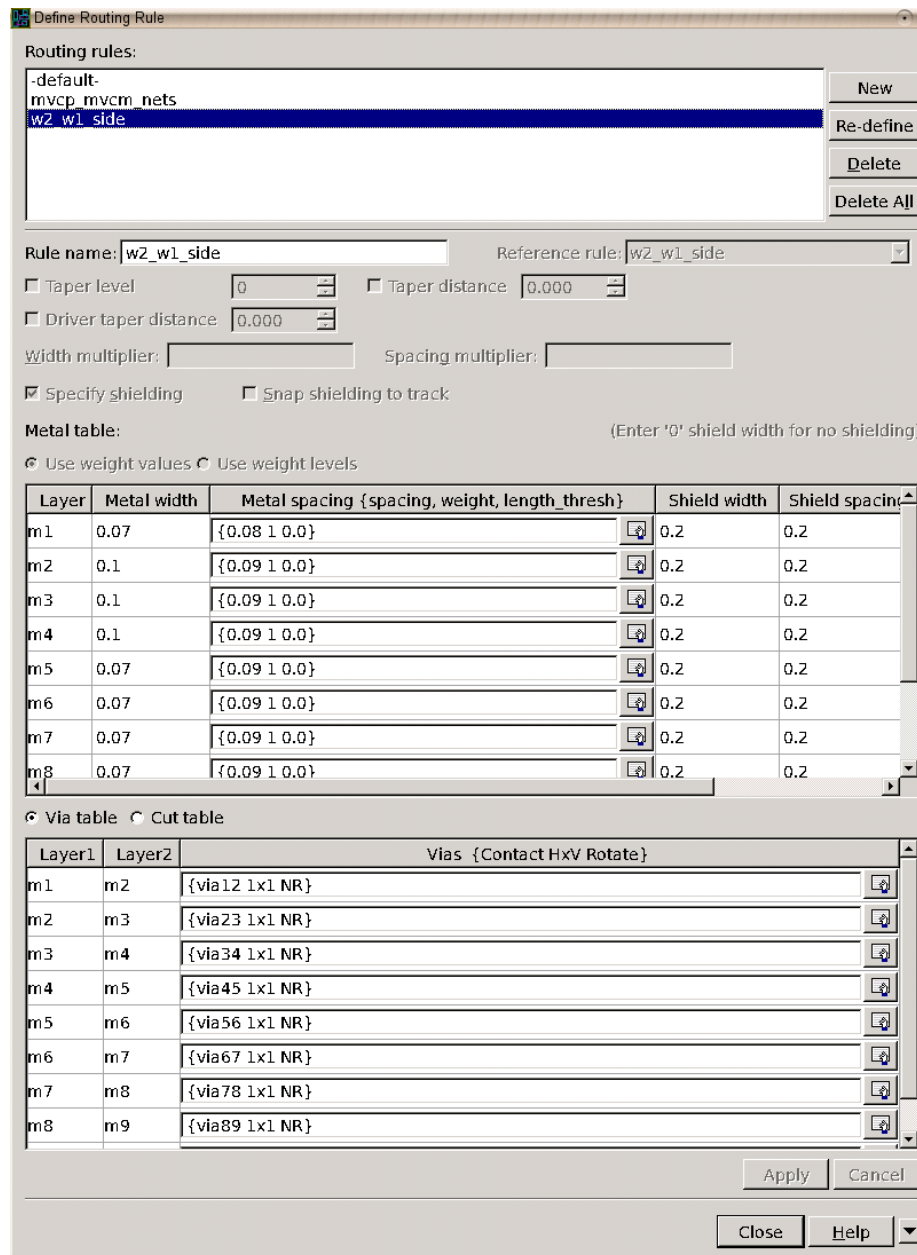
```
% icc_shell -gui &
```

2. Choose **File > Open Design** and:

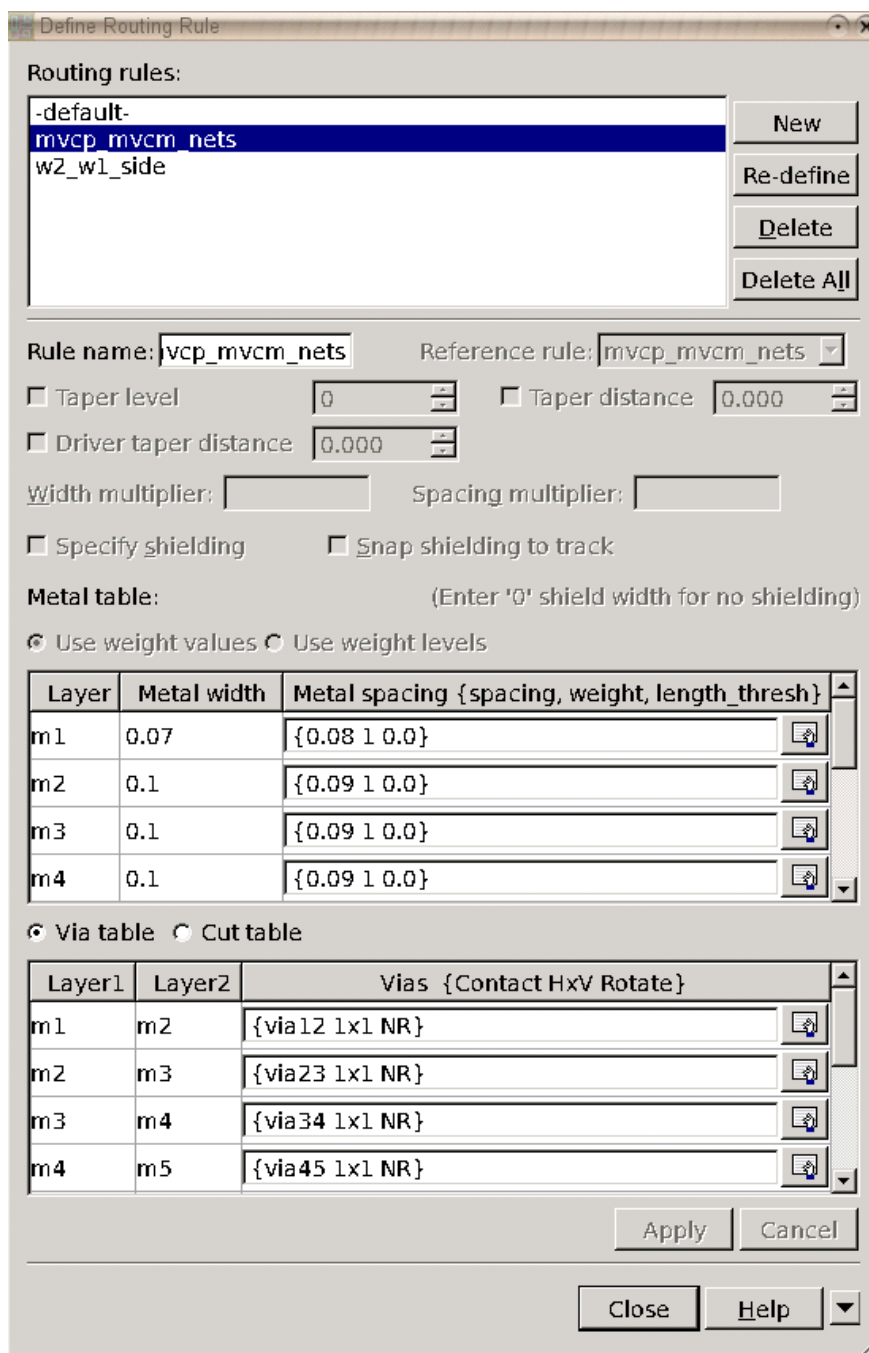
- a. Select **chip_mstop_icc** for the **Library Name**.
- b. Select **cpu_preRoute** for **CEL**.

Two non-default rules (**w2_w1_side** and **mvcp_mvcm_nets**) are defined in the design. Review these rules in the following steps.

3. Choose **Define Routing Rule (Route > Routing Setup > Define Routing Rule)**.
 - a. Click on the **w2_w1_side** rule.
 - b. Observe that layers m2, m3, m4 have non-default rules for width set to 0.1u and shield width and spacing 0.2u defined for all layers.



- Click on the **mvcp_mvcm_nets** rule.
- Observe that layers m2-m5 have non-default rules for width set to 0.1u.



The image shows the 'Define Routing Rule' dialog box in the IC Compiler tool. It contains several sections for configuring routing rules.

Routing rules: A list box containing '-default-', 'mvcp_mvcm_nets' (selected), and 'w2_w1_side'. To the right are buttons: 'New', 'Re-define', 'Delete', and 'Delete All'.

Rule name: 'vcp_mvcm_nets'. **Reference rule:** 'mvcp_mvcm_nets' (dropdown).

Taper level: 0. **Taper distance:** 0.000. **Driver taper distance:** 0.000.

Width multiplier: [empty]. **Spacing multiplier:** [empty].

Specify shielding: ☐. **Snap shielding to track:** ☐.

Metal table: (Enter '0' shield width for no shielding). Radio buttons: ☒ Use weight values, ☐ Use weight levels.

Layer	Metal width	Metal spacing {spacing, weight, length_thresh}
m1	0.07	{0.08 1 0.0}
m2	0.1	{0.09 1 0.0}
m3	0.1	{0.09 1 0.0}
m4	0.1	{0.09 1 0.0}

Via table: Radio buttons: ☒ Via table, ☐ Cut table.

Layer1	Layer2	Vias {Contact HxV Rotate}
m1	m2	{via12 1x1 NR}
m2	m3	{via23 1x1 NR}
m3	m4	{via34 1x1 NR}
m4	m5	{via45 1x1 NR}

Buttons at the bottom: 'Apply', 'Cancel', 'Close', 'Help'.

e. Close the **Define Routing Rule** window.

4. Type the following IC Compiler commands to find the nets associated with the routing rules you just reviewed:

```
icc_shell> get_nets -filter "var_route_rule == w2_w1_side"{vreg}
```

```
icc_shell> get_nets -filter "var_route_rule == mvcp_mvcm_nets"{mvcm  
mvcp}
```

5. Close the IC Compiler design.

Task 2: Add IC Compiler Libraries to the Custom Compiler Environment

In this task, you need to add the IC Compiler libraries for this design to the Custom Compiler environment to make them available for capturing design changes.

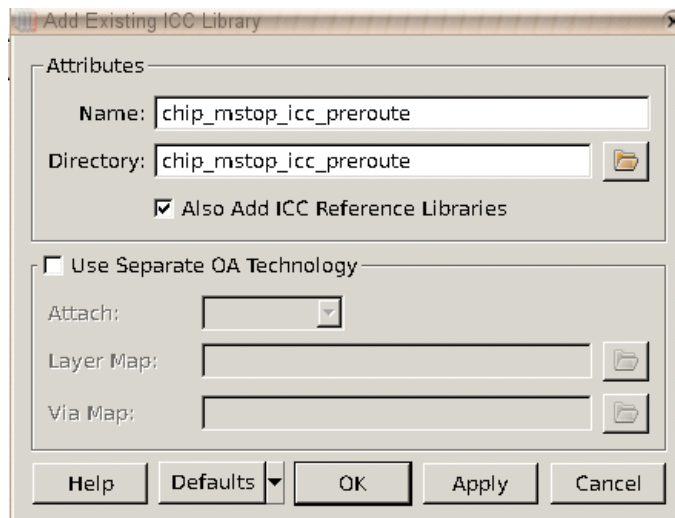
1. Start the Custom Compiler.

```
% custom_compiler &
```

2. Choose the **Library Manager** tool.
3. In the **Library Manager**, choose **File > Add ICC Library**.

This opens the **Add Existing Library** dialog box with default values.

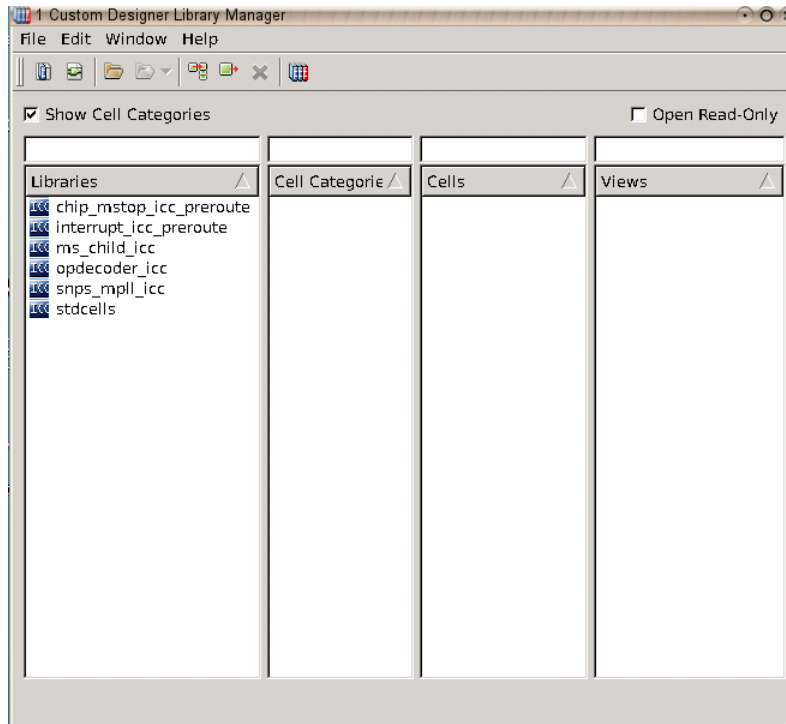
4. Using the browse button, choose the **chip_mstop_icc_preroute** library, which adds the name to the **Directory** field.
5. Specify the **chip_mstop_icc_preroute** in the **Name** field.



6. Click **OK**.

The system adds this library and lists it in the `libs.defs` file.

7. The **Library Manager** now lists the IC Compiler library `chip_mstop_icc_preroute` and its reference libraries available to capture changes.



Task 3: Open the Design Layout

In this task you will open a design originating in the IC Compiler tool and analyze the layout design editor configuration when opened directly.

1. In the **Library Manager** window, click on the `chip_mstop_icc_preroute/cpu_preRoute/layout`.
2. Double-click on the layout view to open the IC Compiler design, which opens the design in a layout window.

Task 4: Review NDRs with the Custom Compiler Layout Editor

In this task, you will learn how non-default rules (NDRs) created with the IC Compiler tool are mapped for use on the Custom Compiler environment.

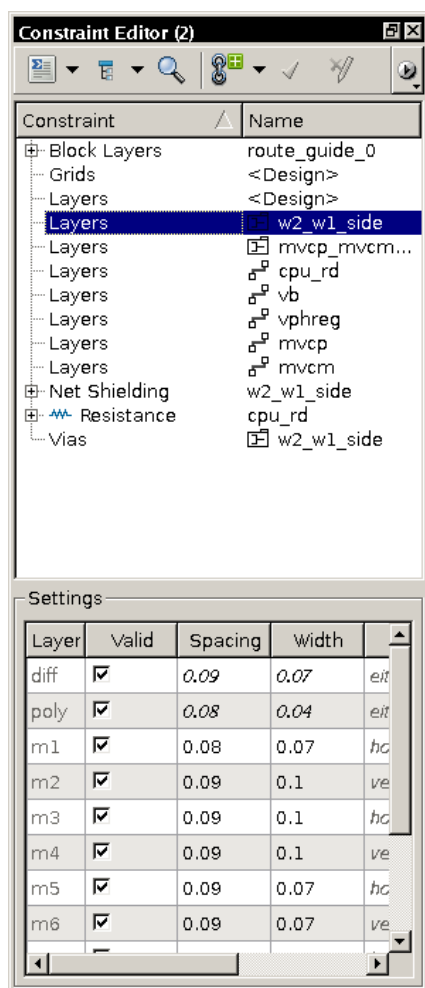
1. Open the Constraint Editor assistant.

- a. Click the **Layers** constraint associated with Group **w2_w1_side**. This is equivalent to an IC Compiler non-default rule.

Note:

For each IC Compiler non-default rule that is applied to any net, the Constraint Editor assistant creates a net group with the same name and adds the nets in compliance with the non-default rule to the group.

- b. Observe that width values match IC Compiler values.

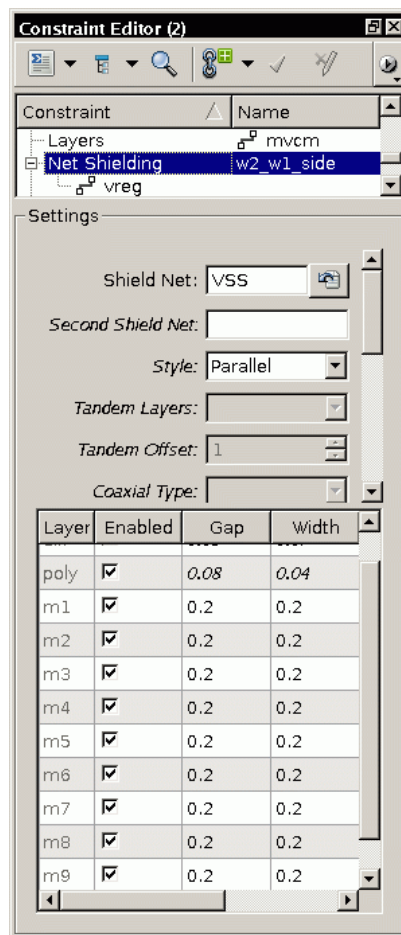


The shield spacing/width values associated with the IC Compiler NDR are also mapped to a shielding constraint.

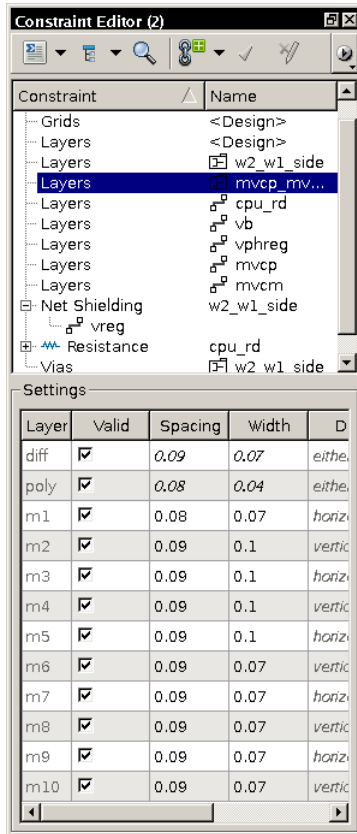
Note:

If the IC Compiler design has a single net/group of nets belonging to an NDR that defines shield width and spacing, then the NDR will be mapped to a net group with the same name and a shield constraint in the Custom Compiler environment with a shield net name. The shield net name will be set to the default ground net name.

- c. Click and expand the **Net Shielding** constraint associated with Group **w2_w1_side**.
- Observe that **Shield Net** is set to VSS, which is the ground net.
 - Observe that shield width and spacing values match IC Compiler values.



2. Similarly review the **Layers** constraint associated with the **mvcp_mvcm_nets** non-default rule. Observe that width values match IC Compiler values.



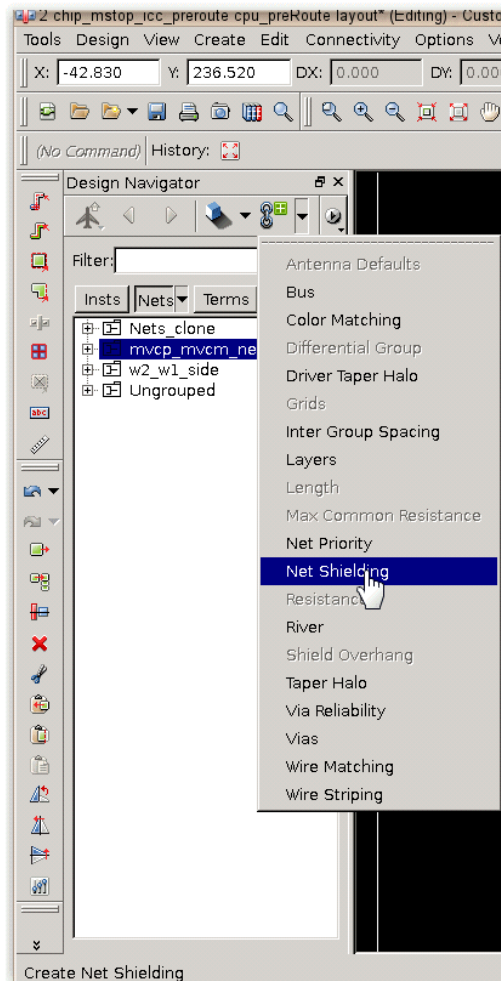
Task 5: Edit and Create NDRs and Shield Constraints

In this task, you will edit the design by creating new non-default rules (NDRs) and constraints using the Custom Compiler environment's unique features.

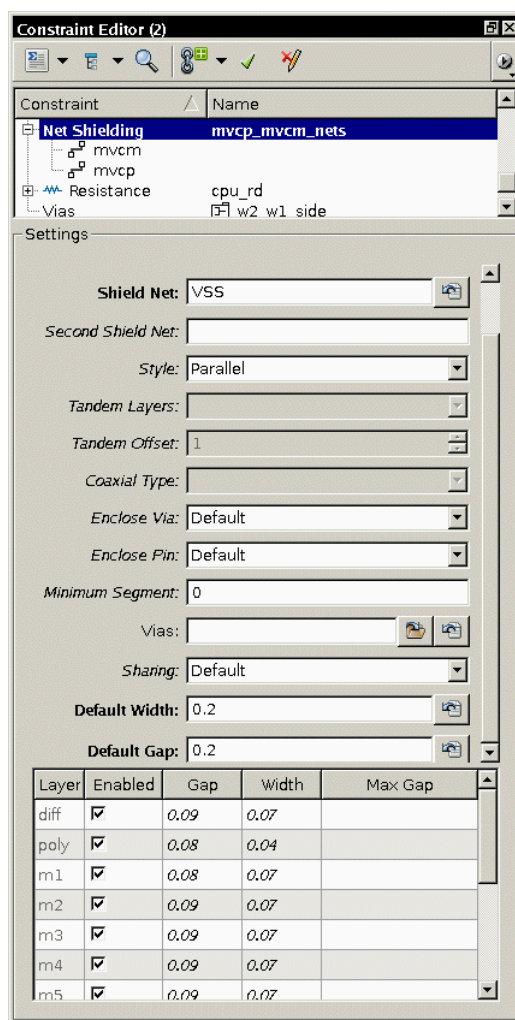
Beginning with the latest version of the Custom Compiler environment, for IC Compiler designs:

- Shield constraint creation is supported for net groups. The shield constraint name option will not be available.
 - If the net group originated from IC Compiler through a routing rule, then when a new shielding constraint is created on that net group in the Custom Compiler environment, the shielding constraint is added to the routing rule when saving the design.
 - If the net group is created in the Custom Compiler environment, then on design save a new IC Compiler NDR is created with same name as the net group, with the signal and shield NDR values.
 - Shield constraint creation is restricted for individual nets.
1. To create a shielding constraint for net **mvcm**, the shielding constraint has to be created on the net group by following these steps:
 - a. Open the **Design Navigator** assistant.
 - b. From the Nets menu () , choose **Net by Group**.
 - c. Select the net group **mvcp_mvcm_nets**.

- d. Click **Create Constraint** , and then choose the **Net Shielding** constraint to create a net shielding constraint for the selected net.



2. Modify the shielding constraint using the following steps:
 - a. Open the Constraint Editor.
 - b. Select the **Net Shielding** constraint associated with the group **mvcp_mvcm_nets**.
 - c. Edit the **Shield Net**, **Default Width**, and **Default Gap** options.

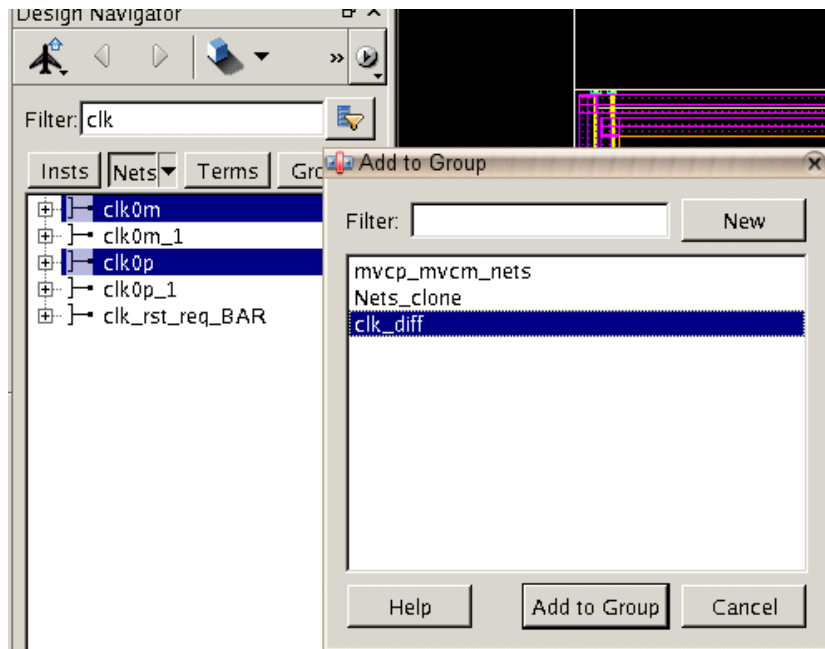


- d. Click **Apply** to save the changes.
3. In the following steps you will create an NDR and shield constraints for nets clk0p and clk0m. Because shield creation is restricted for nets, you will create a net group called clk_diff and add nets clk0m and clk0p to this net group.

For nets clk0p and clk0m, create a net group and assign nets to it by following these steps:

- a. Open the **Design Navigator** assistant.
- b. Click Nets () to display the Nets menu.
- c. Choose **All Nets**.

- d. In the **Nets** list, select the nets **clk0p** and **clk0m** by name.
- e. In the Design Navigator toolbar, click **Add to Group**.
- f. In the **Add to Group** dialog box, click **New**.
- g. In the text box, enter the name **clk_diff** for the group.
- h. Select the group **clk_diff**.
- i. Click **Add to Group**.



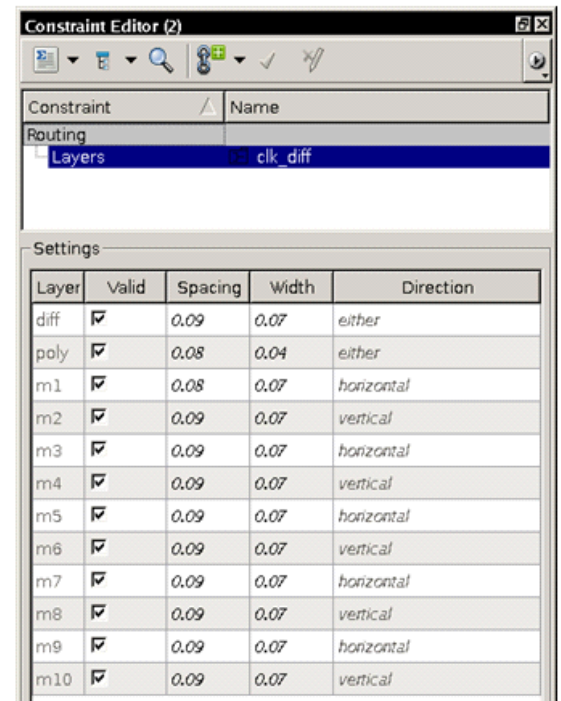
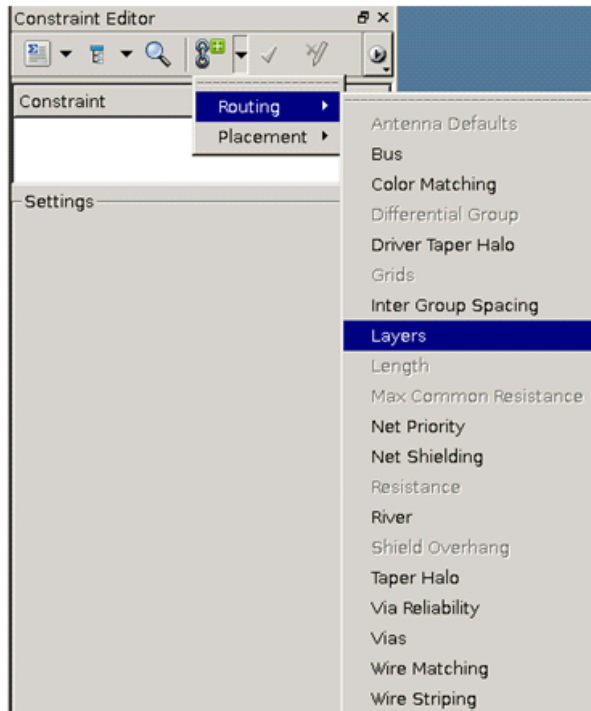
- j. Close the dialog box.
4. For the net group **clk_diff** created in previous steps, create an NDR with the following steps.

In the Design Navigator assistant:

- a. Choose the **Nets** drop-down menu using the drop-down arrow
- b. Choose **Net by Group**.
- c. Select the **clk_diff** net group.

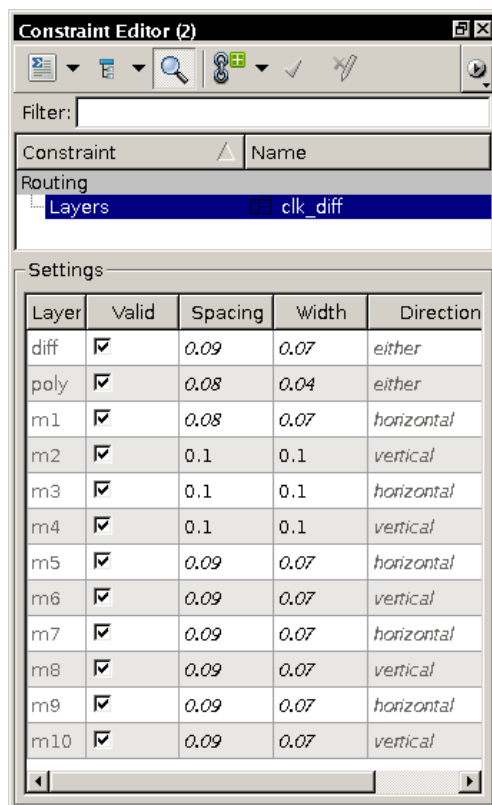
5. Open the Constraint Editor assistant.

- Observe that there is no NDR defined on the net and thus the Constraint Editor assistant does not show any constraints.
- Click **Create Constraint**  to open the list of available constraints.
- Select **Routing** from the list, which opens the list of available routing constraints.
- Choose **Layers** from the list, which adds the Layer constraint.

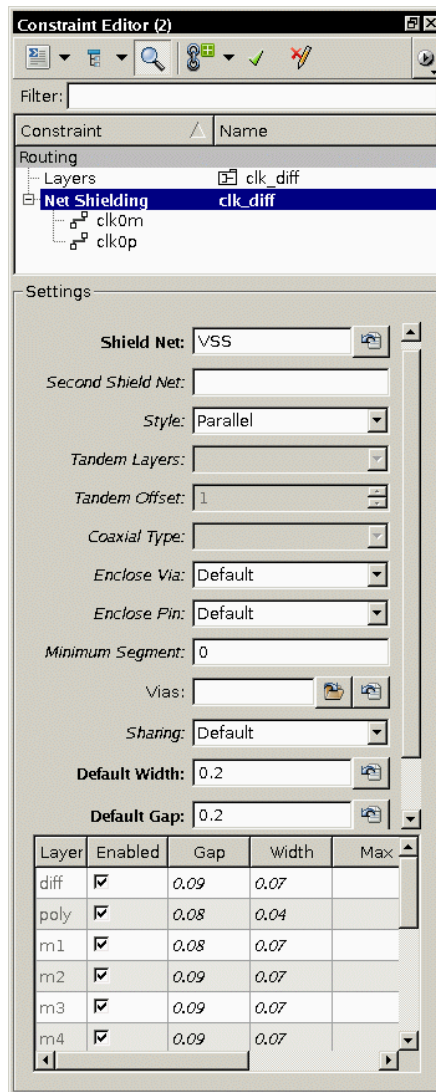


6. In the Constraint Editor, modify the Layer constraints as follows:

- a. Set the **Spacing** and **Width** of layers m2, m3, and m4 to 0.1u.



- b. Click **Apply** to save the changes.
7. For the net group **clk_diff** created in previous steps, create a shielding constraint with the following steps.
 - a. In the Design Navigator assistant, select the **clk_diff** net group and create a shielding constraint.
 - b. In the Constraint Editor assistant, select the shielding constraint associated with **clk_diff**.
 - c. Edit **Shield Net**, **Default Width**, and **Default Gap** options.



- d. Click **Apply** to save the changes.
8. For the net **vphreg**, change the default width for metal layers m2 through m5 from 0.07 to 0.2.
9. Choose **Design > Save** to save the changes made to the design.
10. Choose **Design > Close** to close the design.

Task 6: Evaluate Edits with the IC Compiler Tool

In this task you will evaluate the edits you have performed in the Custom Compiler Layout Editor. You will check to make sure they are synchronized by using the IC Compiler tool.

1. Launch the IC Compiler tool using the following Linux command.

```
%icc_shell -gui &
```

2. From **File > Open Design**:
 - a. Select **chip_mstop_icc_preroute** for **Library Name**.
 - b. Select **cpu_preRoute** for **CEL**.

You will review the routing rules for net **mvcm**.

3. Use the IC Compiler commands below to find the nets associated with the routing rule **mvcp_mvcm_nets**:

```
get_nets -filter "var_route_rule == mvcp_mvcm_nets" {mvcp mvcm}
```

Net **mvcm** remains part of the routing rule because a new shielding constraint was created for the net group in the Custom Compiler environment.

4. Choose **Define Routing Rule (Route > Routing Setup > Define Routing Rule)**.

- Click on the **mvcp_mvcm_nets** rule.
- Observe that metal width, spacing are same as original route rule and shield width, spacing 0.2u defined for all layers (as defined in [Task 5: Edit and Create NDRs and Shield Constraints](#)).

Define Routing Rule

Routing rules:

- default-
- OA_NET_RULES_vphreg
- clk_diff
- mvcp_mvcm_nets**
- w2_w1_side

Rule name: mvcp_mvcm_nets Reference rule: mvcp_mvcm_nets

☐ Taper level 0 ☐ Taper distance 0.000

☐ Driver taper distance 0.000

Width multiplier: Spacing multiplier:

☒ Specify shielding ☐ Snap shielding to track

Metal table: (Enter '0' shield width for no shielding)

☒ Use weight values ☐ Use weight levels

Layer	Metal width	Metal spacing {spacing, weight, length_thresh}	Shield width	Shield spacing
m2	0.1	{0.09 1 0.0}	0.2	0.2
m3	0.1	{0.09 1 0.0}	0.2	0.2
m4	0.1	{0.09 1 0.0}	0.2	0.2
m5	0.1	{0.09 1 0.0}	0.2	0.2

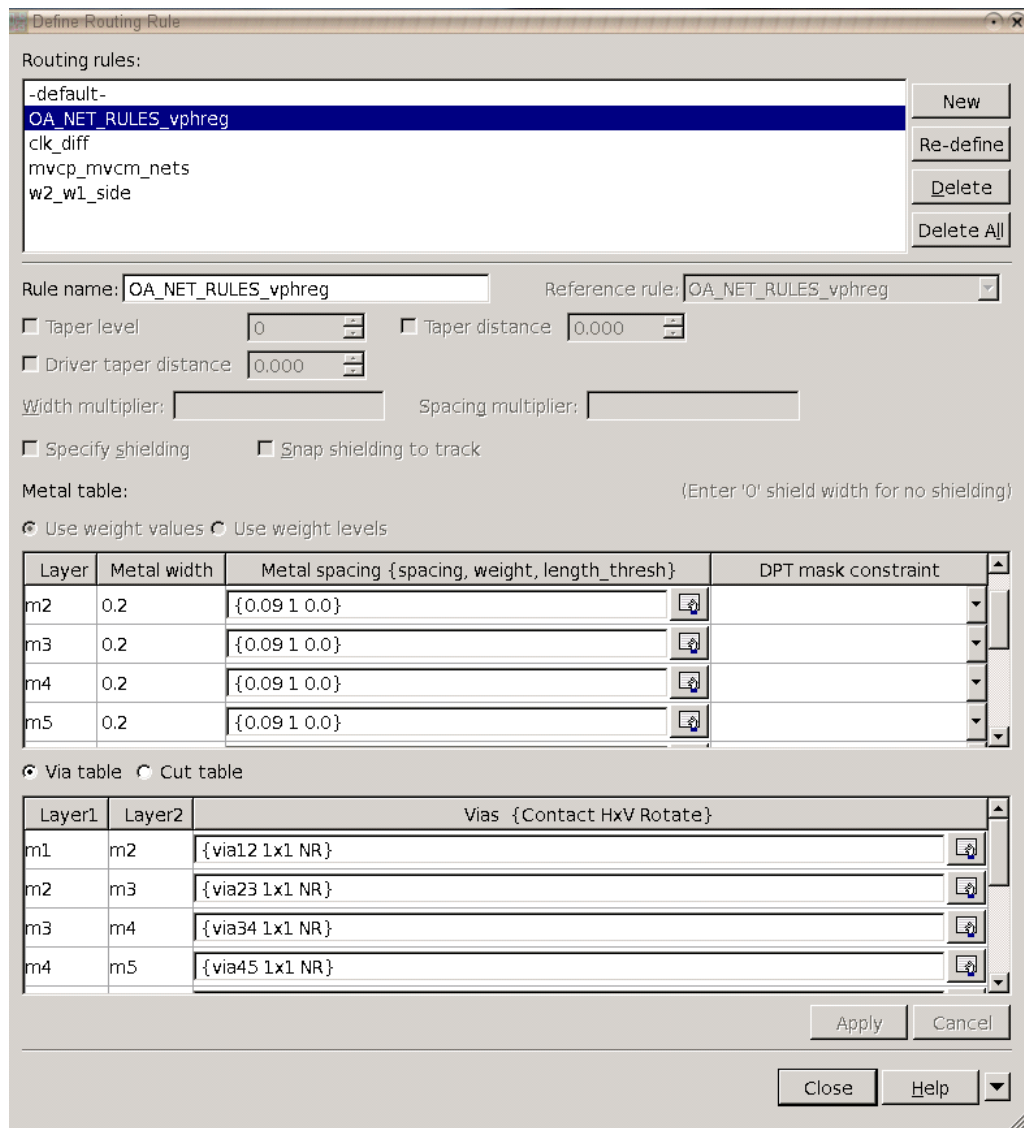
☒ Via table ☐ Cut table

Layer1	Layer2	Vias {Contact HxV Rotate}
m1	m2	{via12 1x1 NR}
m2	m3	{via23 1x1 NR}
m3	m4	{via34 1x1 NR}
m4	m5	{via45 1x1 NR}

Apply Cancel

Close Help

- Observe that there is a new routing rule called **OA_NET_RULES_vphreg**. When signal width/spacing is modified for a net, on design save the net will be assigned to a net-specific route rule (that is, the route rule name begins with "OA_NET_RULES_").
- Click on the **OA_NET_RULES_vphreg** rule and review the values.



- Observe that there is also a routing rule named **clk_diff**.
- Click on the **clk_diff** rule.

The **clk_diff** shielding group created in Task 5 is saved to IC Compiler as a routing rule with appropriate constraints, where the net routing rule contains all the nets assigned to the routing rule.

Define Routing Rule

Routing rules:

- default-
- OA_NET_RULES_mvcm
- clk_diff**
- mvcp_mvcm_nets
- w2_w1_side

Rule name: Reference rule:

☐ Taper level ☐ Taper distance

☐ Driver taper distance

Width multiplier: Spacing multiplier:

☒ Specify shielding ☐ Snap shielding to track

Metal table: (Enter '0' shield width for no shielding)

☒ Use weight values ☐ Use weight levels

Layer	Metal width	Metal spacing {spacing, weight, length_thresh}	Shield width	Shield sp
m1	0.07	{0.08 1 0.0}	0.2	0.2
m2	0.1	{0.10 1 0.0}	0.2	0.2
m3	0.1	{0.10 1 0.0}	0.2	0.2
m4	0.1	{0.10 1 0.0}	0.2	0.2
m5	0.07	{0.09 1 0.0}	0.2	0.2
m6	0.07	{0.09 1 0.0}	0.2	0.2

☒ Via table ☐ Cut table

Layer1	Layer2	Vias {Contact HxV Rotate}
m1	m2	{via12 1x1 NR}
m2	m3	{via23 1x1 NR}
m3	m4	{via34 1x1 NR}
m4	m5	{via45 1x1 NR}
m5	m6	{via56 1x1 NR}
m6	m7	{via67 1x1 NR}
m7	m8	{via78 1x1 NR}

9. Close the **Define Routing Rule** window.

10. Use the following IC Compiler command to validate the nets associated with this rule:

```
icc_shell> get_nets -filter "var_route_rule == clk_diff"
{clk0p clk0m}
```

Task 7: Edit Routing Rules in the IC Compiler Tool

In this task you will learn to edit the routing rules created in the Custom Compiler environment and save them back to the IC Compiler Tool.

1. Choose **Define Routing Rule (Route > Routing Setup > Define Routing Rule)**.
2. Click on the **OA_NET_RULES_vphreg** rule
3. Click **Re-define** to modify the rule
4. Change the **Shield width** and **Shield spacing** values for layers m2-m4 to 0.1u.

Define Routing Rule

Routing rules:

- default-
- OA_NET_RULES_vphreg
- clk_diff
- mvcp_mvcm_nets
- w2_w1_sdc

Buttons: New, Re-define, Delete, Delete All

Rule name: OA_NET_RULES_vphreg Reference rule: OA_NET_RULES_vphreg

☐ Taper level 0 ☐ Taper distance 0.000

☐ Driver taper distance 0.000

Width multiplier: Spacing multiplier:

☒ Specify shielding ☐ Snap shielding to track

Metal table: (Enter '0' shield width for no shielding)

☒ Use weight values ☐ Use weight levels

Layer	Metal width	Metal spacing (spacing, weight, length_thresh)	Shield width	Shield spacing	DPT mask cons
m1	0.07	{0.08 1 0.0}	0.07	0.08	
m2	0.2	{0.09 1 0.0}	0.1	0.1	
m3	0.2	{0.09 1 0.0}	0.1	0.1	
m4	0.2	{0.09 1 0.0}	0.1	0.1	
m5	0.2	{0.09 1 0.0}	0.07	0.09	

☒ Via table ☐ Cut table

Layer1	Layer2	Vias (Contact HxV Rotate)
m1	m2	{via12 1x1 NR}
m2	m3	{via23 1x1 NR}
m3	m4	{via34 1x1 NR}
m4	m5	{via45 1x1 NR}
m5	m6	{via56 1x1 NR}
m6	m7	{via67 1x1 NR}

5. Click **Apply** to save the changes.
6. Use the following IC Compiler command to validate the nets associated with this rule. You will receive the warning shown.

```
icc_shell> get_nets -filter "var_route_rule == OA_NET_RULES_vphreg"

"Warning: No net objects matched '*' (SEL-004)"
```

In IC Compiler, editing a routing rule basically redefines the routing rule and doing so causes the removal of the nets associated with the routing rule. Thus, after redefining the routing rule, the nets have to be reassigned the routing rule.

```
icc_shell> set_net_routing_rule vphreg -rule OA_NET_RULES_vphreg
```

7. Save and close the design.
8. Optionally, evaluate the edits in the Custom Compiler Layout Editor.
9. Exit the IC Compiler tool.

Summary

- IC Compiler NDRs (Non-Default Rules) are mapped to equivalent OA as a net group with appropriate constraints (non-default width and spacing, shield width and spacing) and the net group contains all the nets assigned to the route rule.
- When the IC Compiler design has a single ground net and if the IC Compiler design has an NDR that defines shield width and spacing, then the NDR will be mapped to a shield constraint in the Custom Compiler environment with shield net name set to ground net name.
- When the IC Compiler design has multiple ground nets and if the IC Compiler design has an NDR that defines shield width and spacing, then the NDR will be mapped to shield constraint in the Custom Compiler environment without a shield net name and you have to specify the desired shield net name for the routing purposes.
- When the IC Compiler design has a group of nets belonging to an NDR and for one of the nets (e.g., netA) a signal width/spacing is modified in the Custom Compiler environment, then netA is assigned to a net-specific routing rule (that is, the routing rule name begins with "OA_NET_RULES_") and is not part of the original routing rule and net group. This happens during design save.
- For IC Compiler designs, shielding constraints can be created only for net groups. Shielding constraint creation is disabled for nets.

Limitations

- If a shield or NDR constraint is created for a net in the Custom Compiler environment and if the design was saved to IC Compiler, then deleting the constraint does not delete the NDR rule in IC Compiler.
 - Use the IC Compiler `remove_routing_rules` command to remove the complete rule.
 - If the rule has to be partially removed, use the **Re-define** option in the **Define Routing Rule** window to update the rules.

A

Setting the User Environment

Custom Compiler and IC Compiler co-designing supports various flows. This appendix describes how to set up the Custom Compiler and IC Compiler environments for different scenarios of technology information availability.

This chapter covers:

- [Technology Mappings](#)
- [Export to the IC Compiler Tool](#)
- [Import From the IC Compiler Tool](#)
- [Pre-Route Flow](#)
- [Setup Using LEF Files](#)

Technology Mappings

This section provides information about the basic mapping between the Custom Compiler environment and IC Compiler technology layers and their display attributes.

Data Type Mapping

The IC Compiler tool's physical data are organized on layers and are assigned a *dataType*. In the Custom Compiler environment, the corresponding terms are *layer* and *purpose*.

The IC Compiler Layer specifies a layer with a data type number of 0. The Layer corresponds to the Custom Compiler Layer Purpose Pair (LPP) of the matching Custom Compiler layer with the Custom Compiler drawing purpose, which is a system-reserved purpose defined in the `techLayers` section of the Custom Compiler technology file.

Note:

Milkyway schema 7 and schema 8 support `extend_mw_layers`, which extends Milkyway database layer number support to 4095 layers, using layers 4001

through 4095 as the system-reserved layers. See the [IC Compiler user documentation](#) for more details.

Table 18 IC Compiler Tool -Custom Compiler Layer Mapping

IC Compiler Tech Rule Attributes	Custom Compiler Tech Constraint
<pre>Layer <layerName> { layerNumber = <value> } LayerDataType <dataTypeName>{ layerNumber = <value> dataTypeNumber = <dtvalue> }</pre>	<pre>techLayers((<layerName> <value>)) techPurposes((drawing -1 drw) (<dataTypeName> <dtvalue>)) NOTE: drawing is a reserved purpose name</pre>

When mapping from IC Compiler to the Custom Compiler environment, all the IC Compiler via<m> maskName mask names map to "cut" and all the metal<n> maskName mask names become "metal". The IC Compiler polyCont mask name also maps to "cut".

Table 19 IC Compiler Tool -Custom Compiler Layer Mapping

IC Compiler Tech Rule Attributes	Custom Compiler Tech Constraint
<pre>Layer <layerName> { layerNumber = <value> maskName = <value> }</pre>	<pre>layerRules(functions(;(layer function [maskNumber]) <layerName> <value> 30))))</pre>

The following table maps IC Compiler tool mask names to Custom Compiler material names.

Table 20 IC Compiler Tool -Custom Compiler maskName Mapping

IC Compiler Mask Name	Custom Compiler Material Name
poly	poly
via1...vian	cut
metal1.. metaln	metal

Table 20 IC Compiler Tool -Custom Compiler maskName Mapping (Continued)

IC Compiler Mask Name	Custom Compiler Material Name
nwell	nWell
pwell	pWell
diffusion	diffusion
polyCont	cut

Display Attribute Mapping

Custom Compiler Display Resource data, colors, lineStyles, stipples, packets and packetAliases, are all defined in the context of a "display" to display a design, such as a graphics display, laser printer or pen plotter.

IC Compiler display resource data has no context; only one set of display resource definitions exists in an IC Compiler library. The IC Compiler tool also lacks the concepts of "packet" and "packetAlias."

IC Compiler tool to Custom Compiler environment translation maps the Milkyway display resource data to the "display" of the corresponding Custom Compiler `display.drf` file.

Through packet and packetAlias definitions, the Custom Compiler display resource data defines the drawing attributes of the LPPs.

Table 21 IC Compiler Tool -Custom Display Attributes Mapping

IC Compiler Tech Rule Attributes	Custom Compiler Tech Constraint
<pre> Stipple "solid" { width = 16 height = 16 } Color 15 { name = "cyan" rgbDefined = 0 } Stipple "dot4" { width = 16 height = 16 } Layer "m1" { layerNumber = 8 visible = 1 selectable = 1 blink = 0 color = "cyan" lineStyle = "solid" pattern = "dot4" } </pre>	<pre> In display.tcl: dr::createLineStyle solid -display \$display_object -width 1 -pattern [list 1 1 1] dr::createColor cyan -display \$display_object -red 0 -green 255 -blue 255 dr::createStipple dot4 -display \$display_object -pattern {.....} dr::createPacket m1 -display \$display_object -stipple dot4 -lineStyle solid -fill cyan -outline cyan In Custom Compiler technology file: techDisplays (;(LayerName Purpose Packet Vis Sel Con2ChgLy DrgEnbl Valid) ;(-----) (m1 drawing m1 tt nil tt))) </pre>

Export to the IC Compiler Tool

This topic covers the setup required while exporting a Custom Compiler database to create a new IC Compiler database or write to an existing IC Compiler database.

The exporting scenarios are:

- [Scenario 1: Without Available IC Compiler Technology or Design Data](#)
- [Scenario 2: To an Existing IC Compiler Library with Matching Layers](#)
- [Scenario 3: To an Existing IC Compiler Library With Different Layer Names](#)
- [Scenario 4: To an Existing IC Compiler Library with No Matching Layers](#)

Scenario 1: Without Available IC Compiler Technology or Design Data

In this scenario, there is a Custom Compiler design library. There is no IC Compiler related data available; that is, there is no IC Compiler technology file or IC Compiler library.

For the given Custom Compiler library, the user requirement could be one of the following:

- [Generate an IC Compiler Technology File](#)
- [Generate IC Compiler Tech File and Library](#)

Generate an IC Compiler Technology File

The `cd2icc_tech` executable can be used for this purpose.

This can be run outside of the Custom Compiler environment from a Linux terminal, which allows you to quickly create an IC Compiler technology file.

The following is an example that exports an IC Compiler tech file from a Custom Compiler library.

```
% cd2icc_tech -oaLib snps_mpll -techFile mw.tf
```

Note:

The Custom Compiler library must have the tech database or a tech library reference in order to generate an IC Compiler technology file.

Generate IC Compiler Tech File and Library

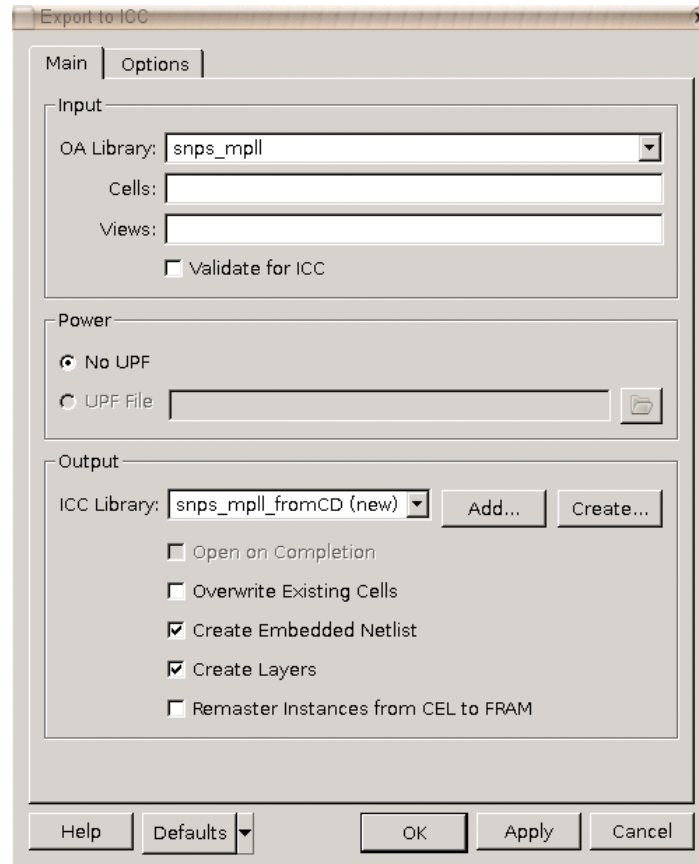
To create an IC Compiler technology file and a library, the Export To ICC feature must be used.

This feature creates an IC Compiler technology file and also an IC Compiler library for the given Custom Compiler library.

You can choose **File > Export > To ICC** from the Console window, which allows you to export the Custom Compiler library to an IC Compiler library.

Figure 5 shows a Custom Compiler library specified under the **Input** section of the **Main** tab. The **OA Library** you specify here is exported to an IC Compiler library, which is specified in the **Output** section under **ICC Library**.

Figure 5 *Export To ICC Dialog Box*



During the export process, the tool creates the `./iccbridge` directory, which contains all the output files that are generated.

The exported IC Compiler technology file can be identified by the name `./iccbridge/cd2icc_tech.tf`.

The IC Compiler library is created and placed under the CWD directory.

Scenario 2: To an Existing IC Compiler Library with Matching Layers

In this scenario, there is a Custom Compiler design and a technology database exists for it. There is an IC Compiler library available to export the design from the Custom Compiler environment into the existing IC Compiler library.

The tech database for both the Custom Compiler environment and the IC Compiler tool matches in all respects as follows:

- The layer names/numbers are same for both the IC Compiler and Custom Compiler versions.
- For the Custom Compiler purposes associated with each layer, there is an equivalent IC Compiler dataType of the same name associated with the layer.

During the Export process, the tool checks to see if the Custom Compiler layer names in the tech database match with the IC Compiler layer names.

If the layer names are matched, then no layer is created and they are automatically mapped by name.

Similarly, for the purpose, if an IC Compiler dataType name already exists in the destination technology file, no creation is necessary.

There would then be no environment setup required for this case.

Example:

For the Custom Compiler design library `snps_mpl1`, the LPP m1 drawing is on lpp number 8:-1.

There is already an IC Compiler library `snps_mpl1_icc`, which has the metal1 layer (identified by the mask name "metal1") with layer name m1 on layer number 8 and dataType 0.

Similarly, the other layers also match.

When the library is exported to IC Compiler, the tech layer match is identified by the layer names. A mapping file is generated during the process.

The layer mapping file is available in the `iccbridge` directory. The file is called `layer.map`.

For the given example, the `layer.map` will look like this:

```
CDLayerName CDLayerPurpose ICCLayerNumber ICCLayerDatatypeNumber m1
drawing                        8                0
```

Scenario 3: To an Existing IC Compiler Library With Different Layer Names

In this scenario, there are existing Custom Compiler design and tech databases. There is also an IC Compiler library available for export.

Caution:

In this case the layer names are mismatched between the Custom Compiler and IC Compiler technology databases.

By default, during the export process, for the Custom Compiler layers that do not have an IC Compiler layer with same name, a new layer is created in the IC Compiler technology database.

During the export process, the tool checks the Custom Compiler layer names in the tech database against the IC Compiler layer names.

- If the layer number is not already in use in the destination IC Compiler technology file, a new layer is created in the destination IC Compiler technology file using the source layer name and number.
- If the layer number is in use, a new layer is created in the destination IC Compiler technology file with the source layer name and an unused layer number.

Note:

IC Compiler tool layer names are limited to 31 characters. If an OA layer name is longer than this, then the name is truncated to 27 characters and '_n' is appended where 'n' is 1 to 3 digits and is chosen, in order to generate a unique name.

To avoid creating new layers during the export process, the required setup would be to create a `layer.map` file to resolve the layer name mismatches.

Example:

In this example, the layer names do not match between the Custom Compiler and IC Compiler tech data.

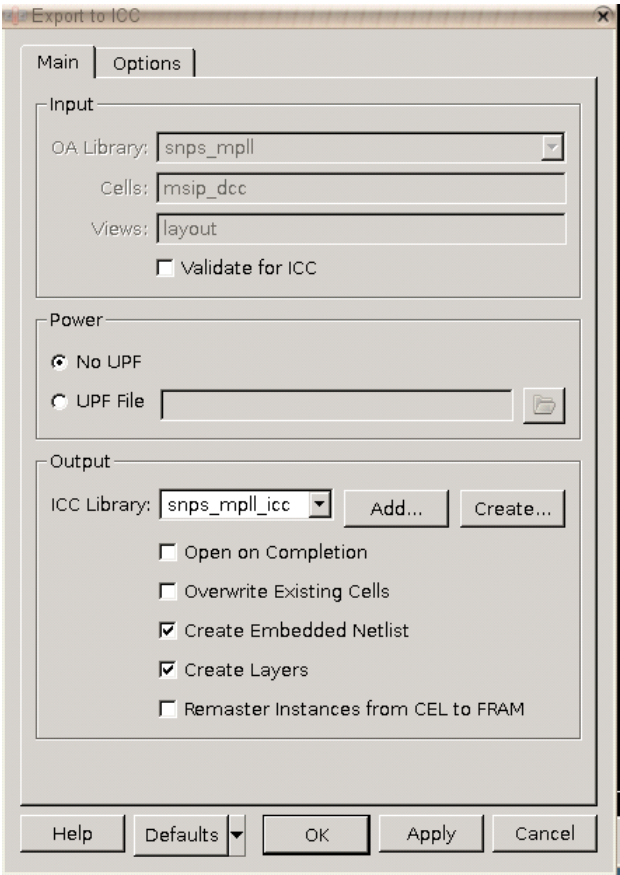
In the Custom Compiler library `snps_mpl1`, metal layers are on the LPPs: m# drawing, where #=1, 2... 10.

In the IC Compiler library `snps_mpl1_icc`, metal layers are on the IC Compiler tech layers: M#: dataType 0, where #=1, 2,...,10 respectively.

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	M1 (8) : dataType 0

Custom Compiler Layers	IC Compiler Layers
m2(10):drawing (-1)	M2(10): dataType 0
.....	
m10(85):drawing(-1)	M10(85):dataType 0

The design `snps_mpll/msip_dcc/layout` is exported to `snps_mpll_icc`, as shown.



If no layer map file is provided during export, a `layer.map` file (similar to the one below) is generated by the tool.

CDLayerName	CDLayerPurpose	ICCLayerNumber	ICLayerDatatypeNumber
m1	drawing	52	0
m10	drawing	51	0
m2	drawing	50	0
m3	drawing	48	0
m4	drawing	47	0

m5	drawing	46	0
m6	drawing	45	0

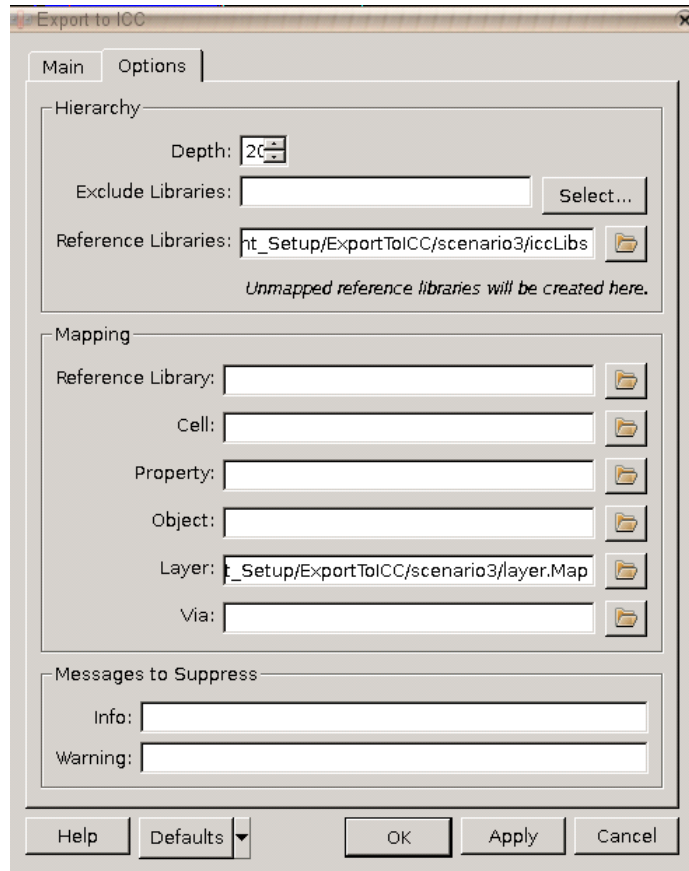
For the Custom Compiler LPPs m#: drawing, which do not have an IC Compiler layer with same name, a new layer is created in the IC Compiler technology database on the available unused layers.

For the given example, to make sure that the tool uses the existing IC Compiler layers, a layer mapping file must be used to achieve the correct layer mapping.

The `layer.map` file used to resolve the mismatch is shown below:

CDLayerName	CDLayerPurpose	ICCLayerNumber	ICLayerDatatypeNumber
m1	drawing	8	0
m2	drawing	10	0
m3	drawing	12	0
m4	drawing	14	0
m5	drawing	70	0
m6	drawing	72	0
m7	drawing	74	0
m8	drawing	76	0
m9	drawing	78	0
m10	drawing	85	0

The layer map file has to be specified in the **Export To ICC** dialog box, in the **Options** tab as shown:



During the export process, the tool uses the user-specified layer mappings to map the given Custom Compiler data to IC Compiler data.

Scenario 4: To an Existing IC Compiler Library with No Matching Layers

In this scenario, there is a Custom Compiler design and a tech database. There is also an IC Compiler library available for export.

In this case, there are some Custom Compiler LPPs that do not have an equivalent IC Compiler layer: dataType match.

There are two methods that allow for creation of the missing layers/purposes in the target IC Compiler technology:

- [Using the Automated Method of Exporting to IC Compiler](#)
- [Using the Layer Mapping File](#)

A third method involves creating a new Technology File and editing the existing IC Compiler technology file to include the missing layers and purposes.

Using the Automated Method of Exporting to IC Compiler

During the export process, the tool checks the Custom Compiler layer names in the tech database against the IC Compiler layer names:

- If the layer number is not already in use in the destination IC Compiler technology file, a new layer is created in the destination IC Compiler technology file with the source layer name and number.
- If the layer number is in use in the destination IC Compiler technology file, then a new layer is created in the destination IC Compiler technology file with the source layer name and an unused layer number.

The same process is also applied for layer purposes.

- If the purpose number is not already in use in the destination IC Compiler technology file, a new dataType is created in the destination IC Compiler technology file with the source purpose name and number. Otherwise, a new purpose is created in the destination IC Compiler techfile with the source purpose name and an unused dataType number.
- IC Compiler LayerDataType names are constructed from the Custom Compiler purpose as '<purpose>_cd####', where 'purpose' is the name of the Custom Compiler purpose.

Note:

Datatype names in MW are limited to 31 characters. If an OA purpose with more than 27 characters is found, then the name used in MW is truncated to 27 characters and '_nnn' is appended, where 'nnn' is 1 to 3 decimal digits selected to be unique for all the datatypes created.

Example:

For the Custom Compiler library `snps_mpl1`, the tech and design database has

- shapes on the LPP "m# drawing," where #=1,2,3 and 4
- pin shapes on the LPP "m# pin".

The IC Compiler library `snps_mpll_icc` is available for export, and its technology database contains the equivalent metal layer definitions for LPPs: "m# drawing," but no `dataType` definitions for the purpose pin.

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	M1 (8) : dataType 0
m1(8):pin (251)	m1 (8):??
m2(10) :drawing(-1)	m2(10):dataType 0
m2(10):pin (251)	m2 (10):??

When the design `snps_mpll/msip_dcc/layout` is exported from the Custom Compiler environment to the IC Compiler tool, the tech layer matching is resolved by the layer names.

The pin purpose is a reserved purpose in Custom Compiler and has the purpose number as 251. So, in the IC Compiler tool, this is mapped to metal layer number: `dataType 251`.

The generated layer map file looks like the following:

m1	pin	8	251
m2	pin	10	251
m3	pin	12	251
m4	pin	14	251

A new `dataType pin_cd###` is created for each metal layer, which has a pin purpose as shown :

```
Layer "m1" {
  layerNumber    = 8
  maskName       = "metall"
  .....
}
LayerDataType "pin_cd011" {
  layerNumber    = 8
  dataTypeNumber = 251
  .....
}
```

Using the Layer Mapping File

To map the missing layers in the IC Compiler tool to a specific layer number and `dataType`, you can provide an explicit mapping using the `layer.map` file.

If the destination layer number/`dataType` number provided in the `layer.map` file does not exist, it is created.

Example:

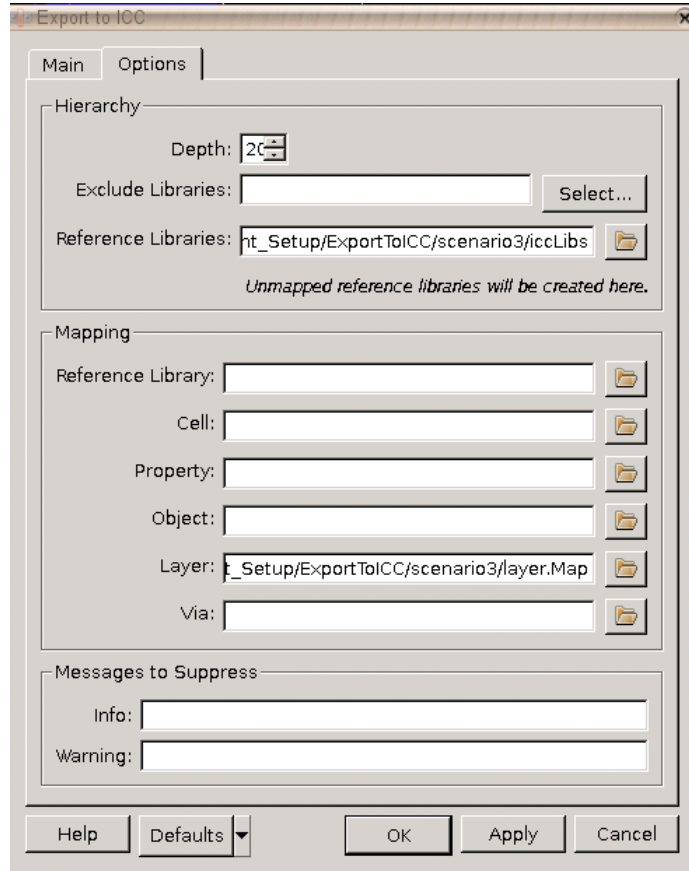
For the example described in Example 3, assume that you need to map the LPP "m# pin" to IC Compiler layer m#(#):dataTypeName(1).

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	m1 (8) : dataType 0
m1(8):pin (251)	m1(8): pin (1)

So, a `layer.map` file has to be created as shown:

m1	pin	8	1
m2	pin	10	1
m3	pin	12	1
m4	pin	14	1

When the export process is performed for the design `snps_mpl1/msip_dcc/layout`, the layer map file has to be specified in the **Export To ICC** dialog box under the **Options** tab, as shown:



The tool uses the specified layer mapping file and creates a new dataType `pin_cd####` on the specified numbers. Layer definitions are then created for each metal layer, which has the pin purpose, where 'pin' is the name of the Custom Compiler purpose.

The dumped IC Compiler technology file looks like the following:

```
Layer "m1" {
  layerNumber = 8
  maskName = "metal1"
  .....
}
LayerDataType "pin_cd011" {
  layerNumber = 8
  dataTypeNumber = 1
  .....
}
```

Import From the IC Compiler Tool

In this scenario, there is an IC Compiler design library. There is no Custom Compiler technology file or Custom Compiler library.

For the given IC Compiler library, your requirement could be either one of the options that follow:

- Generate a Custom Compiler technology file [OR]
- Generate a Custom Compiler technology file and library.

This topic covers the different scenarios for importing an IC Compiler technology database to create a new Custom Compiler database or to update an existing one. The importing scenarios are:

- [Scenario 1: Without Available Technology or Design Data](#)
- [Scenario 2: Into an Existing Library With Matching Layers](#)
- [Scenario 3: Into an Existing Library With Different Layer Names](#)
- [Scenario 4: Into an Existing Library With No Matching Layers](#)
- [Scenario 5: Into an Existing Library With Read-Only Technology](#)

Scenario 1: Without Available Technology or Design Data

If you do not have Custom Compiler technology file or Custom Compiler design data available, you can generate them when importing the IC Compiler design.

This scenario describes the following:

- [Generate a Custom Compiler Technology File](#)
- [Generate a Custom Compiler Technology File and Library](#)

Generate a Custom Compiler Technology File

Use the `icc2cd_tech` command to generate Custom Compiler technology file and library.

You can run the command outside the Custom Compiler environment, in a Linux window, which allows you to quickly create a Custom Compiler technology file.

The following is an example that exports a Custom Compiler technology file from an IC Compiler Design library:

```
% icc2cd_tech -mwLib opdecoder_icc -techFile custom_compiler.tf
```

Note:

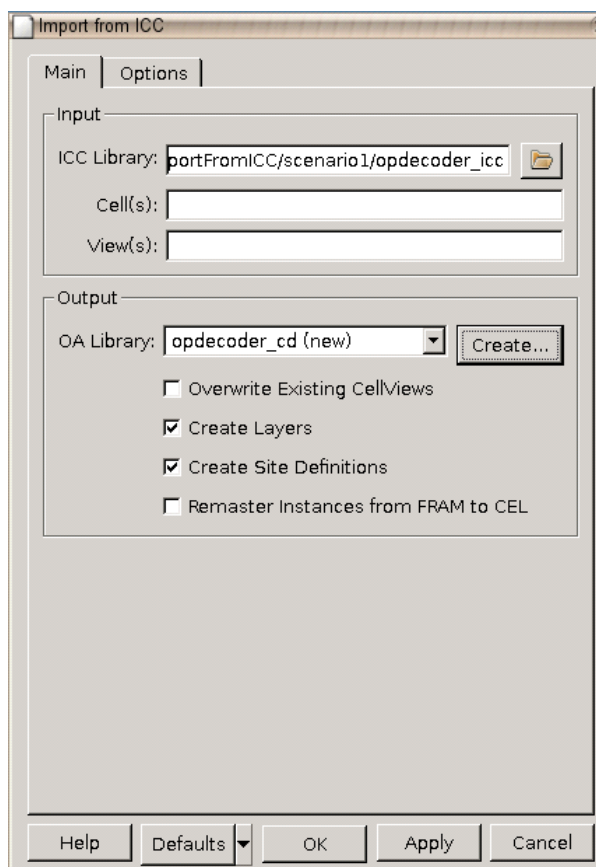
The IC Compiler library should have the technology information to generate the Custom Compiler technology file.

Generate a Custom Compiler Technology File and Library

Use the **Import from ICC** command to create a Custom Compiler technology file and a library for a given IC Compiler library.

You can choose **File > Import > From ICC** from the Console, which allows you to import the IC Compiler library to a Custom Compiler library.

In the **Import from ICC** dialog box, you can specify the IC Compiler library in the **Input** section. This IC Compiler library is imported to a Custom Compiler library, which you specify in the **Output** section under **OA Library**.



During the import process, the tool creates the `iccbriidge` directory, which has all of the output files generated.

The Custom Compiler technology file can be identified by the name `icc2cd_tech.tf`.

The Custom Compiler library is created and placed in the `oaLibs` directory.

Scenario 2: Into an Existing Library With Matching Layers

In this scenario, there is an IC Compiler design library database. There is also a Custom Compiler library available for import.

Caution:

The Custom Compiler technology database matches with IC Compiler technology in all respects.

During the import process:

- The tool checks the IC Compiler layer names against the Custom Compiler technology layer names. If the layer names match, no layers are created, as the layer mapping is done by name.
- Similarly for the `dataType`, if a purpose name already exists in the destination Custom Compiler technology file, no creation is necessary.
- If the IC Compiler design contains metal fill shapes that have data type 0, the Custom Compiler co-design assistant automatically creates a fill purpose to distinguish the metal fill shapes from routing shapes.

There is no special setup required for this case.

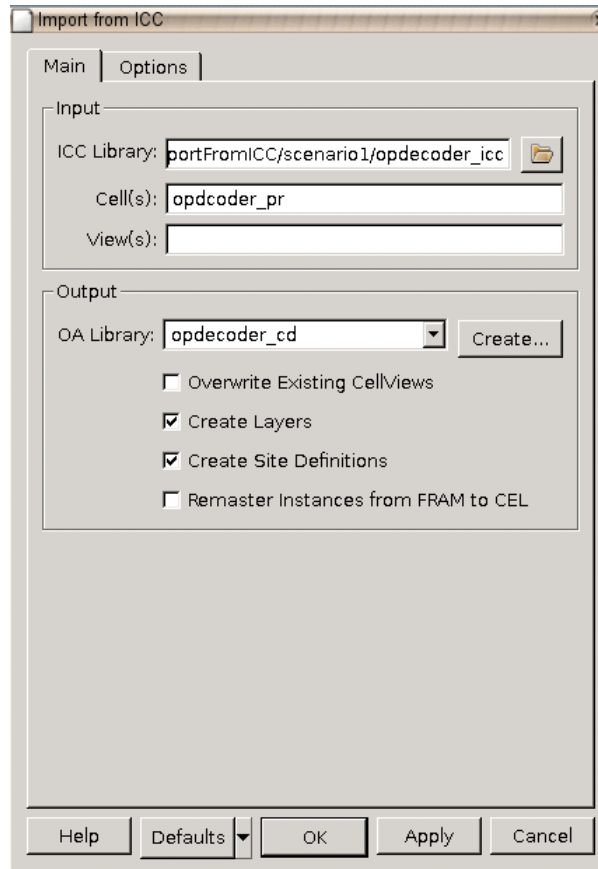
Example:

The IC Compiler library, `opdecoder_icc` has the `metal1` layer (identified by the mask name "metal1") with layer name "m1" on layer number 8 with `dataType` 0.

The Custom Compiler design library, `opdecoder_cd` with the Custom Compiler tech database has the LPP "m1 drawing" on the LPP - 8:-1.

Similarly, the other layers also match.

When the IC Compiler design `opdecoder_icc/opdecoder.CEL` is imported to the Custom Compiler library `opdecoder_cd`, the technology layer match is resolved by layer names.



A layer mapping file is generated during the process. The layer mapping file is created and placed under the `iccbridge` directory. The file is called `layer.map`.

For the given example, the `layer.map` looks like the following:

CDLayerName	CDLayerPurpose	ICCLayerNumber	ICCDatatypeNumber
m1	drawing	8	0

Scenario 3: Into an Existing Library With Different Layer Names

In this scenario, there is an IC Compiler library. There is also a Custom Compiler library with a technology database available for import.

Caution:

In this case the layer names are mismatched between the IC Compiler and Custom Compiler technology.

During the import process, for the IC Compiler layers on dataType 0 (which does not have a Custom Compiler layer with same name), a new layer would be created in the Custom Compiler technology database.

For the IC Compiler layers not on dataType 0, a new layer and purpose would be created if no other matching layer/purpose exists.

To avoid creating new layers/purposes during the import process, the required setup process would be to create a `layer.map` file to resolve the layer name mismatch.

Example:

The IC Compiler library `opdecoder_icc` has metal layers on M# (where #=1,, 10).

There are M#:M#DMY layer dataTypes that do not have an equivalent Custom Compiler purpose.

The Custom Compiler library has metal layers on m# (where #=1,,10).

IC Compiler Layers	Custom Compiler Layers
M1 (8) : dataType 0	m1 drawing
M1(8): M1DMY(1)	???

As shown in the table, all metal layers on dataType 0 map to the m# drawing LPPs on the Custom Compiler side.

For the IC Compiler layers M#:M#DMY, there is no equivalent purpose on the Custom Compiler side.

When the design `opdecoder_icc/opdecoder_pr.CEL` is imported into the Custom Compiler library `opdecoder_cd`, using the default mapping (that is, where no `layer.map` file is provided), then a new LPP "M1:M1DMY " would be created for IC Compiler layer M1:M1DMY.

On opening the `opdecoder_cd/opdecoder/layout`, the following messages are printed:

```
Information: create a new lpp(M1/drawing) with layer number 10019 and
purpose number -1 in tech lib opdecoder_cd. (TECH-020)Information: create
a new lpp(M1/M1DMY) with layer number 10019 and purpose number 1 in tech
lib opdecoder_cd. (TECH-020)
```

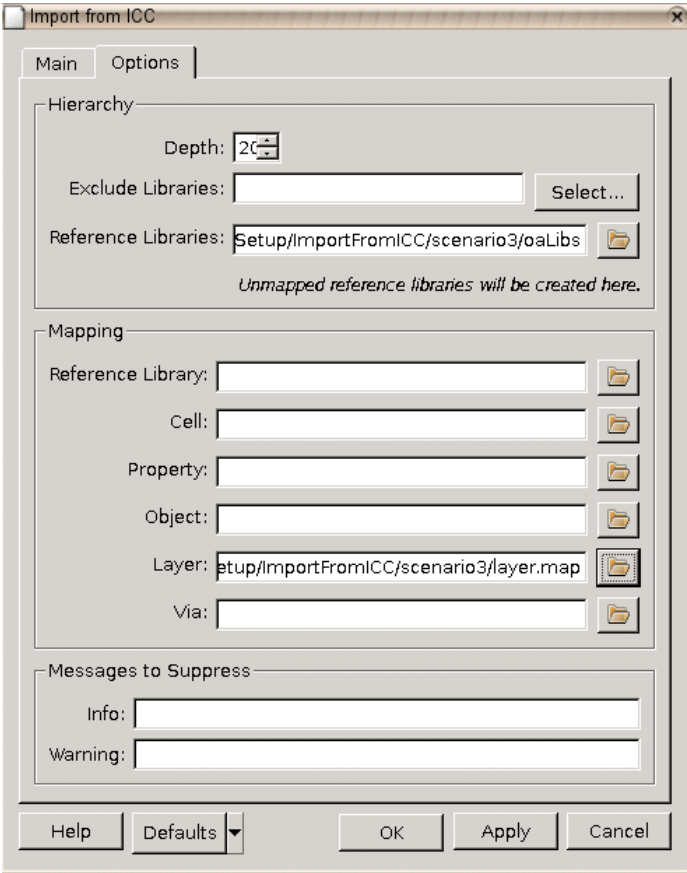
To avoid creating a new LPP and using the existing layers, a `layer.map` file should be created.

The `layer.map` file must be used to resolve the mismatch, as shown:

CDLayerName	CDLayerPurpose	ICCLayerNumber	ICCDatatypeNumber
m1	drawing	8	0

m1	M1DMY	8	1
m2	drawing	10	0
m2	M2DMY	10	1

The `layer.map` file has to be specified in the **Import From ICC** dialog box under the Mapping Tab as shown:



The tool uses the specified layer mapping file and creates a new purpose M#DMY for each metal layer.

Scenario 4: Into an Existing Library With No Matching Layers

In this scenario, there is an IC Compiler library. There is also a Custom Compiler library with a technology database that is available for import.

Caution:

In this case, there are some IC Compiler layer dataType elements that do not have an equivalent matching LPP in the Custom Compiler environment.

There are two methods for creating the missing layers/purposes in target Custom Compiler technology:

- [Using the Automated Import From the IC Compiler Tool](#)
- [Using the Layer Mapping File](#)

A third method involves creating a new Technology File and editing the existing IC Compiler technology file to include the missing layers and purposes.

Using the Automated Import From the IC Compiler Tool

During the import process, the tool checks the IC Compiler layer names in the tech database against the Custom Compiler layer names.

- If the layer number is not already in use in the destination technology file, a new layer is created in the destination technology file with the source layer name and number.
- If the layer number is in use, a new layer is created in the destination technology file with the source layer name and an unused layer number.

The same process is applied for dataTypes, also.

- If the dataType number is not already in use in the destination Custom Compiler technology file, a new purpose is then created in the destination technology file with the source dataType name and number. Otherwise, a new purpose created in the destination technology file with the source dataType name and an unused dataType number.

Example:

For the IC Compiler design library, `opdecoder_icc`, the design database has

- shapes on the Layer: dataType m#: dataType0 , where #=1,2,3 and 4
- dummy layers on the Layer: dataType m#: m#dummy, where #=1,2,3,4

There is already a Custom Compiler library `opdecoder_cd`, which contains the equivalent metal layer definitions for layer: dataType m#: dataType, but no purpose definitions for the dataType m#dummy.

IC Compiler Layers	Custom Compiler Layers
m1 (8) : dataType 0	m1 (8) :drawing (-1)
m1(8):m1dummy(1)	m1(8): ???
m2(10):dataType 0	m2(10) :drawing(-1)
m2(10):m2dummy(1)	m2(10):???

IC Compiler Layers	Custom Compiler Layers
SCRB(88): dataType 0	???

When the IC Compiler design `opdecoder_icc/opdecoder.CEL` is imported into the Custom Compiler library `opdecoder_cd`, the tech layer matching is resolved by the layer names.

On design open in Custom Compiler, new LPPs are created for the missing layers and purposes and the messages below are output:

Information: create a new lpp(m1/mldummy) with layer number 8 and purpose number 10 in tech lib `opdecoder_cd`. (TECH-020)Information: create a new lpp(SCRB/drawing) with layer number 88 and purpose number -1 in tech lib `opdecoder_cd`. (TECH-020)

The generated layer map file looks as follows:

m1	drawing	8	0
m1	mldummy	8	10
SCRB	drawing	88	0

The tech database is accordingly updated with the new LPPs as follows:

```
techPurposes (
; ( purposeName purposeNum abbreviation )
; ( ----- )
  ( mldummy      10              )
) ;techPurposes
techLayers (
  ( SCRB          88              )
) ;techLayers
techLayerPurposePriorities (
  ( m1            mldummy        )
  ( SCRB          drawing        )
...
) ;techLayerPurposePriorities
techDisplays (
  ( m1            mldummy        m1_mldummy      t t nil nil
t )
  ( SCRB          drawing        SCRB_drawing    t t nil nil
t )
)
```

Using the Layer Mapping File

If you want to map the missing layers in the IC Compiler technology database to a specific layer number and purpose, you can provide an explicit mapping using the `layer.map` file.If the destination layer number/purpose number provided in the `layer.map` file does not exist, then it is created.

Example:

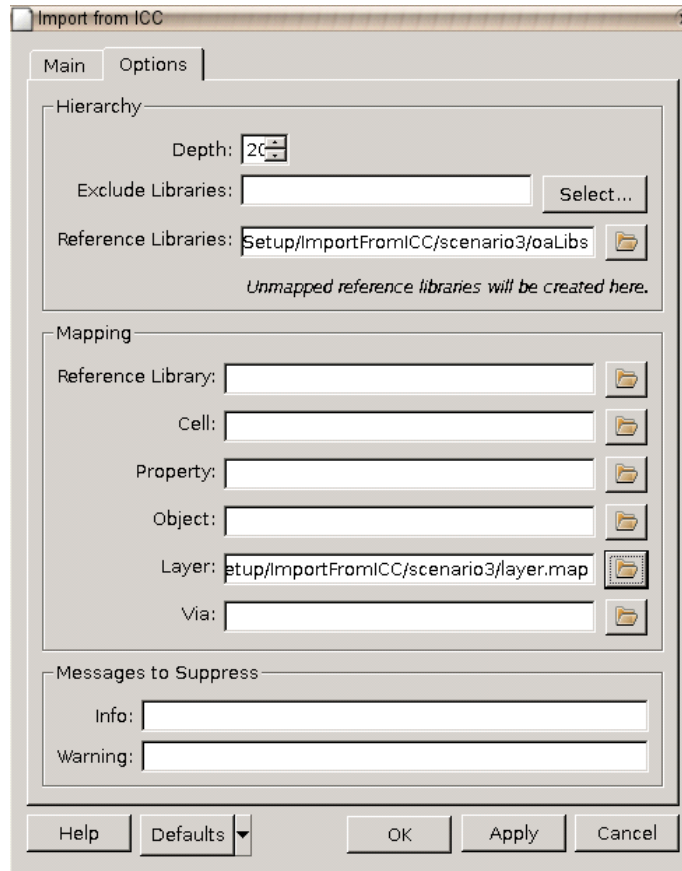
For the example described, assume that your requirement is to map the missing layers as shown in the following table:

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	m1 (8) : dataType 0
m1(8): <i>dmy1</i> (1)	m1(8):m1dummy(1)
m2(10) :drawing(-1)	m2(10):dataType 0
m2(10): <i>dmy1</i> (1)	m2(10):m2dummy(1)
<i>SCRB</i> (88): <i>drawing</i>	SCRB(88): dataType 0

The `layer.map` file has to be created as shown below:

m1	dmy1	8	10
m2	dmy1	10	10
.....			
SCRB	drawing	88	0

The `layer.map` file has to be specified in the **Import From ICC** dialog box under the **Options** tab as shown:



Scenario 5: Into an Existing Library With Read-Only Technology

In this scenario, there is an IC Compiler library. There is also a Custom Compiler library with a technology database available for import.

In this case, there are some IC Compiler layer:dataType pairs that do not have a corresponding Custom Compiler LPP and the Custom Compiler technology database is read-only.

The setup process required is to create an incremental technology file.

Example:

For the IC Compiler design library, `opdecoder_icc`, the tech database and design database have:

- shapes on the Layer:dataType m#:dataType0 , where #=1,2,3 and 4
- dummy layers on the Layer:dataType m#:m#dummy where #=1,2,3,4

There is already an existing Custom Compiler library `opdecoder_cd` which contains the corresponding metal layer definitions for layer: dataType m#: dataType, but no purpose definitions for the dataType m#dummy.

Table 22

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	m1 (8) : dataType 0
m1(8): ???	m1(8):m1dummy(1)
m2(10) :drawing(-1)	m2(10):dataType 0
m2(10):???	m2(10):m2dummy(1)
???	SCRB(88): dataType 0

When the IC Compiler design `opdecoder_icc/opdecoder.CEL` is imported into the Custom Compiler library `opdecoder_cd_techreadonly`, the import fails with the following message because there is no write permission to the Custom Compiler technology database.

```
Error: Caught base exception: Error: Cannot open OpenAccess library
'opdecoder_cd_techreadonly' with 'write' mode (MWBRIDGE-008)
(MWBRIDGE-051)
```

An incremental technology file can be created to incrementally assemble technology information by creating references from one oaTech database to another oaTech database.

In this case, an incremental technology file has to be created using the `opdecoder_cd_techreadonly` as the base technology reference.

The incremental technology file would look as follows.

```
controls (
  techArray(opdecoder_cd_techreadonly)
) ;controls

layerDefinitions (
```

Appendix A: Setting the User Environment

Import From the IC Compiler Tool

```

techLayers (
; ( LayerName                Layer#                Abbreviation )
; ( -----                -----                ----- )
; User-Defined Layers:
( SCRB                        88                        scrb      )
) ;techLayers
techPurposes (
( mldummy                    10                        )
) ;techPurposes

techLayerPurposePriorities (
; layers are ordered from lowest to highest priority
; ( LayerName                Purpose                )
; ( -----                -----                )
( m1                        mldummy                )
( SCRB                      drawing                )
) ;techLayerPurposePriorities
techDisplays (
( m1                        mldummy                designFlow1      t t nil
nil t )
( SCRB                      drawing                designFlow2      t t nil
nil t )
) ;techDisplays

```

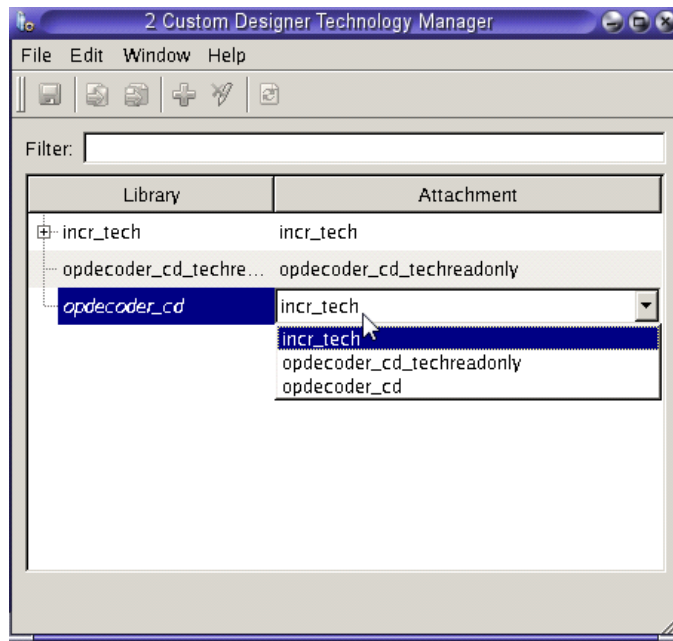
As shown above, the incremental technology file contains only the missing purpose and layers.

The display attributes are chosen to be the default system-defined display attributes and can be modified by you.

The next steps are:

1. Create a new `incr_tech` library using the above technology file `incr_tech.tf`.
2. Create a new top-design library `opdecoder_cd`.

3. Using the Technology Manager, set `incr_tech` library as tech reference for `opdecoder_cd` library:



See Technology Manager Toolin the *Custom Compiler Environment User Guide* for more detailed information on using the Technology Manager tool.

4. Import the design `opdecoder_icc/opdecoder_pr.CEL` into the library `opdecoder_cd` and it should be successful.

Note:

If there is layer name mismatch between the IC Compiler and Custom Compiler technology, use a layer mapping file during the import process as explained in [Scenario 3: Into an Existing Library With Different Layer Names](#).

Pre-Route Flow

This section describes aspects of the pre-route flow in the following sections:

- [Creating Guard Ring Definitions for an IC Compiler Design](#)
- [Using Separate OA Technology for an Opened IC Compiler Library](#)

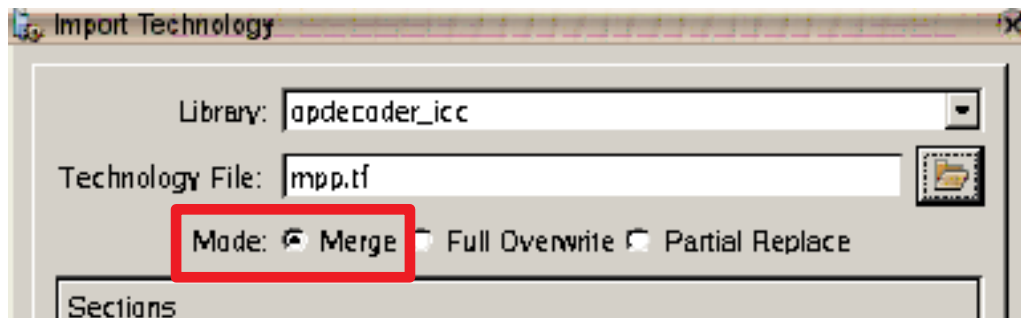
Creating Guard Ring Definitions for an IC Compiler Design

This section covers the method for creating Multi Part Path (MPP) guard rings on an IC Compiler design opened in the Custom Compiler environment through the Milkyway plugin.

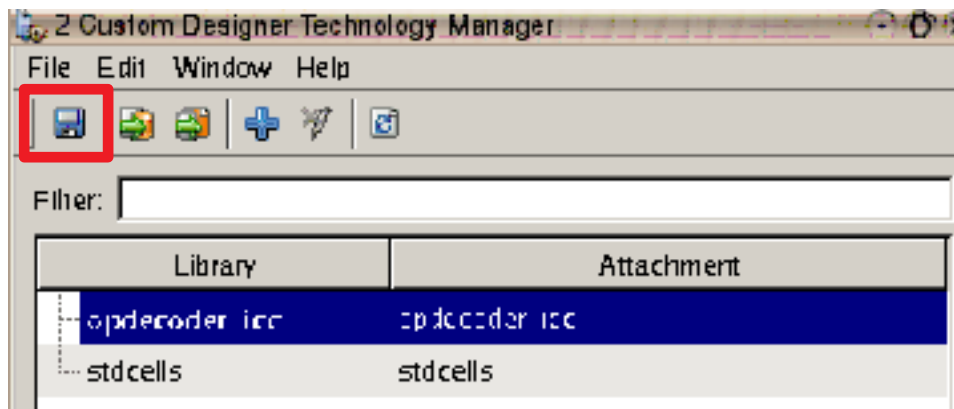
Example 11

The following steps describe the method you can use to create MPPs in an IC Compiler design. Note that this example applies to adding layer definitions; alternatively, a Custom Compiler technology library can also be used as a technology reference.

1. Create a custom technology file with MPP definitions.
2. Include any MPP layers that are not defined in the IC Compiler library.
3. Import the technology file in the IC Compiler library using Merge mode.



4. Save the IC Compiler library technology after import.



Using Separate OA Technology for an Opened IC Compiler Library

This section covers using an OpenAccess (OA) technology file for an IC Compiler design opened in Custom Compiler through the Milkyway plug-in.

To use a Milkyway library in the context of a technology defining Custom Compiler, the required setup is:

- Use an OpenAccess technology attachment, which disables technology synchronization; and
- (Optional) Use mapping files if necessary to reconcile differences between layer and via names defined in OpenAccess versus those defined in Milkyway.

Note:

Using a separate OA technology requires write access to the IC Compiler library.

The setup is explained in the following example.

Example 12

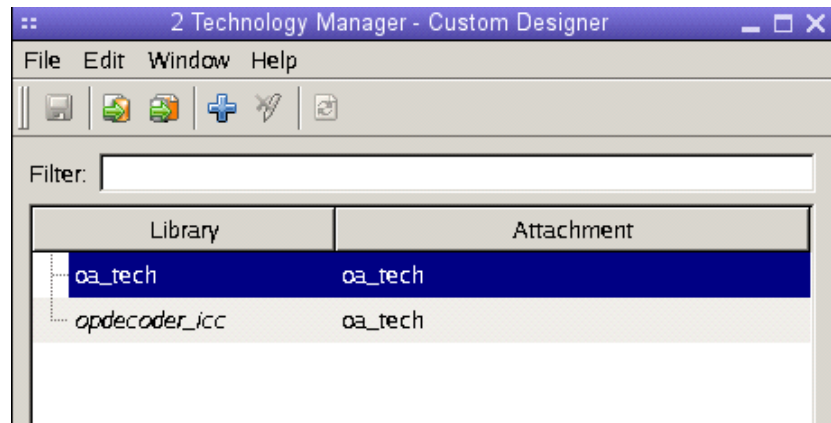
In this example, the layer names are mismatched between the Custom Compiler and IC Compiler technology databases. The requirement is to view the IC Compiler design in the context of the OpenAccess technology file.

Table 23

Custom Compiler Layers	IC Compiler Layers
m1 (8) :drawing (-1)	M1 (8) : dataType 0
m2(10):drawing (-1)	M2(10): dataType 0
.....	
m10(85):drawing(-1)	M10(85):dataType 0

The required steps are:

- Set this `oa_tech` library as tech reference for `opdecoder_icc`.



The `layer.map` file used to resolve the mismatch is shown below:

```
CDLayerName      CDLayerPurpose  ICCLayerNumber  ICLayerDatatypeNumberm1
drawing          8                0                0                m2
drawing          10               0                m3                m4
drawing          12               0                m4                m5
drawing          14               0                m5                m6
drawing          70               0                m6                m7
drawing          72               0                m7                m8
drawing          74               0                m8                m9
drawing          76               0                m9                m10
drawing          78               0                m10
drawing          85               0
Disable the synchronization using the dm::setICCTechSync command:
set lib [dm::getLibs opdecoder icc]
dm::setICCTechSync $lib -layerMap layer.map -sync false
```

Setup Using LEF Files

This section explains how to

- Create an IC Compiler library from the LEF files description available for the technology and standard cells
- Create an OA library from the LEF files description available for the technology and standard cells

Scenario 1: Importing LEF Files to Create an IC Compiler Library

The Milkyway tool can be used to read the LEF file to create IC Compiler technology file and IC Compiler libraries. See [Library Preparation Using LEF/DEF](#) in the IC Compiler user documentation.

Example 13

In this example, there are two LEF files available:

- `cells.lef`, which contains the descriptions for the standard cells.
- `tech.lef`, which contains the technology information.

In Milkyway, use the `read_lef` command to create an IC Compiler technology file and library from the above LEF files:

1. Invoke Milkyway in Tcl mode:

```
% Milkyway -tcl &
```

2. In the Milkyway window, run the command to create an IC Compiler library stdcells:

```
read_lef -lib_name stdcells -tech_lef_files tech.lef -cell_lef_files  
cells.lef
```

3. Dump the IC Compiler technology file for the above stdcells library by choosing **Tech File > Write to File**.

Scenario 2: Importing LEF files to create a Custom Compiler library

To create Custom Compiler libraries, the `lef2oa` standalone translator can be used to import the LEF files.

Example 14

In this example, there are two LEF files available:

- `cells.lef`, which contains the descriptions for the standard cells.
- `tech.lef`, which contains the technology information.

To create a Custom Compiler technology file and library from the above LEF files, use the `lef2oa` standalone command.

To import the LEF, a layer mapping file is required to map LEF objects found on the specified `layerName` to the specified `layerNumber`.

If a layer mapping file is not specified, the layer names are auto-generated.

The following is an example of a layer mapping file to use:

LayerName	Layer Number
poly	7
m1	9
m2	11

m3	13
m4	15
m5	17
m6	19
m7	21
m8	23
m9	25
m10	27

The next steps are:

1. In the Linux window, type the following command to create an OA library `stdcells_oa` and a tech library `tech_lib`:

```
lef2oa -lib stdcells_oa -lef tech.lef -layerMap layer.map -techLib  
tech_lib
```

The above command creates a new cell library called `stdcells_oa`. The option `techLib` specifies the name of a technology library to use when creating a new cell library. If the specified technology library does not exist, the library is created using the technology information in the LEF file.

2. In the Linux window, type the following command to create an OA library `stdcells_oa` and a tech library `tech_lib`.

```
lef2oa -lib stdcells_oa -lef cells.lef -layerMap layer.map -techLib  
tech_lib
```

The above command imports the cell information from the `cells.lef`. During import, it uses the technology information from `tech_lib` created in [Step 1](#).

B

Basic Troubleshooting

This appendix describes some common error and warning messages, identifies probable causes, and provides possible solutions.

This chapter includes the following topics:

- [Error Messages](#)
- [Warning Messages](#)

Error Messages

A Custom Compiler error message is the most obvious clue that an unexpected condition has occurred when creating and editing IC Compiler and Custom Compiler co-designs. Unlike warning messages, the Custom Compiler tool will not complete processing until the necessary changes are made. Because one error can result in multiple error messages, fixing the first reported error often clears up the remaining error messages.

The following sections describe common error messages when creating and editing IC Compiler and Custom Compiler co-designs.

- [E MWBRIDGE-002](#)
- [E MWBRIDGE-011](#)
- [E MWBRIDGE-050](#)
- [E MWBRIDGE-051](#)
- [E MWBRIDGE-060](#)
- [E MWBRIDGE-101](#)
- [E MWBRIDGE-171](#)
- [E MWBRIDGE-209](#)
- [E MWBRIDGE-225](#)
- [E MWBRIDGE-226](#)

- [E MWBRIDGE-247](#)
- [E MWBRIDGE-306](#)
- [E MWBRIDGE-369](#)
- [E MWBRIDGE-371](#)
- [E MWBRIDGE-397](#)

E MWBRIDGE-002

```
Error: Cannot create Milkyway library at path '<path>/test2', reason:  
Error: Failed to attach technology 'mw.tfl': check the technology file  
(MWBRIDGE-060) (MWBRIDGE-002)
```

Cause

Library creation in the current session has failed for the library you specified.

Solution

Check that the technology file provided is correct and the path has write permission.

See also

[E MWBRIDGE-060 on page 170](#)

E MWBRIDGE-011

```
Error: Cannot create 'test' cell in  
'/remote/amscae18/CRM/F-2011.09-SP3/9000536666/iccLibs/testLib' library  
(MWBRIDGE-011)
```

Cause

The specified cell has not been created in the design because of the lack of permission to access or create the file, or you tried to create the file in a directory for which you do not have write permission.

Solution

Check the cell name and the view name to ensure they are valid. Check the permission of the directory.

E MWBRIDGE-050

```
Error: Caught OA exception: Invalid string index. Skipping translation of  
test.CEL cellView (MWBRIDGE-050)
```

Cause

An internal exception occurred.

Solution

Report this error to your Synopsys representative.

E MWBRIDGE-051

Error: Caught OA exception: Indexed names are not valid (MWBRIDGE-051)

Cause

An internal exception occurred.

In the example above, the error indicates that there are indexed instance names in the design. For example : x4<0>. Indexed names of this format are not supported.

Solution

For the example above, to run the export successfully, change the indexed instance names from angle brackets <> to parentheses ().

E MWBRIDGE-060

Error: Cannot create Milkyway library at path '<path>/test2', reason:
Error: Failed to attach technology 'mw.tfl': check the technology file
(MWBRIDGE-060) (MWBRIDGE-002)

Cause

The technology file cannot be imported during library creation for the library you specified to create in the current session.

Solution

Check that the technology file provided is correct and the path has write permission.

See also

[E MWBRIDGE-002 on page 169](#)

E MWBRIDGE-101

Error: Syntax error in file 'demo_snps/layer.map' at line '2' :
' ' (MWBRIDGE-101)

Cause

A syntax error is found in the specified mapping file.

Solution

Check to make sure the syntax of the mapping file is correct. If not, correct the mapping file.

For the correct syntax for mapping files, see [IC Compiler - Custom Compiler Library Mapping Files on page 44](#).

E MWBRIDGE-171

Error: The technology for library testOA is not writable - technology changes will not be saved. (MWBRIDGE-171)

Cause

The IC Compiler CEL is imported into a Custom Compiler library and it is attempting to modify the read-only technology database.

For example, if the technology library portion of the Milkyway technology data includes layer x, and layer x is missing from the existing OpenAccess technology library but that library only has read permission, then the translator cannot update it with layer x.

Solution

Use the technology reference approach to create a writable technology library. Change the read-only technology library attachment to a technology reference to the top OA library and re-run the **Import** process. Any new layers or technology-specific information will now be created in the writable incremental technology library.

For details, see SolvNetPlus article #000005222, "[Importing an IC Compiler Design into a Custom Compiler Library That Has a Read-Only Technology Database.](#)"

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

See also

[E MWBRIDGE-171 on page 171](#)

E MWBRIDGE-209

Error: The technology library portion of the MW technology data includes Layer 'SCRB' that is missing from the existing OA technology library at <path>/techlib but that library only has read permission and therefore the translator cannot update it (MWBRIDGE-209)

Cause

The IC Compiler CEL is imported into a Custom Compiler library and it is attempting to modify the read-only technology database.

For example, if the technology library portion of the Milkyway technology data includes layer x, and layer x is missing from the existing OpenAccess technology library but that library only has read permission, then the translator cannot update it with layer x.

Solution

Use the technology reference approach to create a writable technology library. Change the read-only technology library attachment to a technology reference to the top OA library and re-run the **Import** process. Any new layers or technology-specific information will now be created in the writable incremental technology library.

For details, refer to SolvNetPlus article #000005222, ["Importing an IC Compiler Design into a Custom Compiler Library That Has a Read-Only Technology Database."](#)

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

See also

[E MWBRIDGE-171 on page 171](#)

E MWBRIDGE-225

Error: Cannot import MW library 'test' with length precision of 1000 to OA library 'test_oa' with a DBU/UU of 300 because length precision and DBU/UU are not integral multiples of one another and thus the design might not be integrally scaled between them. (MWBRIDGE-225)

Cause

The manufacturing grid resolutions can vary between the Custom Compiler and IC Compiler environments when the technology database headers for the two do not originate from one another. OpenAccess expresses the manufacturing grid resolution in user units, whereas Milkyway expresses the grid resolution in database units (DBUs).

To determine the relative scaling between the two technology headers, the OpenAccess default manufacturing grid is converted to DBUs by multiplying it by the DBUPerUU ratio of the OpenAccess technology header. The resultant DBU grid resolution of OpenAccess is compared to the Milkyway grid resolution to determine whether or not one is an integral multiple of the other. If the ratio is not integral or either grid value is zero, the translation returns the above error.

When the two resolutions are integral multiples of one another, during translation of each object with coordinates or distances the translation scales each value according to the manufacturing grid resolution ratio between the two databases, in order to give adjusted DBUs. The translation scales all DBU values according to this ratio between the two databases so that their corresponding user units' values correctly match across the two database representations.

For example, if the Custom Compiler manufacturing grid resolution is 0.005 user units and the DBUPerUU is 10,000 then the Custom Compiler grid resolution is 50 DBUs. If the IC Compiler manufacturing grid resolution is 5 DBUs and the length precision is 1,000, then the ratio between the two grid resolutions is $50 \div 5$, or 10. Therefore, translation divides each Custom Compiler object's DBU value by 10 to maintain constant user units between the two database representations. A rectangle that is 0.5×0.5 in user units in size would be stored in the Custom Compiler environment as a $5,000 \times 5,000$ DBU rectangle; but in IC Compiler, it would be stored as a 500×500 DBU rectangle to maintain a consistent 0.5×0.5 user units value.

Solution

Check both the Milkyway and Custom Compiler technology files and adjust the resolution of one of the technology files so that the design can be exported successfully.

E MWBRIDGE-226

Error: Cannot export OA library 'test_oa' with DBU/UU of 300 to MW library 'test_exp' with a length precision of 2000 because length precision and DBU/UU are not integral multiples of one another and thus the design may not be integrally scaled between them. (MWBRIDGE-226)

Cause

The manufacturing grid resolutions can vary between the Custom Compiler and IC Compiler environments when the technology database headers for the two do not originate from one another. OpenAccess expresses the manufacturing grid resolution in user units, whereas Milkyway expresses the grid resolution in database units (DBUs). To determine the relative scaling between the two technology headers, the OpenAccess default manufacturing grid is converted to DBUs by multiplying it by the DBUPerUU ratio of the OpenAccess technology header. The resultant DBU grid resolution of OpenAccess is compared to the Milkyway grid resolution to determine whether or not one is an integral

multiple of the other. If the ratio is not integral or either grid value is zero, the translation returns the above error.

When the two resolutions are integral multiples of one another, during translation of each object with coordinates or distances the translation scales each value according to the manufacturing grid resolution ratio between the two databases, in order to give adjusted DBUs. The translation scales all DBU values according to this ratio between the two databases so that their corresponding user units' values correctly match across the two database representations.

For example, if the Custom Compiler manufacturing grid resolution is 0.005 user units and the DBUPerUU is 10,000, then the Custom Compiler grid resolution is 50 DBUs. If the IC Compiler manufacturing grid resolution is 5 DBUs and the length precision is 1,000, then the ratio between the two grid resolutions is $50 \div 5$, or 10. Therefore, translation divides each Custom Compiler object's DBU value by 10 to maintain constant user units between the two database representations. A rectangle that is 0.5×0.5 in user units in size would be stored in the Custom Compiler environment as a $5,000 \times 5,000$ DBU rectangle; but in IC Compiler, it would be stored as a 500×500 DBU rectangle to maintain a consistent 0.5×0.5 user units value.

Solution

Check both the Milkyway and Custom Compiler technology files and adjust the resolution of one of the technology files so that the design can be exported successfully.

E MWBRIDGE-247

```
Error: Caught mwbridge exception: Error: Cannot import MW cell 'PCI_TOP'
because it is stored in schema version 5. Instead, either open the
design for write or use the convert_mw_lib command in ICC to upgrade the
design's schema version. (MWBRIDGE-051) (MWBRIDGE-247)
```

Cause

The IC Compiler cell specified for import is created in a Milkyway schema version that is not supported by the tool.

Solution

To upgrade the Milkyway schema version, open the IC Compiler design in an IC Compiler environment for writing and save the design. Or use the `convert_mw_lib` command in IC Compiler.

See also

[E MWBRIDGE-051 on page 170](#)

E MWBRIDGE-306

```
Error: Failed to import translated technology for library
_cache_oaTech_testICC2: Tech parser error: 19 syntax error in
file /
remote/amscae30/custom_compiler/cd-icc-flowtestcase/releases/I-2013.12/90
00658233/iccbridge/cache/I-2013.12-BETA/oaTech_testICC2/tech.tf, line
number: 399 . (MWBRIDGE-306)
```

Cause

The IC Compiler library cannot be opened because of a syntax error in the Custom Compiler technology file created from the IC Compiler library technology.

Solution

Check the Custom Compiler technology file for syntax errors. If the error is due to incorrect syntax, report this error to your Synopsys representative.

E MWBRIDGE-369

```
Error: Skipping translation of 'forCD.CEL' cellView. Unable to retrieve
Milkyway Striped wire info. (MWBRIDGE-369)
```

Cause

The specified IC Compiler CEL cannot be saved or exported to IC Compiler library. The error message gives the reason for the error followed by an explanation of the failure. There could be many reasons for the error: for example, if a Custom Compiler-specific object was modified in IC Compiler, then the object cannot be recreated in the Custom Compiler environment as a Custom Compiler-specific object.

Solution

If the error is similar to the example issue above, delete the object and recreate it in the Custom Compiler environment. If the object was not modified in IC Compiler, report this error to your Synopsys representative.

E MWBRIDGE-371

```
Error: Failed to export OA Rectangle because (3, 3) point of PointArray
cannot be scaled between the OA resolution of 2000 and the MW resolution
of 1000. (MWBRIDGE-371)
```

Cause

The manufacturing grid resolutions can vary between the Custom Compiler and IC Compiler environments when the technology database headers for the two do not

originate from one another. OpenAccess expresses the manufacturing grid resolution in user units, whereas Milkyway expresses the grid resolution in database units (DBUs). To determine the relative scaling between the two technology headers, the OpenAccess default manufacturing grid is converted to DBUs by multiplying it by the DBUPerUU ratio of the OpenAccess technology header. The resultant DBU grid resolution of OpenAccess is compared to the Milkyway grid resolution to determine whether or not one is an integral multiple of the other. If the ratio is not integral or either grid value is zero, the translation returns the above error.

When the two resolutions are integral multiples of one another, during translation of each object with coordinates or distances the translation scales each value according to the manufacturing grid resolution ratio between the two databases, in order to give adjusted DBUs. The translation scales all DBU values according to this ratio between the two databases so that their corresponding user units' values correctly match across the two database representations.

For example, if the Custom Compiler manufacturing grid resolution is 0.005 user units and the DBUPerUU is 10,000, then the Custom Compiler grid resolution is 50 DBUs. If the IC Compiler manufacturing grid resolution is 5 DBUs and the length precision is 1,000, then the ratio between the two grid resolutions is $50 \div 5$, or 10. Therefore, translation divides each Custom Compiler object's DBU value by 10 to maintain constant user units between the two database representations. A rectangle that is 0.5×0.5 in user units in size would be stored in the Custom Compiler environment as a $5,000 \times 5,000$ DBU rectangle; but in IC Compiler, it would be stored as a 500×500 DBU rectangle to maintain a consistent 0.5×0.5 user units value.

Solution

Check both the Milkyway and Custom Compiler technology files and adjust the resolution of one of the technology files so that the design can be exported successfully.

E MWBRIDGE-397

```
Error: Caught mwbridge exception: Error: Could not import MW Rectangle
with ID 1929258 because net with ID 152405 is assigned to it, which does
not exist in the MW database. (MWBRIDGE-397).
```

Cause

Rectangles in the IC Compiler design are associated with a net that no longer exists.

Solution

To open the design successfully in the Custom Compiler environment, you need to remove the shapes that have the invalid net ID.

In IC Compiler, open the design, then delete the rectangles with invalid net IDs using the following Tcl commands:

```
set shapes [get_net_shapes -filter {net_type == "illegal object type"}]
remove_net_shape $shapes
```

Save the IC Compiler design, then reopen the design in the Custom Compiler environment.

Warning Messages

Custom Compiler warning messages indicate problems in processing, but do not necessarily stop the processing. Warning messages should always be investigated.

The following sections describe common warning messages when co-designing.

- [W MWBRIDGE-419](#)

W MWBRIDGE-419

Warning: '.pcell_9c3a4df71f8b5eaf9e644b538a42ada1.CEL' specifies 'reference90RF/n_4t/layout' as its corresponding OpenAccess superMaster which does not exist. Imported instances of this cell are bound to a copy of the original subMaster. (MWBRIDGE-419)

Cause

When exporting a design with PCells to IC Compiler, information from the PCells is stored as cell properties in IC Compiler. The **Import from ICC** command makes use of the information stored in the cell properties defined to recreate PCell instances. For example, a cell property of type string named OA_PCELL_SUPERMASTER holds the LCV of the PCell supermaster, in the <lib>/<cell>/<view> format. When recreating the PCell, this LCV information is used to bind the PCell to the correct PDK library, to recreate the PCell instead of a fixed layout. While importing the IC Compiler design which has PCells (which was created by **Export to ICC**), if the parent PDK library does not exist in the Custom Compiler environment, then the PCells are imported as fixed layouts or static instances with a warning.